



Programación dinámica

En el capítulo anterior veíamos que es posible dividir los ejemplares en subejemplares, resolver los subejemplares (posiblemente, subdividiéndolos de nuevo) y combinar entonces las soluciones de los subejemplares para resolver el caso original. A veces sucede que la forma natural de dividir un ejemplar, tal como lo sugiere la estructura del problema, nos lleva a considerar subejemplares solapados. Si los resolvemos todos independientemente, éstos darán a su vez lugar a toda una colección de subejemplares idénticos. Si no prestamos atención a esta duplicación, es probable que acabemos por tener un algoritmo ineficiente; si por el contrario aprovechamos la duplicación y nos las arreglamos para resolver cada subejemplar una sola vez, guardando la solución para su uso posterior, entonces tendremos un algoritmo más eficiente. La idea que subyace a la programación dinámica es, por tanto, bastante sencilla: evitar calcular dos veces una misma cosa, normalmente manteniendo una tabla de resultados conocidos que se vaya llenando a medida que se resuelven los subcasos.

El método de divide y vencerás es un método de *refinamiento progresivo*. Cuando se resuelve un problema mediante divide y vencerás, atacamos de inmediato el caso completo, que a continuación dividimos en subcasos más y más pequeños, a medida que progresa el algoritmo. La programación dinámica, por otra parte, es una técnica *ascendente*. Normalmente, se empieza por los subcasos más pequeños, y por tanto más sencillos. Combinando sus soluciones, obtenemos las respuestas para subcasos de tamaños cada vez mayores, hasta que finalmente llegamos a la solución del caso original.

Comenzaremos el capítulo por dos ejemplos sencillos de programación dinámica, que ilustran la técnica general en una situación poco complicada. Las secciones

siguientes abordan los problemas de devolver cambio, que mencionábamos en la Sección 6.1, y el de la mochila, que encontrábamos en la Sección 6.5.

8.1 DOS EJEMPLOS SENCILLOS

8.1.1 Cálculo del coeficiente binomial

Considere el problema consistente en calcular el coeficiente binomial

$$\binom{n}{k} = \begin{cases} 1 & \text{si } k = 0 \text{ o } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{si } 0 < k < n \\ 0 & \text{en caso contrario} \end{cases}$$

Supongamos que $0 \leq k \leq n$. Si calculamos directamente $\binom{n}{k}$ mediante

función $C(n, k)$
 si $k = 0$ **o** $k = n$ **entonces devolver** 1
 sino devolver $C(n-1, k-1) + C(n-1, k)$

entonces muchos de los valores de $C(i, j)$, con $i < n, j < k$ se calculan una y otra vez. Por ejemplo, el algoritmo calcula $C(5,3)$ como la suma de $C(4,2)$ y $C(4,3)$. Ambos resultados intermedios exigen calcular $C(3,2)$. De forma similar, el valor de $C(2,2)$ se utiliza varias veces. Dado que el resultado final se obtiene sumando un cierto número de unos, el tiempo de ejecución de este algoritmo estará con certeza en $\Omega(\binom{n}{k})$. Encontrábamos anteriormente un fenómeno similar, en el algoritmo *Fibrec* para calcular la sucesión de Fibonacci; véase la Sección 2.7.5.

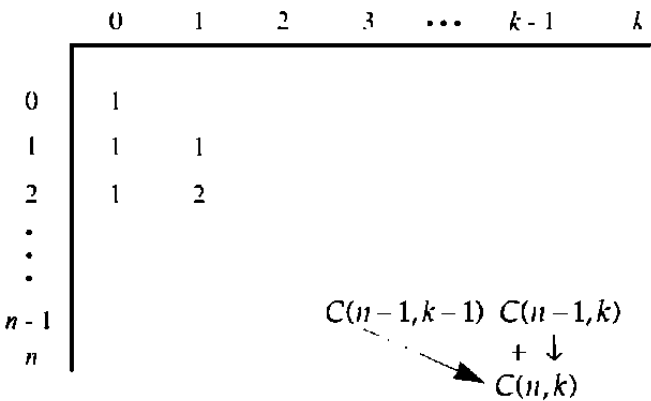


Figura 8.1. El triángulo de Pascal

Si por otra parte utilizamos una tabla de resultados intermedios —que por supuesto se trata del *triángulo de Pascal*—, entonces obtenemos un algoritmo más eficiente; véase la figura 8.1. La tabla debería ir llenándose línea por línea. De hecho, ni siquiera es necesario llenar toda la tabla: basta con mantener un vector k , que

representa la línea actual, y actualizar el vector de derecha a izquierda. Por tanto, para calcular $[i]$ el algoritmo requiere un tiempo que está en $\Theta(nk)$, y un espacio que está en $\Theta(k)$, si suponemos que la suma es una operación elemental.

8.1.2 El campeonato mundial

Considere una competición en la cual hay dos equipos A y B que juegan un máximo de $2n - 1$ partidas, y en donde el ganador es el primer equipo que consiga n victorias. Suponemos que no hay empates, que los resultados de todos los partidos son independientes, y que para cualquier partido dado hay una probabilidad constante p de que el equipo A sea el ganador, y por tanto una probabilidad constante $q = 1 - p$ de que gane el equipo B .

Sea $P(i, j)$ la probabilidad de que el equipo A gane el campeonato, cuando todavía necesita i victorias más para conseguirlo, mientras que el equipo B necesita j victorias más para ganar. Por ejemplo, antes del primer partido del campeonato la probabilidad de que gane el equipo A es $P(n, n)$: ambos equipos necesitan todavía n victorias para ganar el campeonato. Si el equipo A necesita cero victorias más, entonces lo cierto es que ya ha ganado el campeonato, y por tanto $P(0, i) = 1$, con $1 \leq i \leq n$. De manera similar, si el equipo B necesita 0 victorias más, entonces ya ha ganado el campeonato, y por tanto $P(i, 0) = 0$, con $1 \leq i \leq n$. Como no puede producirse una situación en la que ambos equipos hayan ganado todos los partidos que necesitaban, $P(0, 0)$ carece de significado. Por último dado que el equipo A gana cualquier partido con una probabilidad p y pierde con una probabilidad q :

$$P(i, j) = pP(i-1, j) + qP(i, j-1), \quad i \geq 1, j \geq 1$$

Entonces podemos calcular $P(i, j)$ en la forma siguiente:

función $P(i, j)$

si $i = 0$ **entonces devolver** 1

sino si $j = 0$ **entonces devolver** 0

sino devolver $pP(i-1, j) + qP(i, j-1)$

Sea $T(k)$ el tiempo necesario en el caso peor para calcular $P(i, j)$ en donde $k = i + j$. Con este método, vemos que es

$$\begin{aligned} T(1) &= c \\ T(k) &\leq 2T(k-1) + d, \quad k > 1 \end{aligned}$$

en donde c y d son constantes. Reescribiendo $T(k-1)$ en términos de $T(k-2)$, y así sucesivamente, obtenemos

$$\begin{aligned}
 T(k) &\leq 4T(k-2) + 2d + d, \quad k > 1 \\
 &\vdots \\
 &\leq 2^{k-1}T(1) + (2^{k-2} + 2^{k-3} + \dots + 2 + 1)d \\
 &= 2^{k-1}c + (2^{k-1} - 1)d \\
 &= 2^k(c/2 + d/2) - d
 \end{aligned}$$

Por tanto $T(k)$ está en $O(2^k)$, que es $O(4^n)$ si $i = j = n$. De hecho, si examinamos la forma en que se generan las llamadas recursivas, encontramos el patrón que se muestra en la figura 8.2, que es idéntico al cálculo menos sofisticado del coeficiente binomial. Para ver esto imaginemos que toda llamada a $P(m, n)$ en la figura se sustituye por $C(m + n, n)$.

De esta manera, $P(i, j)$ se sustituye por $C(i + j, j)$, $P(i - 1, j)$ se sustituye por $C(i + j - 1, j)$ y $P(i, j - 1)$ se sustituye por $C(i + j - 1, j - 1)$. Ahora el patrón de llamadas que muestran las flechas corresponde al cálculo

$$C(i + j, j) = C(i + j - 1, j) + C(i + j - 1, j - 1)$$

de un coeficiente binomial. El número total de llamadas recursivas, por tanto, es exactamente de $2^{\binom{i+j}{j}} - 2$; véase el Problema 8.1. Así pues para calcular la probabilidad $P(n, n)$ de que el equipo A sea el ganador, dado que todavía no ha empezado el campeonato, el tiempo requerido está, por tanto, en $\Omega(2^{2^n})$.

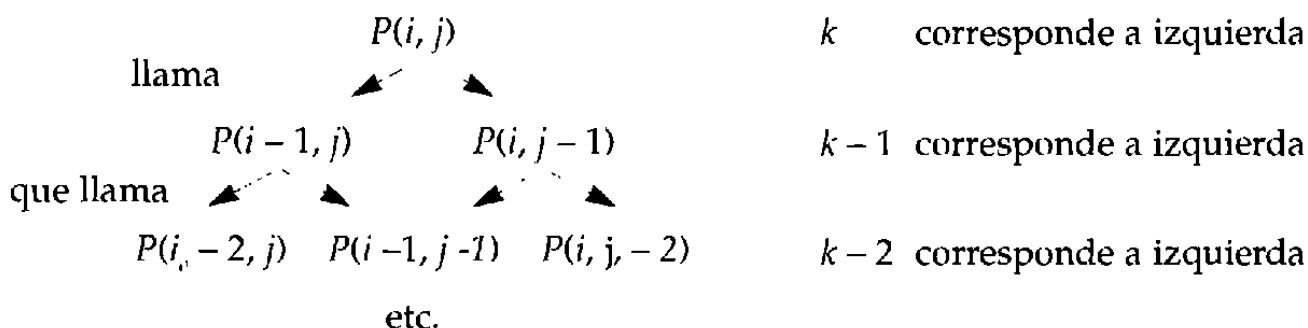


Figura 8.2. Llamadas recursivas efectuadas por una llamada a $P(i, j)$

El Problema 1.42 pide al lector demostrar que $\binom{2n}{n} \geq 4^n / (2n + 1)$. Combinando estos resultados, se ve que el tiempo necesario para calcular $P(n, n)$ está en $O(4^n)$ y en $\Omega(4^n / n)$. El método, por tanto, no resulta práctico para valores grandes de n . (Aun cuando las competiciones deportivas con $n > 4$ sean la excepción, ¡este problema tiene otras aplicaciones!)

Para acelerar el algoritmo, procederemos más o menos igual que con el triángulo de Pascal: declaramos un vector del tamaño adecuado y después vamos rellenando las entradas. Esta vez, sin embargo, en lugar de ir llenando el vector línea por línea, vamos a trabajar diagonal por diagonal. Éste es el algoritmo para calcular $P(n, n)$.

```

función serie( $n, p$ )
  matriz  $P[0..n, 0..n]$ 
   $q \leftarrow 1 - p$ 
  {Llenamos desde la esquina izquierda hasta la diagonal principal}
  para  $s \leftarrow 1$  hasta  $n$  hacer
     $P[0, s] \leftarrow 1; P[s, 0] \leftarrow 0$ 
    para  $k \leftarrow 1$  hasta  $s - 1$  hacer
       $P[k, s - k] \leftarrow pP[k - 1, s - k] + qP[k, s - k - 1]$ 
  {Llenamos desde debajo de la diagonal principal hasta la esquina derecha}
  para  $s \leftarrow 1$  hasta  $n$  hacer
    para  $k \leftarrow 0$  hasta  $n - s$  hacer
       $P[s + k, n - k] \leftarrow pP[s + k - 1, n - k] + qP[s + k, n - k - 1]$ 
  devolver  $P[n, n]$ 

```

Dado que el algoritmo tiene que llenar una matriz $n \times n$, y dado que se necesita una constante temporal para calcular cada entrada, su tiempo de ejecución se encuentra en $O(n^2)$. Al igual que en el triángulo de Pascal, resulta sencillo implementar este algoritmo de tal manera que baste con un espacio de almacenamiento en $\Theta(n)$.

8.2 DEVOLVER CAMBIO (2)

Recuerde que el problema consiste en desarrollar un algoritmo para *pagar una cierta cantidad a un cliente*, empleando el menor número posible de monedas. En la Sección 6.1 describíamos un algoritmo voraz para este problema. Desafortunadamente, aunque el algoritmo voraz es muy eficiente, funciona solamente en un número limitado de casos. Con ciertos sistemas monetarios, o cuando faltan monedas de una cierta denominación (o su número es limitado), el algoritmo puede encontrar una respuesta que no sea óptima, o incluso puede no hallar respuesta ninguna.

Por ejemplo, supongamos que vivimos en un lugar en el cual hay monedas de 1, 4 y 6 unidades. Si tenemos que cambiar 8 unidades, el algoritmo voraz propondrá hacerlo con una moneda de 6 unidades y dos de una unidad, con un total de tres monedas. Sin embargo, está claro que podemos hacerlo mejor: basta con dar al cliente su cambio empleando tan sólo dos monedas de cuatro unidades. Aunque el algoritmo voraz no halla esta solución, resulta sencillo obtenerla empleando programación dinámica.

Como en la sección anterior, el *quid* del método consiste en preparar una tabla que contenga resultados intermedios útiles, que serán combinados en la solución del caso que estamos considerando. Supongamos que el sistema monetario que estamos considerando tiene monedas de n denominaciones diferentes, y que una moneda de denominación i , con $1 \leq i \leq n$ tiene un valor de d_i unidades. Supondremos, como es habitual, que todos los $d_i > 0$. Por el momento, supondremos también que se dispone de un suministro ilimitado de monedas de cada denominación. Por último, supongamos que tenemos que dar al cliente monedas por valor de N unidades, empleando el menor número posible de monedas.

Para resolver este problema mediante programación dinámica, preparamos una tabla $c[1..n, 0..N]$ con una fila para cada denominación posible y una columna para las cantidades que van desde 0 unidades hasta N unidades. En esta tabla, $c[i, j]$ será el número mínimo de monedas necesarias para pagar una cantidad de j unidades, con $0 \leq j \leq N$, empleando solamente monedas de las denominaciones desde 1 hasta i , con $1 \leq i \leq n$. La solución del ejemplar, por tanto, está dada por $c[n, N]$ si lo único que necesitamos saber es el número de monedas que se necesitan. Para rellenar la tabla, obsérvese primero que $c[i, 0]$ es cero para todos los valores de i . Después de esta inicialización, la tabla se puede rellenar o bien fila por fila de izquierda a derecha, o bien columna por columna avanzando hacia abajo. Para abonar una cantidad j utilizando monedas de las denominaciones entre 1 e i , tenemos dos opciones en general. En primer lugar, podemos decidir que no utilizaremos monedas de la denominación i , aun cuando esto está permitido ahora, en cuyo caso $c[i, j] = c[i - 1, j]$. Como alternativa, podemos decidir que emplearemos al menos una moneda de la denominación i . En este caso, una vez que hayamos entregado la primera moneda de esta denominación, quedan por pagar $j - d_i$ unidades. Para pagar esto se necesitan $c[i, j - d_i]$ unidades, así que $c[i, j] = 1 + c[i, j - d_i]$. Dado que deseamos minimizar el número de monedas utilizadas, seleccionaremos aquella alternativa que sea mejor. En general, por tanto:

$$c[i, j] = \min(c[i - 1, j], 1 + c[i, j - d_i])$$

Cuando $i = 1$, uno de los elementos que hay que comparar cae fuera de la tabla. Lo mismo sucede cuando $j < d_i$. Resulta cómodo pensar que tales elementos poseen el valor $+\infty$. Si $i = 1$ y $j < d_1$ entonces los dos elementos que hay que comparar caen fuera de la tabla. En este caso, hacemos $c[i, j]$ igual a $+\infty$ para indicar que es imposible pagar una cantidad j empleando solamente monedas del tipo 1.

La figura 8.3 ilustra el caso dado anteriormente, en el que teníamos que pagar 8 unidades con monedas que valían 1, 4 y 6 unidades. Por ejemplo, $c[3, 8]$ se obtiene en este caso como el menor de $c[2, 8] = 2$ y $1 + c[3, 8 - d_3] = 1 + c[3, 2] = 3$. Las entradas en el resto de la tabla se obtienen de forma similar. La respuesta para este caso concreto es que podemos pagar ocho unidades empleando únicamente dos monedas. De hecho, la tabla nos da la solución de nuestro problema para todos los casos que supongan un pago de 8 unidades o menos.

Cantidad:	0	1	2	3	4	5	6	7	8
$d_1 = 1$	0	1	2	3	4	5	6	7	8
$d_2 = 4$	0	1	2	3	1	2	3	4	2
$d_3 = 6$	0	1	2	3	1	2	1	2	2

Figura 8.3 Devolver cambio empleando programación dinámica

Véase a continuación una versión más formal del algoritmo:

función *monedas*(N)

{Devuelve el mínimo número de monedas necesarias
para cambiar N unidades. El vector $d[1..n]$
especifica las denominaciones: en el ejemplo hay monedas
de 1, 4 y 6 unidades}
vector $d[1..n] = [1, 4, 6]$
matriz $c[1..n, 0..N]$
para $i \leftarrow 1$ **hasta** n **hacer** $c[i, 0] \leftarrow 0$
para $i \leftarrow 1$ **hasta** n **hacer**
 para $j \leftarrow 1$ **hasta** N **hacer**
 si $i = 1$ **y** $j < d[i]$ **entonces** $c[i, j] \leftarrow +\infty$
 sino si $i = 1$ **entonces** $c[i, j] \leftarrow 1 + c[1, j - d[1]]$
 sino si $j < d[i]$ **entonces** $c[i, j] \leftarrow c[i - 1, j]$
 sino $c[i, j] \leftarrow \min(c[i - 1, j], 1 + c[i, j - d[i]])$
devolver $c[n, N]$

Si está disponible un suministro inagotable de monedas con un valor de una unidad, entonces siempre podemos hallar una solución para nuestro problema. De no ser así, puede haber valores de N para los cuales no exista una solución. Esto sucede, por ejemplo, si todas las monedas representan un número par de unidades, y se nos pide que paguemos un número impar de unidades. En tales casos, el algoritmo devuelve el resultado artificial $+\infty$. El problema 8.9 invita al lector a modificar el algoritmo para abordar una situación en que esté limitado el suministro de monedas de una cierta denominación.

Aun cuando el algoritmo sólo parece decir cuántas monedas se necesitan para dar el cambio correspondiente a una determinada cantidad, es fácil descubrir las monedas que se necesitan una vez que se ha construido la tabla c . Supongamos que es preciso abonar una cantidad j empleando monedas de las denominaciones 1, 2, ..., i . Entonces el valor de $c[i, j]$ dice cuántas monedas se van a necesitar. Si $c[i, j] = c[i - 1, j]$ entonces no se necesitan monedas de la denominación i , y pasamos a $c[i - 1, j]$, para ver lo que hay que hacer a continuación; si $c[i, j] = 1 + c[i, j - d_i]$ entonces entregamos una moneda de denominación i , que vale d_i , y avanzamos hacia la izquierda hasta $c[i, j - d_i]$ para ver lo que hay que hacer a continuación. Si $c[i - 1, j]$ y $1 + c[i, j - d_i]$ son ambos iguales a $c[i, j]$ entonces podemos seleccionar cualquiera de las dos posibilidades. Siguiendo de esta manera, volveremos eventualmente a $c[0, 0]$ y ya no quedará nada por abonar. Esta fase del algoritmo es esencialmente un algoritmo voraz que basa sus decisiones en la información de la tabla, y no tiene que retroceder nunca.

El análisis del algoritmo es directo. Para ver cuántas monedas se necesitan para dar un cambio de N unidades cuando están disponibles monedas de n denominaciones diferentes, el algoritmo tiene que rellenar una matriz $n \times (N + 1)$, así que el tiempo de ejecución está en $\Theta(nN)$. Para ver las monedas que habría que utilizar, la búsqueda que retrocede desde $c[n, N]$ hasta $c[0, 0]$ da $n - 1$ pasos hasta la fila de encima (que corresponde a no utilizar una moneda de la denominación ac-

tual) y $c[n, N]$ pasos hacia la izquierda (que corresponde a entregar una moneda). Dado que cada uno de estos pasos se puede dar en un tiempo constante, el tiempo total que se requiere está en $\Theta(n + c[n, N])$.

8.3 EL PRINCIPIO DE OPTIMALIDAD

La solución del problema de devolver cambio que se ha obtenido mediante programación dinámica parece sencilla, y no parece ocultar consideraciones teóricas. Sin embargo, es importante observar que se basa en un principio útil denominado *principio de optimalidad*, que en muchas circunstancias parece tan natural que se invoca casi sin pensarlo. Este principio afirma que en una sucesión óptima de decisiones u opciones, toda subsecuencia debe ser también óptima. En nuestro ejemplo, hemos dado por supuesto, cuando calculábamos $c[i, j]$ como el menor entre $c[i - 1, j]$ y $1 + c[i, j - d_i]$, que si $c[i, j]$ es la forma óptima de devolver cambio de j unidades empleando monedas cuyas denominaciones estén entre 1 e i , entonces $c[i - 1, j]$ y $c[i, j - d_i]$ deben dar también las soluciones *óptimas* de los casos que representan. En otras palabras, aun cuando el único valor de la tabla que nos interesa realmente es $c[n, N]$, hemos dado por supuesto que todas las demás entradas de la tabla deben de representar también las selecciones óptimas: y así es, porque en este problema es aplicable el principio de optimalidad.

Aun cuando este principio pueda parecer evidente, no es aplicable a todos los problemas que podamos encontrar. Cuando el principio de optimalidad *no* es aplicable, es probable que no sea posible atacar al problema en cuestión empleando programación dinámica. Tal es el caso, por ejemplo, de un problema que concierne a la utilización óptima de unos recursos limitados. Aquí la solución óptima de un caso puede no ser la obtenida por combinación de las soluciones óptimas de dos o más subcasos, si los recursos utilizados en esos subcasos suman más que el total de recursos disponibles.

Por ejemplo, si el camino más corto entre Montreal y Toronto pasa por Kingston, entonces la parte del camino que va desde Montreal hasta Kingston también debe de seguir el camino más corto posible, al igual que la parte del camino que une Kingston con Toronto. Por tanto, es aplicable el principio de optimalidad. Sin embargo, si el camino más rápido para ir desde Montreal hasta Toronto nos lleva a través de Kingston, no se sigue necesariamente que lo mejor sea ir tan rápidamente como sea posible desde Montreal hasta Kingston, y después ir tan rápidamente como sea posible desde Kingston hasta Toronto. Si utilizamos demasiada gasolina en la primera mitad del viaje, tendremos que repostar en algún sitio en la segunda parte, y así perderemos más tiempo que el ganado al conducir muy deprisa. Los subviajes desde Montreal hasta Kingston y desde Kingston hasta Toronto no son independientes, porque comparten un recurso, así que la selección de una solución óptima para un subviaje puede impedir que utilicemos una solución óptima para el otro. En esta situación, el principio de optimalidad no es aplicable.

Como segundo ejemplo, consideremos el problema consistente en hallar no el camino más corto, sino el camino simple más largo entre dos ciudades, empleando un conjunto dado de carreteras. Un *camino simple* es uno que nunca pasa dos veces

por el mismo sitio, así que esta condición elimina los caminos infinitos que dan vueltas y más vueltas en un bucle. Si sabemos que el camino simple más largo entre Montreal y Toronto pasa por Kingston, entonces *no* se sigue que sea posible obtenerlo tomando el camino simple más largo desde Montreal hasta Kingston, y después el camino simple más largo desde Kingston hasta Toronto. Es excesivo esperar que al pegar estos dos caminos simples se obtenga otro camino simple también. Una vez más, no es aplicable el principio de optimalidad.

Sin embargo, el principio de optimalidad resulta aplicable en muchas ocasiones. En tal caso, es posible volver a enunciarlo en la forma siguiente: la solución óptima de cualquier caso no trivial de un problema es una combinación de soluciones óptimas de *algunos* de sus subcasos. La dificultad para transformar este principio en un algoritmo es que no suele ser evidente cuáles son los subcasos relevantes para el caso considerado. Volviendo al ejemplo de encontrar el camino más corto, ¿cómo podemos saber si el subcaso que consiste en hallar el camino más corto entre Montreal y Ottawa es relevante cuando deseamos el camino más corto entre Montreal y Toronto? Esta dificultad nos impide utilizar una aproximación similar a divide y vencerás, comenzando en el caso original y buscando recursivamente soluciones óptimas para los subcasos relevantes, y sólo para estos. En su lugar, la programación dinámica resuelve todos los subcasos, para determinar los que realmente son relevantes; sólo entonces se combinan éstos en una solución óptima para el caso original.

8.4 EL PROBLEMA DE LA MOCHILA (2)

Como en la Sección 6.5, se nos da un cierto número de objetos y una mochila. Esta vez, sin embargo, suponemos que los objetos *no* se pueden fragmentar en trozos más pequeños, así que podemos decidir si tomamos un objeto o lo dejamos, pero no podemos tomar una fracción de un objeto. Para $i = 1, 2, \dots, n$ supongamos que el objeto i tiene un peso positivo w_i y un valor positivo v_i . La mochila puede llevar un peso que no supere W . Nuestro objetivo es una vez más llenar la mochila de tal forma que se maximice el valor de los objetos incluidos, respetando la limitación de capacidad. Sea x_i igual a 0 si decidimos no tomar el objeto i , o bien 1 si incluimos el objeto i . En símbolos, el nuevo problema se puede enunciar en la forma siguiente:

$$\text{maximizar } \sum_{i=1}^n x_i v_i \quad \text{con la restricción } \sum_{i=1}^n x_i w_i \leq W$$

donde $v_i > 0$, $w_i > 0$ y $x_i \in \{0, 1\}$ para $1 \leq i \leq n$. Aquí las condiciones que afectan a v_i y a w_i son limitaciones que afectan al caso; las de x_i son restricciones que afectan a la solución. Dado que el problema se parece mucho al de la Sección 6.5, es natural preguntarse en primer lugar si una ligera modificación del algoritmo voraz que utilizábamos antes podría funcionar ahora. Supongamos entonces que se adapta el algoritmo de la forma evidente, de tal modo que vaya examinando los objetos en orden descendente de valor por unidad de peso. Si la mochila no está llena, el algoritmo tiene que seleccionar un objeto *completo* si es posible, antes de pasar al siguiente.

Desafortunadamente, el algoritmo voraz no funciona cuando x_i tiene que ser o bien 0 o bien 1. Por ejemplo, supongamos que están disponibles tres objetos, el primero de los cuales pesa 6 unidades y tiene un valor de 8, mientras que los otros dos pesan 5 unidades cada uno y tienen un valor de 5 cada uno. Si la mochila puede llevar 10 unidades, entonces la carga óptima incluye a los dos objetos más ligeros, con un valor total de 10. El algoritmo voraz, por otra parte, comenzaría por seleccionar el objeto que pesa 6 unidades, puesto que es el que tiene un mayor valor por unidad de peso. Sin embargo, si los objetos no se pueden romper, el algoritmo no podrá utilizar la capacidad restante de la mochila. La carga que produce, por tanto, consta de un solo objeto, y tiene un valor de 8 nada más.

Para resolver el problema por programación dinámica, preparamos una tabla $V[1..n, 0..W]$ que tiene una fila para cada objeto disponible, y una columna para cada peso desde 0 hasta W . En la tabla, $V[i, j]$ será el valor máximo de los objetos que podemos transportar si el límite de peso es j , con $0 \leq j \leq W$, y si solamente incluimos los objetos numerados desde el 1 hasta el i , con $1 \leq i \leq n$. La solución de este caso, por tanto, se puede encontrar en $V[n, W]$.

El paralelismo con el problema de dar la vuelta es bastante estricto. Una vez más, es aplicable el principio de optimalidad. Podemos rellenar la tabla bien fila por fila o bien columna por columna. En la situación general, $V[i, j]$ es el máximo (puesto que estamos intentando maximizar el valor) de $V[i-1, j]$ y $V[i-1, j-w_i] + v_i$. La primera de estas opciones corresponde a no añadir el objeto i a la carga. La segunda corresponde a seleccionar el objeto i , lo cual tiene como efecto incrementar el valor de la carga en v_i , y reducir en w_i la capacidad disponible. Por tanto, llenaremos las entradas de la tabla empleando la regla general

$$V[i, j] = \max(V[i-1, j], V[i-1, j-w_i] + v_i)$$

Para las entradas fuera de límites, definimos $V[0, j]$ igual a 0 cuando $j \geq 0$, y definimos $V[i, j]$ igual a $-\infty$ para todo i cuando $j < 0$. El enunciado formal del algoritmo, que se asemeja mucho a la función *monedas* de la sección anterior, se deja como ejercicio para el lector; véase el problema 8.11.

Límite de peso:	0	1	2	3	4	5	6	7	8	9	10	11
$w_1 = 1, v_1 = 1$	0	1	1	1	1	1	1	1	1	1	1	1
$w_2 = 2, v_2 = 6$	0	1	6	7	7	7	7	7	7	7	7	7
$w_3 = 5, v_3 = 18$	0	1	6	7	7	18	19	24	25	25	25	25
$w_4 = 6, v_4 = 22$	0	1	6	7	7	18	22	24	28	29	29	40
$w_5 = 7, v_5 = 28$	0	1	6	7	7	18	22	28	29	34	35	40

Figura 8.4. La mochila, empleando programación dinámica

La figura 8.4 da un ejemplo del funcionamiento del algoritmo. En la figura hay cinco objetos, cuyos pesos son respectivamente 1, 2, 5, 6 y 7 unidades, y cuyos va-

lores son 1, 6, 18, 22 y 28. Sus valores por unidad de peso son por tanto 1,00, 3,00, 3,60, 3,67 y 4,00. Si sólo podemos transportar un máximo de 11 unidades de peso, entonces la tabla muestra que podemos componer una carga cuyo valor es 40.

De igual modo que para el problema de devolver cambio, la tabla V nos permite recuperar no sólo el valor de la carga óptima que podemos transportar, sino también su composición. En nuestro ejemplo, comenzamos por examinar $V[5, 11]$. Dado que $V[5, 11] = V[4, 11]$ pero $V[5, 11] \neq V[4, 11 - w_5] + v_5$, una carga óptima no puede incluir al objeto 5. A continuación, $V[4, 11] \neq V[3, 11]$ pero $V[4, 11] = V[3, 11 - w_4] + v_4$, así que una carga óptima debe incluir al objeto 4. Ahora $V[3, 5] \neq V[2, 5]$ pero $V[3, 5] = V[2, 5 - w_3] + v_3$, así que debemos incluir el objeto 3. Prosiguiendo de esta forma, encontramos que $V[2, 0] = V[1, 0]$ y que $V[1, 0] = V[0, 0]$, así que la carga óptima no incluye al objeto 2 ni al objeto 1. En este caso, por tanto, sólo hay una carga óptima, que consta de los objetos 3 y 4.

En este ejemplo, el algoritmo voraz consideraría primero el objeto 5, puesto que es el que tiene un mayor valor por unidad de peso. La mochila sólo puede llevar uno de estos objetos. A continuación, el algoritmo voraz consideraría el objeto 4, cuyo valor por unidad de peso es el siguiente. Este objeto no se puede incluir en la carga sin violar la restricción de capacidad. Prosiguiendo de esta manera, el algoritmo voraz examinaría los objetos 3, 2 y 1, por este orden, y acabaría por ofrecernos una carga que constaría de los objetos 5, 2 y 1 con un valor total de 35. Una vez más, vemos que el algoritmo voraz no funciona cuando no es posible romper los objetos.

El análisis del algoritmo de programación dinámica es directo, y se asemeja mucho al análisis del algoritmo para devolver cambio. Encontramos que se necesita un tiempo que está en $\Theta(nW)$ para construir la tabla V , y que la composición de la carga óptima se puede determinar en un tiempo que está en $O(n + W)$.

8.5 CAMINOS MÍNIMOS

Sea $G = \langle N, A \rangle$ un grafo dirigido; N es el conjunto de nodos y A es el conjunto de aristas. Toda arista tiene asociada una longitud no negativa. Deseamos calcular la longitud del camino más corto entre cada par de nodo. Compare esto con la Sección 6.4, en la cual buscábamos la longitud de los caminos más cortos entre un nodo concreto, el nodo origen, y todos los demás nodos.

Como antes, supongamos que los nodos de G están numerados desde 1 hasta n , así que $N = \{1, 2, \dots, n\}$, y supongamos que hay una matriz L que da las longitudes de las aristas, con $L[i, i] = 0$ para $i = 1, 2, \dots, n$, $L[i, j] \geq 0$ para todo i y j , y $L[i, j] = \infty$ si no existe la arista (i, j) .

Es aplicable el principio de optimalidad: si k es un nodo del camino mínimo entre i y j , entonces la parte del camino que va desde i hasta k , y la parte del camino que va desde k hasta j debe ser óptimo también.

Construimos una matriz D que da la longitud del camino más corto entre un par de nodos. El algoritmo da a D el valor inicial L , esto es, las distancias directas

entre nodos. A continuación, efectúa n iteraciones. Después de la iteración k , D da la longitud de los caminos más cortos que utilizan solamente los nodos $\{1, 2, \dots, k\}$ como nodos intermedios. Al cabo de n iteraciones, D nos da por tanto la longitud de los caminos más cortos que utilicen alguno de los nodos de N como nodo intermedio, que es el resultado deseado. En la iteración k , el algoritmo debe comprobar para cada par de nodos (i, j) si existe o no un camino que vaya de i a j pasando por el nodo k , y que sea mejor que el camino óptimo actual que pasa sólo por los nodos $\{1, 2, \dots, k-1\}$. Si D_k representa la matriz D después de la k -ésima iteración (de modo que $D_0 = L$), entonces la comprobación necesaria puede implementarse en la forma

$$D_k[i, j] = \min(D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j])$$

donde utilizamos el principio de optimalidad para calcular el camino más corto que va desde i hasta j pasando por k . Tácitamente, hemos utilizado el hecho de que un camino óptimo que pase por k no visitará k dos veces.

$$D_0 = L = \begin{pmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & \infty & 0 & 15 \\ 15 & \infty & 5 & 0 \end{pmatrix}$$

$$D_1 = \begin{pmatrix} 0 & 5 & \infty & \infty \\ 50 & 0 & 15 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{pmatrix} \quad D_2 = \begin{pmatrix} 0 & 5 & 20 & 10 \\ 50 & 0 & 15 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{pmatrix}$$

$$D_3 = \begin{pmatrix} 0 & 5 & 20 & 10 \\ 45 & 0 & 15 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{pmatrix} \quad D_4 = \begin{pmatrix} 0 & 5 & 15 & 10 \\ 20 & 0 & 10 & 5 \\ 30 & 35 & 0 & 15 \\ 15 & 20 & 5 & 0 \end{pmatrix}$$

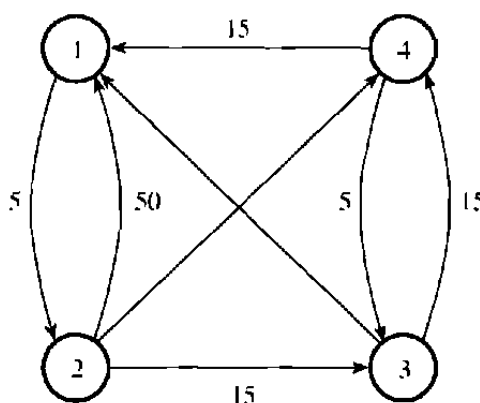


Figura 8.5. El algoritmo de Floyd en funcionamiento

En la k -ésima iteración, los valores de la k -ésima fila y de la k -ésima columna de D no cambian, porque $D[k, k]$ es siempre 0. Por tanto, no hace falta proteger estos valores al actualizar D . Esto nos permite utilizar una única matriz D $n \times n$, mien-

tras que a primera vista pudiera parecer necesario utilizar dos de estas matrices, una que contendría los valores de D_{k-1} y otra los valores de D_k , o incluso una matriz $n \times n \times n$.

El algoritmo, que se conoce con el nombre de *algoritmo de Floyd*, es como sigue:

```
función Floyd( $L[1..n, 1..n]$ ): matriz  $[1..n, 1..n]$ 
  matriz  $D[1..n, 1..n]$ 
   $D \leftarrow L$ 
  para  $k \leftarrow 1$  hasta  $n$  hacer
    para  $i \leftarrow 1$  hasta  $n$  hacer
      para  $j \leftarrow 1$  hasta  $n$  hacer
         $D[i, j] \leftarrow \min(D[i, j], D[i, k] + D[k, j])$ 
  devolver  $D$ 
```

La figura 8.5 da un ejemplo de la forma en que funciona el algoritmo.

Es evidente que este algoritmo requiere un tiempo que está en $\Theta(n^3)$. También podemos utilizar el algoritmo de Dijkstra para resolver el mismo problema; véase la Sección 6.4. En este caso, tenemos que aplicar el algoritmo n veces, seleccionando en cada ocasión un nodo distinto como origen. Si utilizamos la versión del algoritmo de Dijkstra que funciona con una matriz de distancias, entonces el tiempo total de cálculo está en $n \times \Theta(n^2)$, esto es, en $\Theta(n^3)$. El orden es el mismo que para el algoritmo de Floyd, pero la sencillez de este último significa que es probable que tenga una constante oculta más pequeña, y por tanto que sea más rápido en la práctica. Además a los compiladores se les da bien optimizar los bucles **para**. Por otra parte, si utilizamos la versión del algoritmo de Dijkstra que emplea un montículo, y por tanto listas de las distancias hasta los nodos adyacentes, el tiempo total está en $n \times \Theta((a + n) \log n)$, esto es, en $\Theta((an + n^2) \log n)$, donde a es el número de aristas del grafo. Si el grafo es disperso ($a \ll n^2$), entonces quizá sea preferible utilizar el algoritmo de Dijkstra n veces; si el grafo es denso $a \approx n^2$, es mejor utilizar el algoritmo de Floyd.

Normalmente, deseamos saber cuál es el camino más corto, y no solamente su longitud. En tal caso, utilizaremos una segunda matriz P , cuyos elementos tienen todos ellos un valor inicial 0. El bucle más interno del algoritmo pasa a ser

```
si  $D[i, k] + D[k, j] < D[i, j]$  entonces  $D[i, j] \leftarrow D[i, k] + D[k, j]$ 
                                      $P[i, j] \leftarrow k$ 
```

Cuando se detiene el algoritmo, $P[i, j]$ contiene el número de la última iteración que haya dado lugar a un cambio en $D[i, j]$. Para recuperar el camino más corto desde i hasta j , se examina $P[i, j]$. Si $P[i, j] = 0$, entonces $D[i, j]$ nunca ha cambiado, y el camino mínimo pasa directamente por la arista (i, j) ; en caso contrario, si $P[i, j] = k$, entonces el camino más corto desde i hasta j pasa por k . Se examina recursivamente $P[i, k]$ y $P[k, j]$ para buscar cualquier otro posible nodo intermedio que esté en el camino más corto.

Si tomamos como ejemplo el grafo de la figura 8.5, entonces P pasa a ser

$$P = \begin{pmatrix} 0 & 0 & 4 & 2 \\ 4 & 0 & 4 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}.$$

Dado que $P[1, 3] = 4$, el camino mínimo desde 1 hasta 3 pasa por 4. Examinando ahora $P[1, 4]$ y $P[4, 3]$, descubrimos que entre 1 y 4 hay que pasar por 2, pero que de 4 a 3 pasamos directamente. Por último, vemos que los recorridos desde 1 hasta 2 y desde 2 hasta 4 también son directos. Por tanto, el camino mínimo desde 1 hasta 3 es 1, 2, 4, 3.

Si permitimos que las aristas del grafo tengan longitudes negativas, entonces la noción de «camino más corto» pierde gran parte de su significado: si el grafo incluye un ciclo cuya longitud total sea negativa, ¡cuanto más pasemos por el bucle, más corto será nuestro camino! El Problema 8.17 pregunta lo que sucede con el algoritmo de Floyd si le damos un grafo con aristas negativas, pero sin ciclos negativos. Aun cuando un grafo tenga ciclos negativos, sigue teniendo sentido pedir los caminos simples más cortos. (Recuerde que un camino simple es aquella que nunca visita dos veces el mismo nodo.) No se conoce un algoritmo eficiente para hallar los caminos simples más cortos en grafos que puedan tener aristas de longitud negativa. La situación es la misma que para el problema de hallar los caminos simples más largos, mencionado en la Sección 8.3: no se conoce un algoritmo eficiente. Estos dos problemas son *NP*-completos; véase el Capítulo 12.

8.6 MULTIPLICACIÓN ENCADENADA DE MATRICES

Recuerde que el producto de una matriz $p \times q$ llamada A por una matriz $q \times r$ llamada B es la matriz $p \times r$ dada por

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}, \quad 1 \leq i \leq p, 1 \leq j \leq r$$

Algorítmicamente, se puede expresar esto en la forma

```
para  $i \leftarrow 1$  hasta  $p$  hacer
  para  $j \leftarrow 1$  hasta  $r$  hacer
     $C[i, j] \leftarrow 0$ 
    para  $k \leftarrow 1$  hasta  $q$  hacer
       $C[i, j] \leftarrow C[i, j] + A[i, k]B[k, j]$ 
```

de donde se deduce claramente que se necesitan pqr multiplicaciones escalares para calcular la matriz producto empleando este algoritmo. (En esta sección no consideramos la posibilidad de utilizar un algoritmo mejor para la multiplicación de matrices, como puede ser el algoritmo de Strassen, descrito en la Sección 7.6.)

Supongamos ahora que deseamos calcular el producto de más de dos matrices. La multiplicación de matrices es asociativa, así que podemos calcular el producto matricial

$$M = M_1 M_2 \cdots M_n$$

de muchas maneras, todas las cuales darán la misma respuesta:

$$\begin{aligned} M &= (\cdots((M_1 M_2) M_3) \cdots M_n) \\ &= (M_1 (M_2 (M_3 \cdots (M_{n-1} M_n) \cdots))) \\ &= (\cdots((M_1 M_2) (M_3 M_4)) \cdots), \end{aligned}$$

y así sucesivamente. Sin embargo, la multiplicación de matrices no es conmutativa, así que no se nos permite modificar el orden de las matrices.

La selección de método de cálculo puede tener una notable influencia sobre el tiempo requerido. Supongamos, por ejemplo, que deseamos calcular el producto $ABCD$ de cuatro matrices, en donde A es 13×5 , B es 5×89 , C es 89×3 y D es 3×34 . Para medir la eficiencia de los diferentes métodos, contaremos el número de multiplicaciones escalares implicadas. Tal como se ha programado anteriormente, habrá un número igual de sumas escalares, además de algo de mantenimiento, así que el número de multiplicaciones escalares es un buen indicador de la eficiencia general. Por ejemplo, utilizando $M = ((AB)C)D$, se calculan sucesivamente

AB	5.785 multiplicaciones
$(AB)C$	3.471 multiplicaciones
$((AB)C)D$	1.324 multiplicaciones

con un total de 10.582 multiplicaciones escalares. Hay cinco formas esencialmente diferentes de calcular el producto en este caso: cuando se expresa el producto en la forma $(AB)(CD)$, no distinguimos entre el método que calcula primero AB y después CD y el que empieza por CD y después calcula AB , puesto que ambos requieren el mismo número de multiplicaciones. Para cada uno de estos cinco métodos, los números correspondientes de multiplicaciones escalares son:

$((AB)C)D$	10.582
$(AB)(CD)$	54.201
$(A(BC))D$	2.856
$A((BC)D)$	4.055
$A(B(CD))$	26.418

El método más eficiente es casi 19 veces más rápido que el más lento.

Para hallar directamente la mejor manera de calcular el producto, podríamos limitarnos a poner paréntesis en la expresión de todas las maneras posibles, contando en cada ocasión el número de multiplicaciones escalares que se necesita. Sea $T(n)$ el número de formas esencialmente distintas de poner paréntesis en un producto de n matrices. Supongamos que decidimos hacer el primer corte entre las matrices i -ésima e $(i+1)$ -ésima del producto, en la forma siguiente:

$$(M_1 M_2 \cdots M_i)(M_{i+1} M_{i+2} \cdots M_n)$$

Ahora hay $T(i)$ formas de poner paréntesis en el término del lado izquierdo, y $T(n-i)$ formas de poner paréntesis en el término del lado derecho. Cualquier forma de las primeras se puede combinar con cualquiera de las segundas, así que para este valor concreto de i hay $T(i)T(n-i)$ formas de poner paréntesis en toda la expresión. Dado que i puede tomar cualquier valor entre 1 y $n-1$, obtenemos finalmente la recurrencia siguiente para $T(n)$:

$$T(n) = \sum_{i=1}^{n-1} T(i)T(n-i)$$

Añadiendo la condición inicial evidente, $T(1) = 1$, podemos utilizar la recurrencia para calcular cualquier valor que se necesite de T . La tabla siguiente muestra algunos de los valores de $T(n)$.

n	1	2	3	4	5	10	15
$T(n)$	1	1	2	5	14	4.862	2.674.440

Los valores de $T(n)$ se llaman *números de Catalan*.

Para cada manera de poner paréntesis en la expresión de M , se necesita un tiempo que está en $\Omega(n)$ para contar el número de multiplicaciones escalares que se necesitan (al menos, si no intentamos ser sutiles). Dado que $T(n)$ está en $\Omega(4^n / n^2)$ (combine los resultados de los problemas 8.24 y 1.42), la búsqueda de la mejor manera de calcular M empleando la aproximación directa requiere un tiempo que está en $\Omega(4^n / n)$. Este método, por tanto, no resulta práctico para valores grandes de n : hay demasiadas maneras de insertar paréntesis para examinarlas todas.

Un poco de experimentación mostrará que ninguno de los algoritmos voraces evidentes es capaz de calcular productos matriciales de forma óptima; véase el Problema 8.20. Afortunadamente, el principio de optimalidad es aplicable este problema. Por ejemplo, si la mejor manera de multiplicar todas las matrices nos exige hacer el primer corte entre las matrices i -ésima e $(i+1)$ -ésima del producto, entonces los dos subproductos $M_1 M_2 \cdots M_i$ y $M_{i+1} M_{i+2} \cdots M_n$ también deben calcularse de forma óptima. Esto sugiere que deberíamos considerar la posibilidad de utilizar la programación dinámica. Construimos una tabla m_{ij} , $1 \leq i \leq j \leq n$, en donde m_{ij} da la solución óptima, esto es, el número requerido de multiplicaciones escalares para la parte $M_i M_{i+1} \cdots M_j$ del producto solicitado. La solución del problema original, por tanto, viene dada por m_{1n} .

Supongamos que las dimensiones de las matrices están dadas por un vector $d[0..n]$ tal que la matriz M_i , con $1 \leq i \leq n$, es de dimensión $d_{i-1} \times d_i$. Construimos la tabla m_{ij} diagonal por diagonal: la diagonal s contiene los elementos m_{ii} tales que $j - i = s$. Por tanto la diagonal $s = 0$ contiene los elementos m_{ii} con $1 \leq i \leq n$, correspondientes a los «productos» M_i . Aquí no hay que hacer ninguna multiplicación, así que $m_{ii} = 0$ para todo i . La diagonal $s = 1$ contiene los elementos $m_{i,i+1}$ correspondientes a productos de la forma $M_i M_{i+1}$. Aquí no tenemos más opción que calcular directamente el producto, lo cual se puede hacer efectuando $d_{i-1}d_i d_{i+1}$ multiplicaciones escalares, tal como veíamos al principio de la sección. Finalmente, cuando sea $s > 1$, la diagonal s contendrá los elementos $m_{i,i+s}$ correspondientes a productos de la forma $M_i M_{i+1} \dots M_{i+s}$. Ahora tenemos una opción: podemos hacer el primer corte en el producto después de cualquiera de las matrices $M_i M_{i+1}, \dots, M_{i+s-1}$. Si hacemos el corte después de M_k , con $i \leq k < i + s$, entonces se necesitan m_{ik} multiplicaciones escalares para calcular el término de la izquierda, $m_{k+1,i+s}$ para calcular el término de la derecha, y por último $d_{i-1}d_k d_{i+s}$ para multiplicar las dos matrices resultantes con objeto de obtener el resultado final. Para hallar el óptimo, seleccionamos aquel corte que minimice el número requerido de multiplicaciones escalares.

En resumen, llenamos la tabla m_{ij} utilizando las reglas siguientes para $s = 0, 1, 2, \dots, n-1$:

$$\begin{array}{lll} s = 0: & m_{ii} & = 0 & i = 1, 2, \dots, n \\ s = 1: & m_{i,i+1} & = d_{i-1}d_i d_{i+1} & i = 1, 2, \dots, n-1 \\ 1 < s < n: & m_{i,i+s} & = \min_{i \leq k < i+s} (m_{ik} + m_{k+1,i+s} + d_{i-1}d_k d_{i+s}) & i = 1, 2, \dots, n-s \end{array}$$

Sólo se necesita escribir explícitamente el segundo caso por claridad, puesto que cae dentro del caso general con $s = 1$.

Para aplicar esto al ejemplo, deseamos calcular el producto $ABCD$ de cuatro matrices, donde A es 13×5 , B es 5×89 , C es 89×3 y D es 3×34 . El vector d es por tanto $(13, 5, 89, 3, 34)$. Para $s = 1$, encontramos que es $m_{12} = 5785$, $m_{23} = 1335$ y $m_{34} = 9078$. A continuación, para $s = 2$ obtenemos

$$\begin{aligned} m_{13} &= \min (m_{11} + m_{23} + 13 \times 5 \times 3, m_{12} + m_{33} + 13 \times 89 \times 3) \\ &= \min (1.530, 9.256) = 1.530 \\ m_{24} &= \min (m_{22} + m_{34} + 5 \times 89 \times 34, m_{23} + m_{44} + 5 \times 3 \times 34) \\ &= \min (24.208, 1.845) = 1.845 \end{aligned}$$

Finalmente, para $s = 3$

$$\begin{aligned} m_{14} &= \min (m_{11} + m_{24} + 13 \times 5 \times 34, & \{k = 1\} \\ & \quad m_{12} + m_{34} + 13 \times 89 \times 34, & \{k = 2\} \\ & \quad m_{13} + m_{44} + 13 \times 3 \times 34, & \{k = 3\} \\ &= \min (4.055, 54.201, 2.856) = 2.856 \end{aligned}$$

La matriz completa m se muestra en la figura 8.6.

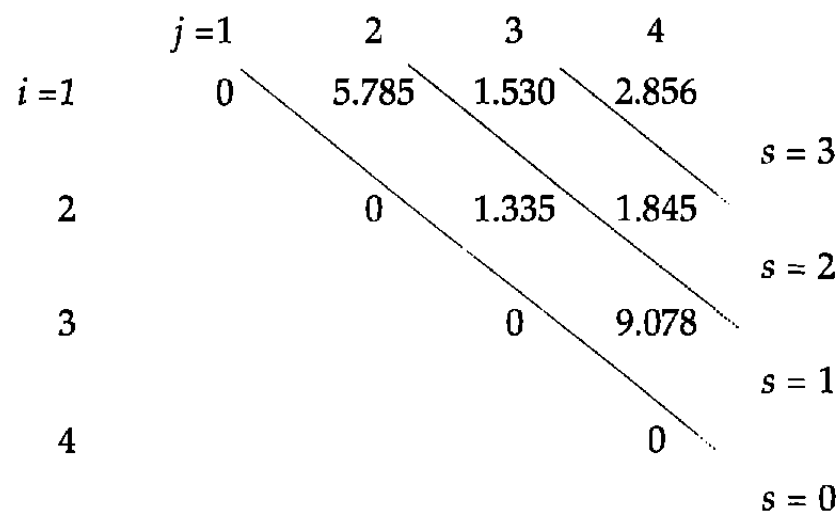


Figura 8.6. Un ejemplo del algoritmo de multiplicación de matrices encadenadas

Una vez más, lo normal será que no sólo queramos saber el número de multiplicaciones escalares necesarias para calcular el producto M , sino también la forma de efectuar eficientemente este cálculo. Como en la Sección 8.5, hacemos esto añadiendo una segunda matriz para llevar la cuenta de las opciones que vamos tomando. Sea *mejork* esta nueva matriz. Ahora, cuando calculemos m_{ij} guardaremos en *mejork*[i, j] el valor de k que corresponde al término mínimo entre los comparados. Cuando se detenga el algoritmo, *mejork*[1, n] nos dirá dónde hay que hacer el primer corte del producto. Procediendo de forma recursiva en los dos términos producidos de esta manera, podemos reconstruir la forma óptima de poner paréntesis en M . Los problemas 8.21 y 8.22 le invitan a rellenar los detalles.

Para $s > 0$ hay que calcular $n - s$ elementos en la diagonal s ; para cada uno de ellos, necesitamos decidirnos entre s posibilidades dadas por los distintos valores de k . Por tanto el tiempo de ejecución del algoritmo está en el orden exacto de

$$\begin{aligned}
 \sum_{s=1}^n (n-s)s &= n \sum_{s=1}^{n-1} s - \sum_{s=1}^{n-1} s^2 \\
 &= n^2(n-1)/2 - n(n-1)(2n-1)/6 \\
 &= (n^3 - n)/6
 \end{aligned}$$

donde hemos utilizado las proposiciones 1.7.14 y 1.7.15 para evaluar las sumas. El tiempo de ejecución de este algoritmo está entonces en $\Theta(n^3)$, si bien existen mejores algoritmos.

8.7 ENFOQUES QUE APLICAN RECURSIÓN

Aun cuando los algoritmos de programación dinámica, tal como el que se acaba de dar para calcular m , son eficientes, hay algo poco satisfactorio acerca del enfoque ascendente. Un método descendente, bien sea divide y vencerás, refinamiento progresivo o recursión, parece más natural, sobre todo para aquellos a quienes se ha enseñado a desarrollar programas de esta manera. Una razón de más peso es que el enfoque ascendente nos lleva a calcular valores que podrían ser totalmente irrelevantes. Resulta tentador, por tanto, ver si podemos alcanzar la misma eficiencia en una versión descendente del algoritmo.

Ilustramos esta idea con el problema de multiplicación matricial descrito en la sección anterior. Una línea de ataque sencilla consiste en sustituir la tabla m por una función fm , que se calcula cuando es necesario. En otras palabras, deseáramos encontrar una función fm tal que $fm(i, j) = m_{ij}$ para $1 \leq i \leq j \leq n$, pero que pueda ser calculada recursivamente, a diferencia de la tabla m , que calculábamos de forma ascendente.

Resulta sencillo escribir esta función, lo único que hay que hacer es programar las reglas para calcular m :

```

función  $fm(i, j)$ 
  si  $i = j$  entonces {sólo hay una matriz }
    devolver 0
   $m \leftarrow \infty$ 
  para  $k \leftarrow i$  hasta  $j-1$  hacer
     $m \leftarrow \text{mín}(m, fm(i, k) + fm(k+1, j) + d[i-1]d[k]d[j])$ 
  devolver  $m$ 

```

El vector global $d[0..n]$ da las dimensiones de las matrices que hay que multiplicar, exactamente igual que antes. Para todos los valores relevantes de k los intervalos $[i..k]$ y $[k+1..j]$ implicados en las llamadas recursivas afectan a menos matrices que $[i..j]$. Sin embargo, toda llamada recursiva sigue afectando al menos a una matriz (siempre y cuando, por supuesto, $i \leq j$ en la llamada original). Eventualmente, por tanto, la recursión finaliza. Para averiguar cuántas multiplicaciones escalares se necesitan para calcular $M = M_1 M_2 \cdots M_n$, nos limitamos a llamar a $fm(1, n)$.

Para analizar este algoritmo, sea $T(s)$ el tiempo requerido para ejecutar una llamada a $fm(i, i + s)$, donde s es el número de multiplicaciones matriciales en el producto correspondiente. Éste es el mismo s que se utilizaba anteriormente para numerar las diagonales de la tabla m . Es claro que $T(0) = c$ para alguna constante c . Cuando $s > 0$, tenemos que seleccionar el menor entre s términos, cada uno de los cuales es de la forma:

$$fm(i, k) + fm(k+1, i + s) + d[i-1]d[k]d[i + s], \quad i \leq k < i + s$$

Sea d una constante tal que se pueden ejecutar dos multiplicaciones escalares, seis adiciones escalares, una comparación con el valor anterior del mínimo y el mantenimiento necesario, todo ello en un tiempo d . Ahora el tiempo necesario para evaluar uno de estos términos es $T(k-i) + T(i+s-k-1) + d$. El tiempo total $T(s)$ necesario para evaluar $fm(i, i+s)$ es por tanto

$$T(s) = \sum_{k=i}^{i+s-1} (T(k-i) + T(i+s-k-1) + b).$$

Si hacemos $m = k - i$, entonces esta expresión se convierte en

$$\begin{aligned} T(s) &= \sum_{m=0}^{s-1} (T(m) + T(s-1-m) + b) \\ &= sb + 2 \sum_{m=0}^{s-1} T(m). \end{aligned}$$

Dado que esto implica que $T(s) \geq 2T(s-1)$, vemos inmediatamente que $T(s) \geq 2^s T(0)$, así que el algoritmo requiere ciertamente un tiempo en $\Omega(2^n)$ para encontrar la mejor manera de multiplicar n matrices. Por tanto, no puede ser competitivo con el algoritmo que utiliza programación dinámica y que requiere un tiempo que está en $\Theta(n^3)$.

El algoritmo se puede acelerar si somos inteligentes y evitamos las llamadas recursivas cuando $d[i-1]d[k]d[i+s]$ ya es mayor que el valor anterior del mínimo. La mejora dependerá del caso, pero es muy improbable que un algoritmo que requiere un tiempo exponencial sea competitivo con respecto a uno que sólo requiere un tiempo polinómico.

Para encontrar una cota superior del tiempo requerido por el algoritmo recursivo, utilizamos inducción constructiva. Examinando la forma de la recursión para T , y recordando que $T(s) \geq 2^s T(0)$, parece posible que T estuviera acotado por una potencia de alguna constante mayor que 2: intentemos demostrar que $T(s) \leq a3^s$, para alguna constante adecuada a . Tomemos esto como hipótesis de inducción, y supongamos que es cierto para todo $m < s$. Al sustituir en la recurrencia, obtenemos

$$\begin{aligned} T(s) &\leq sd + 2 \sum_{m=0}^{s-1} a3^m \\ &= sd + a3^s - a \end{aligned}$$

donde hemos usado la Proposición 1.7.10 para calcular la suma. Desafortunadamente de aquí no podemos concluir que $T(s) \leq a3^s - a$. Sin embargo, si adoptamos la táctica recomendada en la Sección 1.6.4 y reforzamos la hipótesis de inducción, las cosas salen mejor. Como hipótesis de inducción reforzada, suponemos que $T(m) \leq a3^m - b$ para $m < s$ donde b no es una constante desconocida. Ahora, al sustituir en la recurrencia obtenemos

$$\begin{aligned}
 T(s) &\leq sd + 2 \sum_{m=0}^{s-1} (a3^m - b) \\
 &= s(d - 2b) + a3^s - a
 \end{aligned}$$

Esto es suficiente para asegurar que $T(s) \leq a3^s - b$, siempre y cuando $b \geq d/2$ y $a \geq b$. Para empezar la inducción, se necesita que $T(0) \leq a - b$, lo cual se satisface siempre y cuando $a \geq T(0) + b$. En resumen, hemos demostrado que $T(s) \leq a3^s - b$ para todo s , siempre y cuando $b \geq d/2$ y $a \geq T(0) + b$. El tiempo requerido por el algoritmo recursivo para calcular la mejor forma de realizar un producto de n matrices está por tanto en $O(3^n)$.

Concluimos que una llamada a la función recursiva $fm(1, n)$ es más rápida que probar inocentemente todas las posibles formas de poner paréntesis en el producto deseado, lo cual, como ya se vio, requiere un tiempo que está en $\Omega(4^n / n)$. Sin embargo, es más lento que el algoritmo de programación dinámica descrito anteriormente. Esto ilustra algo que se decía anteriormente en este capítulo. Para decidir la mejor manera de poner paréntesis en el producto $ABCDEFG$, digamos, fm resuelve recursivamente 12 subcasos, incluyendo $ABCDEF$ y $BCDEFG$, que se solapan, y resuelven ambos recursivamente $BCDEF$ partiendo de cero. Esta duplicación de esfuerzos es lo que hace ineficiente a fm .

8.8 FUNCIONES CON MEMORIA

El algoritmo de la sección anterior no es el primero que hemos visto en el que una formulación recursiva sencilla de una solución da lugar a un programa ineficiente, porque hay subcasos comunes que se resuelven independientemente más de una vez. La programación dinámica nos permite evitar esta repetición, a costa de complicar el algoritmo. En la programación dinámica, además, es posible que tengamos que resolver algunos subcasos irrelevantes, puesto que sólo posteriormente conoceremos con exactitud las subsoluciones que se necesitan. Un algoritmo descendente recursivo no tiene esta limitación. ¿Será posible combinar las ventajas de ambas técnicas, y mantener la sencillez de una formulación recursiva sin perder la eficiencia que ofrece la programación dinámica?

Una forma sencilla de hacer esto que funciona en muchas situaciones, consiste en utilizar una *función con memoria*. Al programa recursivo se le añade una tabla del tamaño necesario. Inicialmente, todas las entradas de esta tabla contienen un valor especial que muestra que todavía no se han calculado. En lo sucesivo, cada vez que se llama a la función, se examina primero la tabla para ver si ya ha sido evaluada con el mismo conjunto de parámetros. De ser así, se devuelve el valor de la tabla. En caso contrario, seguimos adelante y calculamos la función. Antes de devolver el valor calculado, sin embargo, lo almacenamos en el lugar adecuado de la tabla. De esta forma nunca es necesario calcular dos veces la función para los mismos valores de sus parámetros.

Para el algoritmo recursivo fm de la Sección 8.7, sea $mtab$ una tabla cuyas entradas se inicializan todas a -1 (puesto que el número de multiplicaciones

escalares necesarias para calcular un producto de matrices no puede ser negativo). La siguiente reformulación de la función fm , que utiliza la tabla $mtab$ como variable global, combina la claridad de una formulación recursiva con la eficiencia de la programación dinámica:

```

función  $fm\text{-}mem(i, j)$ 
  si  $i = j$  entonces devolver 0
  si  $mtab[i, j] \geq 0$  entonces devolver  $mtab[i, j]$ 
   $m \leftarrow \infty$ 
  para  $k \leftarrow i$  hasta  $j-1$  hacer
     $m \leftarrow \min(m, fm\text{-}mem(i, k) + fm\text{-}mem(k+1, j)$ 
       $+ d[i-1]d[k]d[j])$ 
   $mtab[i, j] \leftarrow m$ 
  proporcionar  $m$ 

```

Como se indicaba en la Sección 8.7, esta función se puede acelerar evitando las llamadas recursivas si $d[i-1]d[k]d[j]$ ya es mayor que el valor anterior de m .

En algunas ocasiones, el uso de esta técnica exige pagar un cierto precio. Por ejemplo, en la Sección 8.1.1 vimos que se podía calcular el coeficiente binomial $\binom{n}{k}$ utilizando un tiempo que estaba en $\Theta(nk)$ y un espacio que estaba en $\Theta(k)$. Cuando se implementa utilizando una función con memoria, el cálculo requiere la misma cantidad de tiempo, pero necesita un espacio que está en $\Omega(nk)$, véase el Problema 8.26.

Si utilizamos un poco más de espacio (el espacio necesario sólo se multiplica por un factor constante), entonces podemos evitar el tiempo de inicialización que se necesita para dar un valor especial a todas las entradas de la tabla. Se puede hacer esto empleando una *inicialización virtual*, que se describía en la Sección 5.1. Esto resulta especialmente deseable cuando sólo es preciso calcular unos pocos valores de la función, pero no sabemos cuáles por anticipado. Por ejemplo, véase la Sección 9.1.

8.9 PROBLEMAS

Problema 8.1. Demostrar que el número total de llamadas recursivas efectuadas durante el cálculo de $C(n, k)$ empleando el algoritmo de la Sección 8.1.1 es exactamente $2^{\binom{n}{k}} - 2$.

Problema 8.2. El cálculo de la sucesión de Fibonacci ofrece otro ejemplo de la clase de técnica presentada en la Sección 8.1. ¿Qué algoritmo de la Sección 2.7.5 utiliza programación dinámica?

Problema 8.3. Demostrar que el tiempo necesario para calcular $P(n, n)$ empleando la función P de la Sección 8.1.2 está en $O(4^n / \sqrt{n})$.

Problema 8.4. Utilizando el algoritmo *campeonato* de la Sección 8.1.2, calcular la probabilidad de que el equipo A gane el campeonato si $p = 0,45$ y si se necesitan cuatro victorias para ganar.

Problema 8.5. Repetir el problema anterior con $p = 0,55$. ¿Cuál debería ser la relación entre las respuestas a estos dos problemas?

Problema 8.6. Igual que en el problema 8.4, calcular la probabilidad de que el equipo A gane el campeonato si $p = 0,45$ y se necesitan cuatro victorias para ganar. Esta vez, sin embargo, calcular la probabilidad requerida directamente como la probabilidad de que el equipo A gane 4 o más partidos de una serie de 7. (Jugar partidos adicionales después de que el equipo A haya ganado el campeonato no puede modificar el resultado.)

Problema 8.7. Adaptar el algoritmo *campeonato* de la Sección 8.1.2 al caso en que el equipo A gane cualquier partido dado con una probabilidad p , y que lo pierda con una probabilidad q , pero habiendo también una probabilidad r de que haya empate, lo que no supone una victoria para ningún equipo. Suponga que siguen siendo necesarias n victorias para ganar el campeonato. Desde luego, tiene que ser $p + q + r = 1$.

Problema 8.8. Demostrar que un espacio de almacenamiento en $\Theta(n)$ basta para implementar el algoritmo *campeonato* de la Sección 8.1.2.

Problema 8.9. Adaptar el algoritmo *monedas* de la Sección 8.2 para que funcione correctamente aun cuando el número de monedas de una cierta denominación esté limitado.

Problema 8.10. Rehacer el ejemplo ilustrado en la figura 8.4, pero renumerando los objetos por orden inverso (de tal modo que $w_1 = 7, v_1 = 28, \dots, w_5 = 1, v_5 = 1$). ¿Qué elementos de la tabla no deben resultar afectados?

Problema 8.11. Escribir el algoritmo para rellenar la tabla V según se describe en la Sección 8.4.

Problema 8.12. Cuando $j < w$, en el algoritmo para rellenar la tabla V , según se des-

cribe en la Sección 8.4, consideramos que $V[i-1, j-w]$ es $-\infty$. ¿Puede haber entradas que valgan $-\infty$ en la tabla ya terminada? De ser así, ¿qué indican? En caso contrario, ¿por qué no?

Problema 8.13. Puede haber más de una solución óptima para un caso del problema de la mochila. Empleando la tabla V descrita en la Sección 8.4, ¿es posible encontrar todas las posibles soluciones óptimas de un caso, o solo se puede hallar una? De ser así, ¿en qué forma? ¿Y si no es posible, por qué no?

Problema 8.14. Un caso del problema de la mochila que se describe en la Sección 8.4 puede tener diferentes soluciones óptimas. ¿Cómo se descubriría esto? ¿Permite la tabla V recuperar más de una solución en este caso?

Problema 8.15. En la Sección 8.4 asumíamos que se disponía de n objetos, numerados desde 1 hasta n . Supongamos en cambio que se tienen n tipos de objetos disponibles, con un suministro adecuado de todos los tipos. Formalmente, esto no es más que sustituir la limitación de que x_i debe de ser 0 o 1 por la limitación menos restrictiva de que x_i debe de ser un entero no negativo. Adaptar el algoritmo de programación dinámica de la Sección 8.4 para resolver este nuevo problema.

Problema 8.16. Adaptar el algoritmo del problema 8.15 para que funcione aun cuando el número de objetos de un tipo dado sea limitado.

Problema 8.17. ¿Funciona el algoritmo de Floyd (véase la Sección 8.5) en un grafo que tenga algunas aristas cuyas longitudes sean negativas, pero que no contenga ningún ciclo negativo? Justifique su respuesta.

Problema 8.18. (Algoritmo de Warshall)

Tal como en el algoritmo de Floyd (véase la Sección 8.5) nos interesa hallar caminos dentro de un grafo. En este caso, sin embargo, la longitud de las aristas carece de interés; sólo es importante su existencia. Sea una matriz L tal que $L[i, j] = \text{verdadero}$ si existe la arista (i, j) , y $L[i, j] = \text{falso}$ en caso contrario. Deseamos encontrar una matriz D tal que $D[i, j] = \text{verdadero}$ si existe al menos un camino desde i hasta j , y $D[i, j] = \text{falso}$ en caso contrario. Adaptar el algoritmo de Floyd para este caso ligeramente distinto.

Nota: Estamos buscando el *cierre transitivo reflexivo* del grafo en cuestión.

Problema 8.19. Buscar un algoritmo significativamente mejor para el problema anterior en el caso en que la matriz L es simétrica, esto es, cuando $L[i, j] = L[j, i]$.

Problema 8.20. Esperamos (en vano) encontrar un algoritmo voraz para el problema de la multiplicación encadenada de matrices; véase la Sección 8.6. Suponga que tenemos que calcular

$$M = M_1 M_2 \cdots M_n$$

en donde la matriz M_i es $d_{i-1} \times d_i$, $1 \leq i \leq n$. Para todas y cada una de las técnicas sugeridas a continuación, proporcionar un contraejemplo en el que no funcione la técnica en cuestión.

- (a) Multiplicar primero las matrices M_i y M_{i+1} cuya dimensión común d_i sea la más pequeña, y seguir de igual manera.
- (b) Multiplicar primero las matrices M_i y M_{i-1} cuya dimensión común d_i sea la mayor, y seguir de igual manera.
- (c) Multiplicar primero las matrices M_i y M_{i-1} que minimicen el producto $d_{i-1}d_i$, y seguir de igual manera.

- (d) Multiplicar primero las matrices M_i y M_{i+1} que maximicen el producto $d_{i-1}d_i$, y seguir de igual manera.

Problema 8.21. Escriba con detalle el algoritmo para calcular los valores de m_n descrito en la Sección 8.6.

Problema 8.22. Adapte el algoritmo del problema anterior para que no sólo calcule m_n sino que además diga la forma en que hay que calcular el producto de matrices para alcanzar el valor óptimo de m_n .

Problema 8.23. ¿Cuál es el error del siguiente razonamiento sencillo? «El algoritmo para calcular los valores de m dado en la Sección 8.6 tiene que rellenar, en esencia, las entradas de poco más de la mitad de una tabla $n \times n$. Su tiempo de ejecución, por tanto, está claramente en $\Theta(n^2)$.»

Problema 8.24. Sea $T(n)$ un número de Catalán; véase la Sección 8.6. Demostrar que

$$T(n) = \frac{1}{n} \binom{2n-2}{n-1}.$$

Problema 8.25. Demostrar que el número de formas de cortar un polígono convexo de n lados en $n-2$ triángulos, empleando líneas diagonales que no se crucen, es $T(n-1)$, el $(n-1)$ -ésimo número de Catalán; véase la Sección 8.6. Por ejemplo, un hexágono se puede cortar de 14 maneras diferentes, según se muestra en la figura 8.7.

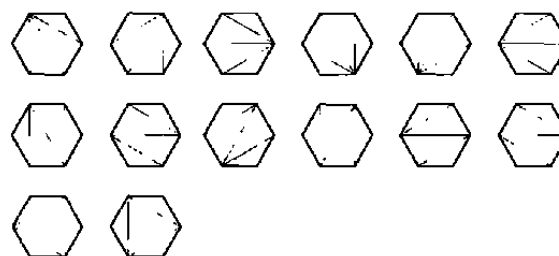


Figura 8.7. Formas de cortar un hexágono en triángulos

Problema 8.26. Mostrar la forma de calcular (i) un coeficiente binomial y (ii) la función $serie(n, p)$ de la Sección 8.1.2 empleando una función con memoria.

Problema 8.27. Mostrar la forma de resolver (i) el problema de dar la vuelta y (ii) el problema de la mochila de la Sección 8.4, empleando una función con memoria.

Problema 8.28. Considere el alfabeto $\Sigma = \{a, b, c\}$. Los elementos de Σ tienen la siguiente tabla de multiplicación, donde las filas muestran el símbolo de la izquierda y las columnas muestran el símbolo de la derecha.

	a	b	c
a	b	b	a
b	c	b	a
c	a	c	c

De esta manera, $ab = b$, $ba = c$ y así sucesivamente. Observe que la multiplicación definida por esta tabla no es conmutativa ni asociativa.

Buscar un algoritmo eficiente que examine una cadena $x = x_1x_2 \dots x_n$ de caracteres de Σ , y decida si es posible o no poner paréntesis en x de tal manera que el valor de la expresión resultante sea a . Por ejemplo, si $x = bbbba$, el algoritmo debería de responder «sí» porque $(b(bb))(ba) = a$. Esta expresión no es única. Por ejemplo, $(b(b(b(ba)))) = a$ también. En términos de n , la longitud de la cadena x , ¿cuánto tiempo requiere este algoritmo?

Problema 8.29. Modifique el algoritmo del ejemplo anterior para que devuelva el número de formas distintas de poner paréntesis en x para obtener a .

Problema 8.30. Sean u y v dos cadenas de caracteres. Deseamos transformar u en v con el menor número posible de operaciones de

los tres tipos siguientes: borrar un carácter, añadir un carácter o modificar un carácter. Por ejemplo, podemos transformar $abbac$ en $abcbc$ en tres etapas:

$abbac \rightarrow abac$ (borrar b)
 $\rightarrow ababc$ (añadir b)
 $\rightarrow abcbc$ (transformar a en c)

Demostrar que esta transformación no es óptima.

Escribir un algoritmo de programación dinámica que busque el número mínimo de operaciones necesarias para transformar u en v y que nos diga cuáles son esas operaciones. En función de las longitudes de u y v , ¿cuánto tiempo requiere este algoritmo?

Problema 8.31. Se dispone de n objetos que es necesario ordenar empleando las relaciones « $<$ » y « $=$ ». Por ejemplo, con tres objetos se tienen 13 ordenaciones posibles

$a = b = c$ $a = b < c$ $a < b = c$ $a < b < c$ $a < c < b$
 $a = c < b$ $b < a = c$ $b < a < c$ $b < c < a$ $b = c < a$
 $c < a = b$ $c < a < b$ $c < b < a$

Dar un algoritmo de programación dinámica que pueda calcular, como función de n , el número de ordenaciones posibles. El algoritmo debe necesitar un tiempo que esté en $O(n^2)$, y un espacio que esté en $O(n)$.

Problema 8.32. A lo largo de un río hay n aldeas. En cada aldea se puede alquilar una canoa que se puede devolver en cualquier otra aldea que esté a favor de la corriente (es casi imposible remar en contra de la corriente). Para todo posible punto de partida i , y para todo posible punto de llegada j , se conoce el coste de un alquiler desde i hasta j . Sin embargo, puede ocurrir que el coste del alquiler desde i hasta j sea mayor que el coste total de una serie de alquileres más breves.

En tal caso, se puede devolver la primera canoa en alguna aldea k entre i y j , y seguir camino en una segunda canoa. No hay cargos adicionales por cambiar de canoa de esta manera.

Dar un algoritmo eficiente para determinar el coste mínimo de un viaje en canoa desde todos los posibles puntos de partida i hasta todos los posibles puntos de llegada j . En términos de n , ¿cuánto tiempo requiere su algoritmo?

Problema 8.33. Cuando se describían los árboles binarios de búsqueda en la Sección 5.5, mencionamos que era buena idea mantenerlos equilibrados. Esto es cierto siempre y cuando la probabilidad de acceder a todos los nodos sea la misma. Sin embargo si se accede a algunos nodos con más frecuencia que a otros, un árbol desequilibrado puede ofrecer un mejor rendimiento medio. Por ejemplo, el árbol de la figura 8.8 es mejor que el de la figura 5.9 si nos interesa minimizar el número medio de comparaciones con el árbol, y si se accede a los nodos con las siguientes probabilidades.

Nodo	6	12	18	20	27	34	35
Prob.	0,2	0,25	0,05	0,1	0,05	0,3	0,05

Más generalmente, supongamos que tenemos un conjunto ordenado $c_1 < c_2 < \dots < c_n$ de n claves distintas. La probabilidad de que una consulta se refiera a la clave c_i es p_i , $1 \leq i \leq n$. Suponga por sencillez que todas las consultas se refieren a claves que están en el árbol de búsqueda, así que $\sum_{i=1}^n p_i = 1$. Recuerde que la profundidad de la raíz de un árbol es 0, la profundidad de sus hijos es 1, y así sucesivamente. Si se almacena la clave c_i en un nodo de profundidad d_i , entonces se necesitan $d_i + 1$ comparaciones para hallarla. Por tanto para un árbol dado, el número medio de comparaciones es:

$$C = \sum_{i=1}^n p_i (d_i + 1)$$

Por ejemplo, el número medio de comparaciones necesarias con el árbol de la Figura 8.8 es

$$0,3 + (0,25 + 0,05) \times 2 + (0,2 + 0,1) \times 3 + (0,05 + 0,05) \times 4 = 2,2.$$

- Calcular el número medio de comparaciones con el árbol de la figura 5.9, y verificar que el árbol de la figura 8.8 es mejor.
- El árbol de la figura 8.8 se obtuvo a partir de las probabilidades dadas empleando un algoritmo sencillo. ¿Puede adivinar cuál es?
- Buscar otro árbol de búsqueda para el mismo conjunto de claves, que sea todavía más eficiente en el caso promedio que el de la figura 8.8. ¿Qué conclusión acerca del diseño de algoritmos pone esto de manifiesto?

Problema 8.34. Continuando con el problema 8.33, diseñar un algoritmo de programación dinámica para hallar un árbol binario de búsqueda óptimo para un conjunto de claves con probabilidades de acceso dadas. ¿Cuánto tiempo requiere su algoritmo en función del número de claves? Aplique su algoritmo al caso dado en el problema 8.33.

Sugerencia: En todo árbol de búsqueda en el que los nodos que contienen las claves $c_i, c_{i+1}, \dots, c_j$ forman un subárbol, sea C_{ij} el mínimo número medio de accesos que se hacen a esos nodos. En particular, $C_{1,n}$ es el número medio de accesos causados por una consulta a un árbol binario de búsqueda óptimo y $C_{ii} = p_i$ para todo i , $1 \leq i \leq n$. Invocar el principio de optimalidad para afirmar que

$$C_{ij} = \min_{i \leq k \leq j} (C_{i,k-1} + C_{k+1,j}) + \sum_{k=i}^j p_k$$

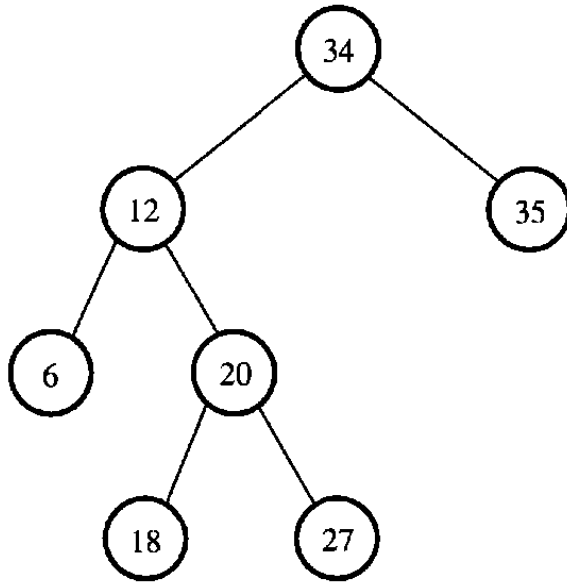


Figura 8.8. Otro árbol de búsqueda

cuando $i < j$. Dar un algoritmo de programación dinámica para calcular C_{ij} para todos los $0 \leq i \leq j \leq n$, y un algoritmo para hallar el árbol binario de búsqueda óptimo a partir de los C_{ij} .

Problema 8.35. Resolver de nuevo el problema 8.34. Esta vez, el algoritmo para calcular un árbol binario de búsqueda óptimo para un conjunto de n claves debe funcionar en un tiempo que esté en $O(n^3)$.

Sugerencia: Demuestre primero que $r_{i-1} \leq r_{ij} \leq r_{i+1,j}$ para todos los $1 \leq i \leq j \leq n$, en donde r_{ij} es la raíz de un subárbol de búsqueda óptimo que contiene a $c_i, c_{i+1}, \dots, c_j$ para todos los $1 \leq i \leq j \leq n$ (los empates se deshacen arbitrariamente) y $r_{i,i-1} = i, 1 \leq i \leq n$.

Problema 8.36. Como función de n , ¿cuántos árboles binarios de búsqueda existen para n claves diferentes?

Problema 8.37. Recuerde que la función de Ackermann $A(m, n)$ definida en el problema 5.38 crece con suma rapidez. Dar un algoritmo de programación dinámica para calcularla. El algoritmo debe constar simplemente de dos bucles anidados, y no se admite la recursión. Además, se tiene la limitación de utilizar un espacio que esté en $O(m)$ para calcular $A(m, n)$. Sin embargo, se puede suponer que una palabra de almacenamiento puede contener un entero arbitrariamente grande.

Sugerencia: Utilice dos vectores *valor*[0..*m*] e *índice*[0..*m*] y asegúrese de que sea *valor*[*i*] = $A(i, \text{índice}[i])$ al final de cada pasada por el bucle interno.

8.10 REFERENCIAS Y TEXTOS MÁS AVANZADOS

Hay varios libros que abarcan la programación dinámica. Mencionaremos únicamente a Bellman (1957), Bellman y Dreyfus (1962), Nemhauser (1966) y Laurière (1979).

El algoritmo de la Sección 8.2 para devolver cambio se discute en Wright (1975) y en Chang y Korsh (1976). Para más ejemplos de resolución del problema de la mochila empleando programación dinámica, véase Hu (1981).

El algoritmo de la Sección 8.5 para calcular todos los caminos más cortos se debe a Floyd (1962). Se conoce un algoritmo teóricamente más eficiente: Fredman (1976) muestra la forma de resolver el problema en un tiempo que está en $O(n^3 \sqrt[3]{\log \log n} / \log n)$. La solución del problema 8.18 se consigue mediante el algoritmo de Warshall (1962). Tanto el algoritmo de Warshall como el de Floyd son en esencia iguales al anterior de Kleene (1956) para determinar la expresión regular que corresponde a un autómata finito; véase Hopcroft y Ullman (1979). Todos estos algoritmos, con la excepción del de Fredman, se unifican en Tarjan (1981).

El algoritmo de la Sección 8.6 para la multiplicación encadenada de matrices se describe en Godbole (1973); un algoritmo más eficiente, que puede resolver el problema en un tiempo que está en $O(n \log n)$ se encuentra en Hu y Shing (1982, 1984). Los números de Catalán se describen

en muchos lugares, incluyendo Sloane (1973) y Purdom y Brown (1985). Las funciones con memoria se introducen en Michie (1968); para más detalles véase Marsh (1970).

El problema 8.25 se discute en Sloane (1973). Una solución del problema 8.30 se da en Wagner y Fischer (1974). El problema 8.31 se les ocurrió a los autores mientras corregían un examen que contenía una pregunta similar al problema 3.21: ¡sentíamos curiosidad por saber qué proporción de todas las posibles respuestas estaba representada por las 69 respuestas distintas sugeridas por los alumnos!

El problema 8.34 acerca de la construcción de árboles binarios de búsqueda óptimos proviene de Gilbert y Moore (1959), lugar en que se amplía a la posibilidad de que la clave solicitada no se encuentre en el árbol. La mejora considerada en el problema 8.35 proviene de Knuth (1971, 1973), pero una solución más sencilla y más general se da en Yao (1980), que también da una condición suficiente para que ciertos algoritmos de programación dinámica se ejecutan en un tiempo cúbico, sean transformables de forma automática en algoritmos de tiempo cuadrático. El árbol de búsqueda óptimo para las 31 palabras más comunes del inglés se compara en Knuth (1973) con el árbol que se obtiene utilizando el algoritmo voraz evidente que se sugiere en el problema 8.33 (b).

Entre los algoritmos importantes de programación dinámica que no hemos mencionado se encuentran el de Kasimi (1965) y Younger (1967) que requiere un tiempo cúbico para efectuar el análisis sistemático de cualquier lenguaje libre de contexto (véase Hopcroft y Ullman 1979), y el de Held y Karp (1962) que resuelve el problema del viajante (véanse las Secciones 12.5.2 y 13.1.2) en un tiempo que está en $O(n^2 2^n)$, mucho mejor que el tiempo en $\Omega(n!)$ que requiere el algoritmo menos sofisticado.

Exploración de grafos

Existe una gran cantidad de problemas que se pueden formular en términos de grafos. Por ejemplo, hemos visto el problema del camino más corto y el problema del árbol de recubrimiento mínimo. Para resolver estos problemas, es frecuente que sea necesario examinar todos los nodos o todas las aristas de un grafo. En algunas ocasiones, la estructura del problema es tal que sólo necesitamos visitar algunos de los nodos, o bien algunas de las aristas. Hasta el momento, los algoritmos que hemos visto imponían implícitamente un orden en estos recorridos: se trataba de visitar el nodo más próximo, o la arista más corta, y así sucesivamente. En este capítulo presentamos algunas técnicas generales que se pueden utilizar cuando no se requiere ningún orden concreto en nuestro recorrido.

9.1 GRAFOS Y JUEGOS: INTRODUCCIÓN

Consideremos el juego siguiente. Se trata de una de las muchas variantes de Nim, que también se conoce con el nombre de juego de Marienbad. Inicialmente, hay un montón de cerillas en la mesa, entre los dos jugadores. El primer jugador puede quitar tantas cerillas como desee, salvo que tiene que tomar una como mínimo, y debe dejar una como mínimo. En el montón inicial tiene que haber un mínimo de dos cerillas. En lo sucesivo, el jugador al que corresponda el turno tiene que tomar como mínimo una cerilla, y como máximo el doble del número de cerillas que acaba de tomar su contrincante. Gana el jugador que se quede con la última cerilla. No hay empates.

Supongamos que en una cierta fase del juego se encuentra usted delante de un montón de cinco cerillas. Su oponente acaba de tomar dos cerillas y le toca jugar a usted. Puede tomar una, dos, tres o cuatro cerillas, pero no puede tomar las cinco porque las reglas prohíben tomar más del doble de las que acaba de tomar el contrario. ¿Qué debe hacer?

La mayoría de las personas que juegan a estos juegos empiezan a revisar las posibilidades en sus mentes: «Si cojo cuatro cerillas, le dejo una sola al contrario, así

que la toma él y gana; si cojo tres, dejo dos para mi oponente, y vuelve a tomarlas y gana; si tomo dos, pasa lo mismo; pero si me quedo con una sola, entonces tendrá delante cuatro cerillas y sólo podrá tomar una o dos. En este caso, no gana de inmediato, así que ciertamente es mi mejor movimiento». Sin más que examinar un movimiento por anticipado, en esta situación sencilla el jugador puede determinar lo que debe hacer a continuación. En un ejemplo más complicado, podrían ser posibles varias jugadas. Para seleccionar la mejor, el jugador quizá tenga que considerar no sólo la situación posterior a su propia jugada, sino más adelante para ver cómo puede contraatacar su oponente en cada una de las jugadas posibles. Y entonces quizá tenga que pensar acerca de su propia jugada después de cada posible contraataque, y así sucesivamente.

Para formalizar este proceso de pensamiento anticipatorio, representamos el juego mediante un grafo dirigido. Cada nodo del grafo corresponde a una situación del juego, y cada arista corresponde a una jugada que nos lleva de una situación a otra. (En algunos contextos, por ejemplo en los informes de juegos de ajedrez, cada jugada consta de una acción por parte de un jugador junto con la respuesta por parte de su oponente. En tales contextos, el término *media jugada* se emplea para denotar la acción de un solo jugador. En este libro, nos limitaremos a la terminología sencilla y llamaremos *jugada* a la acción de cualquiera de los jugadores.) Una situación del juego no sólo se especifica por el número de cerillas que queden en la mesa. También es necesario conocer el límite superior del número de cerillas que se pueden quitar en la jugada siguiente. Sin embargo, no es necesario conocer a quién le toca jugar, puesto que las reglas son las mismas para ambos jugadores (a diferencia de juegos tales como cacos y polis, en que los jugadores tienen objetivos y fuerzas diferentes). Los nodos del grafo que corresponde a este juego son por tanto parejas de la forma $\langle i, j \rangle$. En general, $\langle i, j \rangle$ con $1 \leq j \leq i$ indica que en la mesa quedan i cerillas, y que en la jugada siguiente se puede tomar cualquier número de cerillas entre 1 y j . Las aristas que salen de esta situación, esto es, las jugadas que se pueden hacer, van a los j nodos $\langle i-k, \min(2k, i-k) \rangle$, $1 \leq k \leq j$. El nodo que corresponde a la posición inicial de un juego con n cerillas es $\langle n, n-1 \rangle$, $n \geq 2$. La posición $\langle 0, 0 \rangle$ da lugar a perder el juego: si un jugador se encuentra en esta situación cuando le toca mover, su oponente acaba de tomar la última cerilla, y ha ganado.

La figura 9.1 muestra parte del grafo correspondiente al juego. De hecho, se trata de la parte del grafo que necesita el jugador del ejemplo anterior, que se enfrenta a un montón de cinco cerillas de las cuales puede tomar cuatro: es la situación $\langle 5, 4 \rangle$. No aparece ninguna situación de la forma $\langle i, 0 \rangle$ salvo por la situación de derrota, $\langle 0, 0 \rangle$. Estas situaciones no se pueden alcanzar durante una partida, así que no tienen interés. De forma análoga, los nodos $\langle i, j \rangle$ con j impar y $j < i-1$ no se pueden alcanzar desde ninguna situación inicial, así que también se omiten. Tal como explicaremos dentro de poco, los nodos cuadrados representan situaciones de derrota y los nodos redondos denotan situaciones de victoria. Las aristas gruesas corresponden a jugadas victoriosas: desde una situación de victoria, ganaremos al seleccionar una de las aristas gruesas. En las situaciones de derrota no hay aristas

gruesas que salgan de ellas, lo cual corresponde al hecho de que en estas situaciones no hay ninguna jugada que nos dé la victoria. Observamos que el jugador que tiene que mover primero en un juego con dos, tres o cinco cerillas no tiene ninguna estrategia que le lleve a la victoria, mientras que en el juego de cuatro cerillas sí que dispone de una estrategia adecuada.

Para decidir cuáles son las situaciones de victoria y cuáles son las situaciones de derrota, partimos de la situación de derrota $\langle 0, 0 \rangle$ y retrocedemos. Este nodo no tiene sucesor, y el jugador que se encuentre en esta situación perderá la partida. En cualquiera de los nodos $\langle 1, 1 \rangle$, $\langle 2, 2 \rangle$ o $\langle 3, 3 \rangle$, el jugador puede hacer una jugada que pondrá a su oponente en una situación de derrota. Estos tres nodos, por tanto, son los nodos de victoria. Partiendo de $\langle 2, 1 \rangle$, la única jugada posible es pasar a $\langle 1, 1 \rangle$. En la situación $\langle 2, 1 \rangle$, por tanto, el jugador está obligado a poner a su oponente en una situación de victoria, así que $\langle 2, 1 \rangle$ es una situación de derrota. A la posición de derrota $\langle 3, 2 \rangle$ se le puede aplicar un argumento similar. Hay dos jugadas posibles, pero ambas dejan al oponente en una situación de victoria, así que $\langle 3, 2 \rangle$ es una situación de derrota. Partiendo de $\langle 4, 2 \rangle$ o de $\langle 4, 3 \rangle$ existe una jugada que pone al oponente en una situación de derrota, a saber, $\langle 3, 2 \rangle$; por tanto, estos dos nodos son situaciones de victoria. Por último, las cuatro jugadas posibles desde $\langle 5, 4 \rangle$ dejan todas ellas al contrario en situación de victoria, así que $\langle 5, 4 \rangle$ es una situación de derrota.

En un grafo más grande, este proceso de etiquetado de situaciones de victoria y de derrota se puede prolongar hacia atrás cuanto se quiera. Las reglas que hemos estado aplicando se pueden resumir como sigue: una situación será una situación

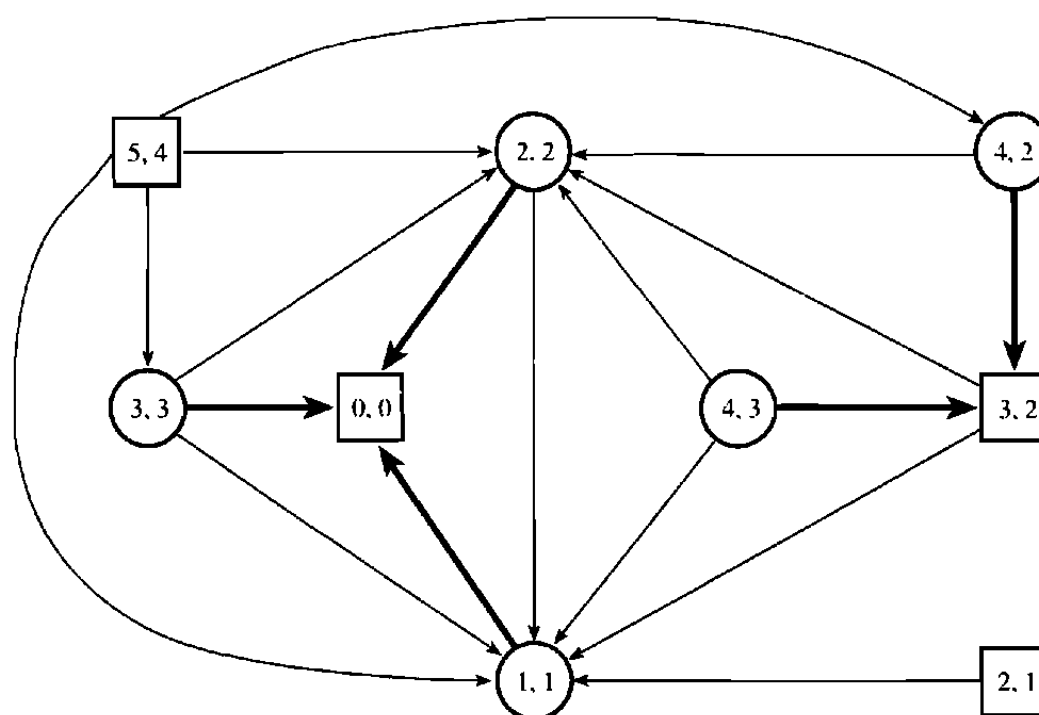


Figura 9.1. Parte del grafo de un juego

de victoria si al menos *uno* de sus sucesores es una situación de derrota, porque entonces el jugador puede jugar para poner a su oponente en una situación de derrota; una posición es de derrota si *todos* sus sucesores son situaciones de victoria, porque el jugador no puede evitar dejar a su oponente en una situación de victoria. El algoritmo siguiente, por tanto, determina si una situación es de victoria o de derrota:

```
función ganarec(i, j)
  {Devuelve verdadero si y sólo si la situación  $\langle i, j \rangle$  es de victoria,
  suponemos que  $0 \leq j \leq i$ }
  para  $k \leftarrow 1$  hasta  $j$  hacer
    si no ganarec( $i - k$ ,  $\text{mín}(2k, i - k)$ )
      entonces devolver verdadero
  devolver falso
```

Este algoritmo adolece de los mismos defectos que el algoritmo *Fibrec* de la Sección 2.7.5: calcula el mismo valor una y otra vez. Por ejemplo, *ganarec*(5, 4) proporciona *falso*, después de haber llamado sucesivamente a *ganarec*(4, 2), *ganarec*(3, 3), *ganarec*(2, 2) y *ganarec*(1, 1), pero *ganarec*(3, 3) también llama a *ganarec*(2, 2) y a *ganarec*(1, 1).

Para eliminar esta ineficiencia hay dos enfoques evidentes. El primero, que consiste en aplicar programación dinámica, nos obliga a crear una matriz booleana G tal que $G[i, j] = \text{verdadero}$ si y sólo si $\langle i, j \rangle$ es una situación de victoria. Como es habitual con la programación dinámica, procederemos de forma ascendente, calculando $G[r, s]$ para $1 \leq s \leq r < i$, así como los valores de $G[i, s]$ para $1 \leq s < j$, antes de calcular $G[i, j]$:

```
procedimiento ganadin(n)
  { Para cada  $1 \leq j \leq i \leq n$  da a  $G[i, j]$  el valor verdadero
  si y sólo si la situación  $\langle i, j \rangle$  es de victoria}
   $G[0, 0] \leftarrow \text{falso}$ 
  para  $i \leftarrow 1$  hasta  $n$  hacer
    para  $j \leftarrow 1$  hasta  $i$  hacer
       $k \leftarrow 1$ 
      mientras  $k < j$  y  $G[i - k, \text{mín}(2k, i - k)]$  hacer
         $k \leftarrow k + 1$ 
       $G[i, j] \leftarrow \text{no } G[i - k, \text{mín}(2k, i - k)]$ 
```

En este contexto la programación dinámica nos lleva a calcular de forma innecesaria algunas entradas de la matriz G que no se llegan a utilizar nunca. Por ejemplo, sabemos que $\langle 15, 14 \rangle$ es una situación de victoria en cuanto descubrimos que su segundo sucesor $\langle 13, 4 \rangle$ es una situación de derrota. Ya no nos interesa saber si el siguiente sucesor, $\langle 12, 6 \rangle$ es una situación de victoria o de derrota. De hecho, sólo son útiles 28 nodos cuando calculamos $G[15, 14]$, a pesar de que el algoritmo de progra-

mación dinámica determina 121 de estos nodos. Se puede ahorrar aproximadamente la mitad de este trabajo si no se calcula $G[i, j]$ cuando j es impar y $j < i - 1$, puesto que estos nodos nunca son interesantes, pero no hay ninguna razón «ascendente» para no calcular $G[12, 6]$. Como de costumbre, las cosas empeoran a medida que va creciendo el caso: para resolver el juego con 248 cerillas basta con explorar 1.000 nodos, y sin embargo *ganadin* examina más de treinta veces esta cantidad.

El algoritmo recursivo dado anteriormente es ineficiente porque recalcula varias veces el mismo valor. Como consecuencia de su naturaleza descendente, sin embargo, nunca calcula valores innecesarios. Una solución que combina las ventajas de ambos algoritmos consiste en utilizar una función con memoria; véase la Sección 8.8. Esto necesita recordar qué nodos hemos visitado ya durante el cálculo recursivo, empleando una matriz booleana *conocido* $[0..n, 0..n]$, donde n es una cota superior del número de cerillas que se van a utilizar. Las inicializaciones necesarias son las siguientes:

```

 $G[0, 0] \leftarrow \text{falso}, \text{conocido}[0, 0] \leftarrow \text{verdadero}$ 
para  $i \leftarrow 1$  hasta  $n$  hacer
    para  $j \leftarrow 1$  hasta  $i$  hacer
         $\text{conocido}[i, j] \leftarrow \text{falso}$ 

```

A continuación, para descubrir si $\langle i, j \rangle$ es una situación de victoria o de derrota, se invoca la función siguiente:

```

función nim( $i, j$ )
    {Para todo  $1 \leq j \leq i \leq n$ , proporciona verdadero
    si y sólo si la situación  $\langle i, j \rangle$  es ganadora}
    si  $\text{conocido}[i, j]$  entonces devolver  $G[i, j]$ 
     $\text{conocido}[i, j] \leftarrow \text{verdadero}$ 
    para  $k \leftarrow 1$  hasta  $j$  hacer
        si no nim( $i - k, \min(2k, i - k)$ ) entonces
             $G[i, j] \leftarrow \text{verdadero}$ 
            devolver verdadero
     $G[i, j] \leftarrow \text{falso}$ 
    devolver falso

```

A primera vista, no hay ninguna razón particular para preferir esta aproximación frente a la programación dinámica, porque en todo caso es preciso invertir un tiempo para inicializar toda la matriz *conocido* $[0..n, 0..n]$. Sin embargo, la iniciación virtual (descrita en la Sección 5.1) nos permite evitar esto y obtener una ganancia de eficiencia.

El juego que hemos considerado hasta el momento es tan sencillo que se puede resolver sin utilizar el grafo asociado; véase el Problema 9.5. Sin embargo, estos mismos principios se pueden aplicar a muchos otros juegos de estrategia. Como antes, un nodo de un grafo dirigido corresponde a una situación particu-

lar del juego, y una arista corresponde a una jugada o movimiento válido entre dos situaciones. El grafo es infinito si no hay un límite *a priori* del número de situaciones posibles en el juego. Por sencillez, supondremos que el juego se desarrolla entre dos jugadores que se van turnando, que las reglas son las mismas para los dos jugadores (decimos que el juego es *simétrico*) y que la casualidad no interviene en el resultado (el juego es *determinista*). Las ideas que presentamos se pueden adaptar fácilmente a contextos más generales. Además suponemos que ningún caso del juego puede tener una duración infinita y que ninguna situación del juego ofrece un número infinito de jugadas válidas para el jugador a quien corresponda ese turno. En concreto, algunas situaciones del juego, llamadas *situaciones terminales*, no ofrecen jugadas válidas, y por tanto algunos nodos del grafo no poseen sucesores.

Para determinar una estrategia ganadora en un juego de esta clase, asociamos a cada nodo del grafo una etiqueta seleccionada dentro del conjunto *victoria*, *derrota* y *tablas*. La etiqueta alude a la perspectiva de un jugador que vaya a pasar a la situación correspondiente, suponiendo que ninguno de los jugadores cometa un error. Las etiquetas se asignan de manera sistemática de la forma siguiente (en el ejemplo sencillo dado anteriormente, no eran posibles las tablas, así que la etiqueta *tablas* no llegó a utilizarse nunca, y las reglas establecidas no estaban completas):

1. Se etiquetan las posiciones terminales. Las etiquetas asignadas dependen del juego en cuestión. Para la mayoría de los juegos, si nos encontramos en una situación terminal, entonces no podemos hacer ningún movimiento, y hemos perdido. Si embargo, esto no siempre es así. Por ejemplo, si uno no puede mover por bloqueo mutuo en el juego de ajedrez, el resultado son tablas. Y además muchos juegos de Nim vienen por parejas, en una de las cuales gana el jugador que toma la última cerilla, y otra (que se llama versión *misère* del juego) en la que pierde el jugador que se quede con la última cerilla.
2. Una situación no terminal es una situación de victoria si al menos uno de los sucesores es una situación de derrota, porque el jugador al que corresponda el turno puede dejar a su oponente en esta situación de derrota.
3. Una situación no terminal es una situación de derrota si *todos* sus sucesores son situaciones de victoria, porque el jugador al que corresponda el turno no puede evitar dejar a su oponente en una de estas posiciones de victoria.
4. Cualquier otra situación no terminal da lugar a tablas. En este caso, entre los sucesores debe contarse al menos una situación de tablas, posiblemente con algunas situaciones adicionales que sean de victoria. El jugador al que corresponda el turno puede evitar dejar a su oponente en una situación de victoria, pero no puede forzarle a una situación de derrota.

Una vez que se han asignado las etiquetas, puede uno leer en el grafo la estrategia para la victoria.

En principio, esta técnica es aplicable incluso a juegos tan complejos como el ajedrez. A primera vista, el grafo asociado al ajedrez parece contener ciclos, puesto que si dos situaciones u y v de las piezas difieren tan sólo en el desplazamiento legal de un caballo, por ejemplo, y el rey no está en jaque, entonces podemos mover igualmente bien desde u hasta v y desde v hasta u . Sin embargo, este problema desaparece tras un examen más detallado. En la variante de Nim que se utilizaba como ejemplo anteriormente, una situación se define no sólo por el número de cerillas que hay en la mesa, sino también por una información invisible que da el número de cerillas que se pueden tomar en la próxima jugada. De manera similar, una situación de ajedrez no queda definida simplemente por la posición de las piezas. También necesitamos saber a quién le corresponde jugar, cuáles son los caballos y reyes que se han movido desde el principio del juego (para saber si es legal enrocar) y si algún peón acaba de avanzar dos casillas (para saber si es posible una captura *al paso*). También hay reglas diseñadas explícitamente para evitar que una partida no acabe nunca. Por ejemplo, se dice que una partida ha quedado en tablas después de que se haya producido un cierto número de jugadas en las que no se haya producido ninguna acción irreversible (el desplazamiento de un peón o una captura). Gracias a estas reglas y otras similares, no hay ciclos en el grafo correspondiente al ajedrez. Sin embargo, esto nos obliga a incluir cosas tales como el número de jugadas habido desde la última acción irreversible en la información que define una situación.

Adaptando las reglas generales que se dan más arriba, podemos etiquetar los nodos como situaciones de victoria para las blancas, situaciones de victoria para las negras, o bien tablas. Una vez construido, este grafo nos permite en principio jugar una partida perfecta de ajedrez, esto es, ganar siempre que sea posible, y perder sólo cuando sea inevitable. Desafortunadamente —o quizá afortunadamente para el juego de ajedrez— el grafo contiene tantos nodos que es totalmente inviable explorarlo en su totalidad, ni siquiera con las computadoras más rápidas existentes en la actualidad. Lo mejor que se puede hacer es explorar el grafo en las proximidades de la situación actual, para ver cómo puede evolucionar la situación, del mismo modo que un novato que razona «si hago esto, él responderá de esta manera, y entonces puedo hacer esto», y así sucesivamente. Sin embargo, ni siquiera esta técnica carece de sutilezas. ¿Debemos examinar *todas* las posibilidades ofrecidas por la situación actual, y para cada una de ellas, *todas* las posibilidades de respuesta? O más bien, ¿debemos seleccionar una línea de ataque prometedor y seguirla durante varias jugadas para ver adónde nos lleva? Las distintas estrategias de búsqueda pueden llevarnos a resultados muy diferentes, como veremos en breve.

Si no tenemos esperanzas de poder explorar todo el grafo para el juego de ajedrez, entonces tampoco podemos tener esperanzas de construirlo y almacenarlo. Lo más que podemos esperar es construir ciertas partes del grafo sobre la marcha, guardándolas si parecen interesantes y descartándolas en caso contrario. Por tanto, a lo largo de este capítulo utilizaremos la palabra «grafo» de dos maneras diferentes.

Por una parte, un grafo puede ser una estructura de datos en la memoria de una computadora. En este caso, los nodos están representados por un cierto número de *bytes*, y las aristas se representan mediante punteros. Las operaciones que hay que efectuar son bastante concretas: «marcar un nodo» significa modificar un *bit* de memoria, «hallar un nodo vecino» significa seguir un puntero, y así sucesivamente.

En otras ocasiones, el grafo solamente existe de forma implícita, tal como sucede cuando exploramos el grafo abstracto correspondiente al juego de ajedrez. Este grafo nunca llega a existir en la memoria de la máquina. La mayor parte del tiempo, lo único que se tiene es una representación de la situación actual (esto es, del nodo que estamos visitando, porque como hemos visto los nodos corresponden a situaciones de las piezas más una cierta información adicional), y posiblemente las representaciones de un pequeño número de situaciones diferentes. Por supuesto, también conocemos las reglas del juego en cuestión. En este caso, «marcar un nodo» significa tomar las medidas adecuadas que nos permiten reconocer una situación que ya hayamos visto, o evitar llegar dos veces a la misma situación. «Encontrar un nodo vecino» significa cambiar la situación actual realizando un único movimiento válido, porque si es posible pasar de una situación a otra efectuando un único movimiento, entonces existe una arista en el grafo implícito que une los dos nodos correspondientes.

Se pueden aplicar unas consideraciones exactamente iguales cuando se explora cualquier grafo de gran tamaño, tal como se verá especialmente en la Sección 9.6. Sin embargo, tanto si el grafo es una estructura de datos, como si es una abstracción que nunca podemos manipular en su conjunto, las técnicas que se utilizan para recorrerlo son esencialmente las mismas. En este capítulo, por tanto, no distinguiremos entre ambos casos.

9.2 RECORRIDOS DE ÁRBOLES

No vamos a invertir demasiado tiempo en describir detalladamente la forma en que se recorren los árboles. Nos limitamos a recordar al lector que en el caso de los árboles binarios se suelen emplear tres técnicas. Si en cada nodo del árbol visitamos primero el nodo, después el subárbol izquierdo y por último el subárbol derecho, entonces estamos recorriendo el árbol en *preorden*; si visitamos el subárbol izquierdo, luego el nodo y finalmente el subárbol derecho, estamos recorriendo el árbol en *orden infijo*; y si visitamos primero el subárbol izquierdo, luego el subárbol derecho y por último el nodo, entonces estamos recorriendo el árbol en *postorden*. El *preorden* y el *postorden* se generalizan de forma evidente para árboles no binarios.

Estas tres técnicas exploran el árbol de izquierda a derecha. Hay tres técnicas correspondientes que exploran el árbol de derecha a izquierda. La implementación de cualquiera de estas técnicas utilizando recursión es inmediata.

Lema 9.2.1. *Para cada una de las seis técnicas mencionadas, el tiempo $T(n)$ que se necesita para explorar un árbol binario que contenga n nodos se encuentra en $\Theta(n)$*

Demostración. Supongamos que visitar un nodo requiere un tiempo que está en $O(1)$, esto es, que el tiempo requerido está acotado superiormente por alguna constante c . Sin pérdida de generalidad, podemos suponer que $c \geq T(0)$. Supongamos además que tenemos que explorar un árbol que contiene n nodos, con $n > 0$; uno de estos nodos es la raíz, así que si g de ellos se encuentran en el subárbol izquierdo, entonces hay $n - g - 1$ en el subárbol derecho. Entonces

$$T(n) \leq \max_{0 \leq g \leq n-1} (T(g) + T(n - g - 1) + c), \quad n > 0.$$

Esto es cierto sea cual fuere el orden en que se exploran los subárboles izquierdo y derecho y la raíz. Demostramos por inducción constructiva que $T(n) \leq an + b$, donde a y b son constantes adecuadas, desconocidas por el momento. Si tomamos $b \geq c$, la hipótesis es verdadera para $n = 0$ porque $c \geq T(0)$. Como paso de inducción, sea $n > 0$ y supongamos que la hipótesis es cierta para todo m , con $0 \leq m \leq n - 1$. Entonces

$$\begin{aligned} T(n) &\leq \max_{0 \leq g \leq n-1} (T(g) + T(n - g - 1) + c) \\ &\leq \max_{0 \leq g \leq n-1} (ag + b + a(n - g - 1) + b + c) \\ &\leq an + 3b - a \end{aligned}$$

Entonces, supuesto que tomemos $a \geq 2b$, tenemos $T(n) \leq an + b$, así que la hipótesis es cierta también para $m = n$. Esto demuestra que $T(n) \leq an + b$ para todo $n \geq 0$, y por tanto que $T(n)$ está en $O(n)$.

Por otra parte, está claro que $T(n)$ está en $\Omega(n)$ puesto que se visitan todos y cada uno de los n nodos. Por tanto $T(n)$ está en $\Theta(n)$.

9.2.1 Precondicionamiento

Si tenemos que resolver varios casos parecidos del mismo problema, puede merecer la pena invertir una cierta cantidad de tiempo en calcular resultados auxiliares que puedan ser utilizados en el futuro para acelerar la resolución de cada caso. Esto es el *precondicionamiento*. Informalmente, sea a el tiempo que se necesita para resolver un caso típico cuando *no* se dispone de información auxiliar, sea b el tiempo que se necesita para resolver un caso típico cuando *sí* se dispone de información auxiliar, y sea p el tiempo que se necesita para calcular esta información auxiliar. Para resolver n casos típicos se necesita un tiempo na , sin precondicionamiento y un tiempo $p + nb$ si se emplea precondicionamiento. Siempre que $b < a$, resulta ventajoso utilizar el precondicionamiento cuando $n > p/(a - b)$.

Desde este punto de vista, el algoritmo de programación dinámica para devolver cambio que se daba en la Sección 2.8 se puede ver como un ejemplo de precondicionamiento. Una vez que se han calculado los valores c_m necesarios, podemos dar rápidamente el cambio siempre que sea necesario.

Aun cuando sean pocos los casos que hay que resolver, el cálculo anticipado de información auxiliar puede resultar útil. Supongamos que algunas veces tenemos

que resolver un cierto caso de un conjunto grande de casos posibles. Cuando se necesita una solución, es preciso proporcionarla muy rápidamente, por ejemplo para asegurar una respuesta suficientemente rápida para una aplicación de tiempo real. En este caso, puede no ser práctico calcular y almacenar por anticipado la solución de todos los casos relevantes. Por otra parte, quizá sea posible calcular y almacenar la información auxiliar suficiente para acelerar la resolución de cualquier caso que pudiera aparecer. Una aplicación de preconditionamiento como ésta puede tener importancia práctica aun cuando sólo se resuelva un caso crucial en toda la vida del sistema: podría ser el caso que nos permita, por ejemplo, detener un reactor desbocado.

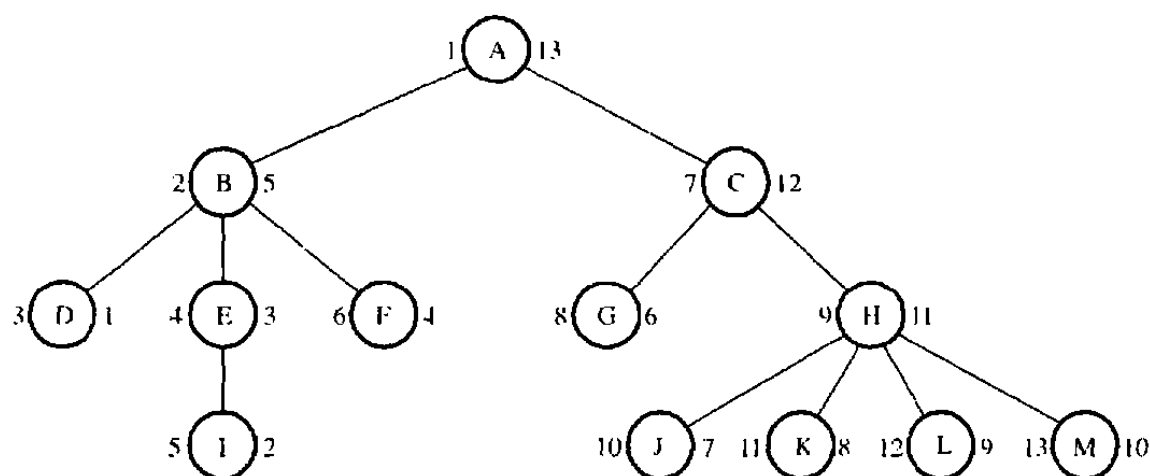


Figura 9.2. Un árbol con raíz, con las numeraciones de preorden y de postorden

Como segundo ejemplo de esta técnica usaremos el problema de determinar los antecesores dentro de un árbol con raíz. Sea T un árbol con raíz, no necesariamente binario. Decimos que un nodo v de T es un *antecesor* del nodo w si v está en el camino que va desde w hasta la raíz de T . En particular, todo nodo es su propio antecesor, y la raíz es un antecesor de todos los nodos. (Aquellos que disfruten con las definiciones recursivas pueden preferir la siguiente: todo nodo es su propio antecesor y, recursivamente, es antecesor de todos los nodos de los cuales sus hijos sean antecesores.) El problema es, dado un par de nodos (v, w) de T , determinar si v es o no un antecesor de w . Si T contiene n nodos, toda solución directa de este caso requiere un tiempo que está en $\Omega(n)$ en el caso peor. Sin embargo, es posible efectuar un preconditionamiento de T en un tiempo que está en $\Theta(n)$ para que en lo sucesivo se pueda resolver cualquier caso particular del problema de la ascendencia en un tiempo constante.

Ilustraremos esta idea usando el árbol de la figura 9.2. Contiene 13 nodos. Para preconditionar el árbol, lo recorreremos primero en preorden y después en post-

orden, numerando secuencialmente los nodos a medida que los visitamos. Para un nodo v , sea $prenum[v]$ el número asignado a v cuando se recorre el árbol en preorden, y sea $postnum[v]$ el número que se le asigna durante el recorrido en postorden. En la figura 9.2, estos números aparecen a la izquierda y a la derecha del nodo, respectivamente.

Sean v y w dos nodos del árbol. En preorden, primero numeramos un nodo y después numeramos sus subárboles de izquierda a derecha. Por tanto:

$$prenum[v] \leq prenum[w] \quad \Leftrightarrow \quad \begin{array}{l} v \text{ es un antecesor de } w, \text{ o bien} \\ v \text{ está a la izquierda de } w \text{ en el árbol} \end{array}$$

En postorden, primero numeramos los subárboles del nodo de izquierda a derecha, y después numeramos el nodo en sí. Por tanto:

$$postnum[v] \geq postnum[w] \quad \Leftrightarrow \quad \begin{array}{l} v \text{ es un antecesor de } w, \text{ o bien} \\ v \text{ está a la derecha de } w \text{ en el árbol} \end{array}$$

De esto se sigue que

$$\begin{array}{l} prenum[v] \leq prenum[w] \text{ y } postnum[v] \geq postnum[w] \\ \Leftrightarrow v \text{ es un antecesor de } w \end{array}$$

Una vez que los valores de $prenum$ y $postnum$ se han calculado en un tiempo que está en $\Theta(n)$, la condición requerida se puede comprobar en un tiempo que está en $\Theta(1)$.

9.3 RECORRIDO EN PROFUNDIDAD: GRAFOS NO DIRIGIDOS

Sea $G = \langle N, A \rangle$ un grafo no dirigido formado por todos aquellos nodos que deseamos visitar. Supongamos que de alguna manera es posible marcar un nodo para mostrar que ya ha sido visitado.

Para efectuar un recorrido en *profundidad* del grafo, se selecciona cualquier nodo $v \in N$ como punto de partida. Se marca este nodo para mostrar que ya ha sido visitado. A continuación, si hay un nodo adyacente a v que no haya sido visitado todavía, se toma este nodo como nuevo punto de partida y se invoca recursivamente al procedimiento de recorrido en profundidad. Al volver de la llamada recursiva, si hay otro nodo adyacente a v que no haya sido visitado, se toma este nodo como punto de partida siguiente, se vuelve a llamar recursivamente al procedimiento, y así sucesivamente. Cuando están marcados todos los nodos adyacentes a v , el recorrido que comenzara en v ha finalizado. Si queda algún nodo de G que no haya sido visitado, tomamos cualquiera de ellos como nuevo punto de partida, y volvemos a invocar el procedimiento. Se sigue así hasta que estén marcados todos los nodos de G . Véase a continuación el algoritmo recursivo:

```

procedimiento recorrido( $G$ )
  para cada  $v \in N$  hacer  $\text{marca}[v] \leftarrow \text{no visitado}$ 
  para cada  $v \in N$  hacer
    si  $\text{marca}[v] \neq \text{visitado}$  entonces  $rp(v)$ 

```

```

procedimiento  $rp(v)$ 
  {El nodo  $v$  no ha sido visitado anteriormente}
   $\text{marca}[v] \leftarrow \text{visitado}$ 
  para cada nodo  $w$  adyacente a  $v$  hacer
    si  $\text{marca}[w] \neq \text{visitado}$  entonces  $rp(w)$ 

```

El algoritmo se llama de recorrido en profundidad porque inicia tantas llamadas recursivas como sea posible antes de volver de una llamada. La recursión sólo se detiene cuando la exploración del grafo se ve bloqueada y no puede proseguir. En ese momento, la recursión «retrocede» para que sea posible estudiar posibilidades alternativas en niveles más elevados. Si el grafo corresponde a un juego, se puede pensar intuitivamente que esto es una búsqueda que explora el resultado de una estrategia particular con tantas jugadas anticipadas como sea posible, antes de explorar los alrededores para ver qué tácticas alternativas pudieran estar disponibles.

Considere por ejemplo el grafo de la figura 9.3. Si suponemos que los vecinos de un nodo dado se examinan por orden numérico, y que el nodo 1 es el primer punto de partida, entonces la exploración del grafo en profundidad progresa en la forma siguiente:

- | | | |
|----|-----------------------------|--|
| 1. | $rp(1)$ | llamada inicial |
| 2. | $rp(2)$ | llamada recursiva |
| 3. | $rp(3)$ | llamada recursiva |
| 4. | $rp(6)$ | llamada recursiva |
| 5. | $rp(5)$ | llamada recursiva; no se puede continuar |
| 6. | $rp(4)$ | no se ha visitado un vecino del nodo 1 |
| 7. | $rp(7)$ | llamada recursiva |
| 8. | $rp(8)$ | llamada recursiva; no se puede continuar |
| 9. | no quedan nodos por visitar | |

¿Cuánto tiempo se necesita para explorar un grafo con n nodos y a aristas? Dado que cada nodo se visita exactamente una vez, hay n llamadas al procedimiento rp . Cuando visitamos un nodo, examinamos las marcas de los nodos vecinos. Si se representa el grafo de tal manera que la lista de nodos adyacentes tenga un acceso directo (el tipo *grafolista* de la Sección 5.4), entonces este trabajo es proporcional a a en total. El algoritmo, por tanto, requiere un tiempo que está en $\Theta(n)$ para las llamadas al procedimiento, y un tiempo que está en $\Theta(a)$ para inspeccionar las marcas. Por tanto el tiempo de ejecución está en $\Theta(\text{máx}(a, n))$.

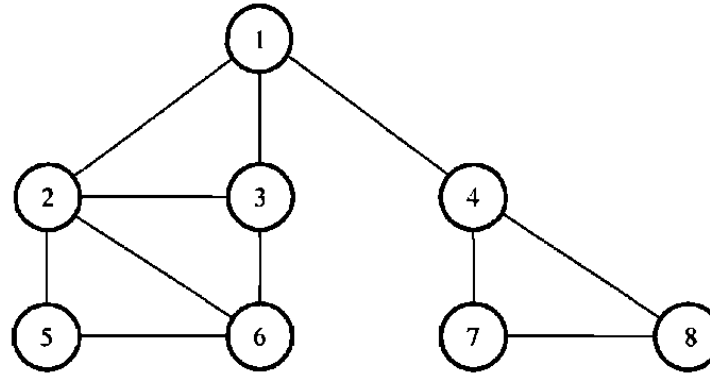


Figura 9.3. Un grafo no dirigido

El recorrido en profundidad de un grafo conexo asocia al grafo un árbol de recubrimiento. Sea T este árbol. Las aristas de T corresponden a las aristas utilizadas para recorrer el grafo; están dirigidas del primer nodo visitado al segundo. Las aristas que no se utilizan en el recorrido del grafo no tienen una arista correspondiente en T . El punto inicial de partida de la exploración pasa a ser la raíz del árbol. Por ejemplo, las aristas utilizadas en el recorrido en profundidad del grafo de la figura 9.3 que se describe más arriba son $\{1, 2\}$, $\{2, 3\}$, $\{3, 6\}$, $\{6, 5\}$, $\{1, 4\}$, $\{4, 7\}$ y $\{7, 8\}$. Las aristas dirigidas correspondientes, $(1, 2)$, $(2, 3)$ y demás, forman un árbol de recubrimiento para este grafo. La raíz del árbol es el nodo 1. El árbol se ilustra en la figura 9.4. Las líneas discontinuas de esta figura corresponden a las aristas de G que no se utilizan en el recorrido en profundidad. Resulta fácil mostrar que una arista de G que no tenga una arista correspondiente en T une necesariamente un nodo v con alguno de sus antecesores en T ; véase el problema 9.17.

Si el grafo que se está explorando no es conexo, entonces un recorrido en profundidad le asocia no sólo a un único árbol, sino a todo un bosque de árboles, uno para cada componente conexo del grafo. Un recorrido en profundidad también ofrece una manera de numerar los nodos del grafo que se está visitando. El primer nodo visitado —la raíz del árbol— recibe el número 1, el segundo recibe el número 2, y así sucesivamente. En otras palabras, los nodos del árbol asociado se numeran en preorden. Para implementar esto, hay que añadir las dos sentencias siguientes al principio del procedimiento *rp*:

```

nump ← nump + 1
prenum[v] ← nump

```

en donde *nump* es una variable global inicializada a cero. Por ejemplo, el recorrido en profundidad del grafo de la figura 9.3 descrita anteriormente numera los nodos del grafo en la forma siguiente.

nodo	1	2	3	4	5	6	7	8
<i>prenum</i>	1	2	3	6	5	4	7	8

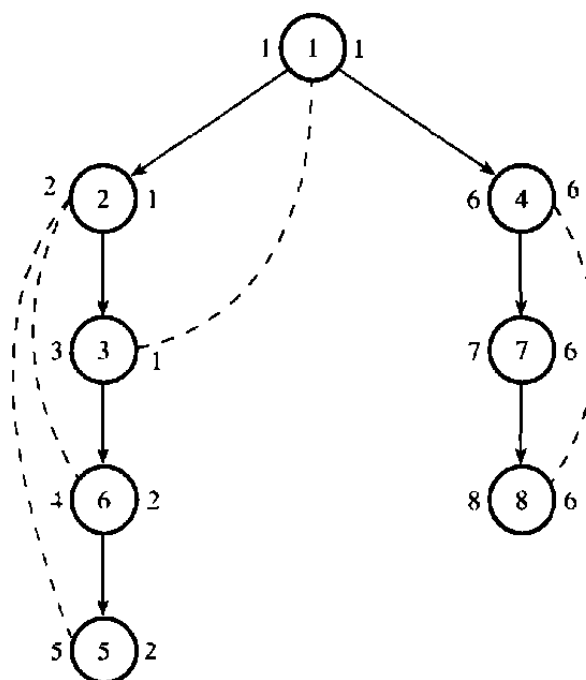


Figura 9.4. Un árbol recorrido en profundidad de prenum a la izquierda y masalto a la derecha.

Éstos son los números que aparecen a la izquierda de cada nodo en la figura 9.4. Por supuesto, el árbol y la numeración generados por un recorrido en profundidad dentro de un grafo no son únicos, sino que dependen del punto de partida seleccionado y del orden en que se visiten los vecinos.

9.3.1 Puntos de articulación

Un nodo v de un grafo conexo es un *punto de articulación* si el subgrafo que se obtiene borrando v y todas las aristas que incidan en v ya no es conexo. Por ejemplo, el nodo 1 es un punto de articulación del grafo de la figura 9.3; si lo borramos, quedan dos componentes conexas $\{2, 3, 5, 6\}$ y $\{4, 7, 8\}$. Un grafo G se llama *biconexo* (o bien *no articulado*) si es conexo y carece de puntos de articulación. Se denomina *bicoherente* (o bien *carente de istmos*, o *2-arista conexo*) si todo punto de articulación está unido mediante al menos dos aristas con cada componente del subgrafo restante. Estas ideas son importantes en la práctica. Si el grafo G representa, digamos, una red de telecomunicaciones, entonces el hecho de que sea biconexo nos asegura que el resto de la red puede seguir funcionando aunque falle el equipo de uno de los nodos. Si G es bicoherente, podemos estar seguros de que los nodos seguirán pudiendo comunicarse entre sí aunque deje de funcionar una línea de transmisión.

Para ver la forma de hallar los puntos de articulación de un grafo conexo G , examinemos de nuevo la figura 9.4. Recordemos que esta figura muestra *todas* las aristas del grafo G de la figura 9.3: las que se muestran con línea continua forman un árbol T de recubrimiento, mientras que las demás se muestran como líneas discontinuas. Como ya se ha visto, estas otras aristas sólo van desde algún nodo v hasta su antecesor dentro del árbol, y no de una rama a otra. A la izquierda de cada nodo v se encuentra $\text{prenum}[v]$, el número asignado mediante un recorrido de T en preorden.

A la derecha del nodo v se encuentra un nuevo número que vamos a llamar $masalto[v]$. Sea w el nodo más alto del árbol que se puede alcanzar desde v siguiendo cero o más líneas continuas, y subiendo entonces como máximo por una línea discontinua. Entonces se define $masalto[v]$ como $prenum[w]$. Por ejemplo, desde el nodo 7 podemos bajar por una línea continua hasta el nodo 8, y después subir por una línea discontinua hasta el nodo 4, y éste es el nodo más alto que podemos alcanzar. Dado que $prenum[4] = 6$, también tenemos que $masalto[7] = 6$.

Dado que las líneas discontinuas no cruzan de unas ramas a otras, el nodo más alto alcanzable de esta forma, w , tiene que ser un antecesor de v . (No puede estar por debajo de v dentro del árbol, porque siempre podemos llegar hasta el propio v sin seguir ninguna línea.) Entre los antecesores de v , el más alto dentro del árbol es el que tiene el valor de $prenum$ más bajo. Si tenemos estos valores, no es necesario por tanto conocer el nivel exacto de cada nodo: de entre los nodos que podemos alcanzar, nos limitamos a seleccionar aquel que minimice $prenum$.

Consideremos ahora cualquier nodo v de T salvo la raíz. Si v no tiene hijos, no puede ser un punto de articulación de G , porque si lo borramos, los nodos restantes siguen estando conectados mediante las aristas que quedan en T . En caso contrario, sea x un descendiente de v . Supongamos primero que $masalto[x] < prenum[v]$. Esto significa que partiendo de x existe una cadena de aristas de G , sin incluir la arista $\{v, x\}$ (porque no está permitido *ascender* por una línea continua), que nos lleva a algún nodo del árbol situado por encima de v . Si borramos v , por tanto, los nodos del subárbol cuya raíz es x no quedarán desconectados del resto del árbol. Esto es lo que sucede con el nodo 3 de la figura, por ejemplo. Aquí, $prenum[3] = 3$, y el único hijo del nodo 3, a saber, el nodo 6, tiene $masalto[6] = 2 < prenum[3]$. Por tanto, si borramos el nodo 3, entonces el nodo 6 y sus descendientes seguirán estando conectados a uno de los antecesores del nodo 3.

Si por otra parte $masalto[x] \geq prenum[v]$, entonces no hay ninguna cadena de aristas que parta de x (excluyendo de nuevo la arista $\{v, x\}$) y que vuelva a entrar en el árbol por encima de v . En este caso, si se borrara v , los nodos del subárbol cuya raíz es x quedarían desconectados del resto del árbol. El nodo 4 de la figura ilustra este caso. Aquí $prenum[4] = 6$, y el único hijo del nodo 4, a saber, el nodo 7, tiene $masalto[7] = 6 = prenum[4]$. Por tanto, si borramos el nodo 4, ningún camino que salga del nodo 7 ó alguno de sus descendientes vuelve a entrar en el árbol por encima del nodo borrado, así que el subárbol cuya raíz está en el nodo 7 se separa del resto de T .

Por tanto, un nodo v que no sea la raíz de T es un punto de articulación de G precisamente si tiene al menos un hijo x con $masalto[x] \geq prenum[v]$. Respecto a la raíz, es evidente que se trata de un punto de articulación de G si y sólo si tiene más de un hijo; en ese caso, dado que no hay aristas que crucen de una rama a otra, al borrar la raíz se desconectan los subárboles restantes de T .

Queda por ver la forma de calcular los valores de $masalto$. Claramente, esto hay que hacerlo desde las hojas hacia arriba. Por ejemplo, desde el nodo 5 podemos quedarnos donde estamos, o subir al nodo 2; éstas son todas las posibilidades. Desde el nodo 6 podemos permanecer donde estamos, subir al nodo 2, o bien bajar primero al nodo 5 y pasar desde allí a donde sea posible llegar, y así sucesivamente. Los

valores de *masalto* se calculan, por tanto, en postorden. En un nodo general v , $masalto[v]$ es el mínimo (correspondiente al nodo más alto) de tres clases de valores: $prenum[v]$ (nos quedamos donde estábamos), $prenum[w]$ para cada nodo w tal que haya una arista $\{v, w\}$ en G sin una arista correspondiente en T (ascendemos por una línea discontinua), y $masalto[x]$ para todo hijo x de v (bajamos por una línea continua y vemos hasta dónde se puede llegar por ese camino).

El algoritmo completo para hallar los puntos de articulación de un grafo no dirigido G se resume de la forma siguiente:

1. Se efectúa un recorrido en profundidad en G , empezando por cualquier nodo. Sea T el árbol generado mediante este recorrido, y para cada nodo v de G , sea $prenum[v]$ el número que le asigna este recorrido.
2. Se recorre T en postorden. Para cada nodo visitado v , se calcula $masalto[v]$ como el mínimo de
 - (a) $prenum[v]$;
 - (b) $prenum[w]$ para todo nodo w tal que haya una arista $\{v, w\}$ en G sin una arista correspondiente en T ; y
 - (c) $masalto[x]$ para todo hijo x de v .
3. Se determinan los puntos de articulación de G en la forma siguiente:
 - (a) La raíz de T es un punto de articulación precisamente si tiene más de un hijo.
 - (b) Cualquier otro nodo v es un punto de articulación si y sólo si tiene un hijo x tal que $masalto[x] \geq prenum[v]$.

No es difícil combinar los pasos 1 y 2 del algoritmo anterior, calculando tanto el valor de *prenum* como el de *masalto* durante el recorrido en profundidad de G .

9.4 RECORRIDO EN PROFUNDIDAD: GRAFOS DIRIGIDOS

El algoritmo es esencialmente el mismo que para los grafos no dirigidos; la diferencia reside en la interpretación de la palabra «adyacente». En un grafo dirigido, el nodo w es adyacente al nodo v si existe la arista dirigida (v, w) . Si existe (v, w) pero (w, v) no existe, entonces w es adyacente a v , pero v no es adyacente a w . Con este cambio de interpretación, los procedimientos *rp* y *recorridop* de la Sección 9.3 se aplican igualmente bien en el caso de un grafo dirigido.

Sin embargo, el algoritmo se comporta de forma bastante distinta. Consideremos un recorrido en profundidad del grafo dirigido de la figura 9.5. Si los vecinos de un nodo dado se examinan por orden numérico, entonces el algoritmo progresa de la forma siguiente:

- | | | |
|----|---------|--|
| 1. | $rp(1)$ | llamada inicial |
| 2. | $rp(2)$ | llamada recursiva |
| 3. | $rp(3)$ | llamada recursiva; no se puede continuar |
| 4. | $rp(4)$ | no se ha visitado un vecino del nodo 1 |
| 5. | $rp(8)$ | llamada recursiva |

6. $rp(7)$ llamada recursiva; no se puede continuar
7. $rp(5)$ nuevo punto de comienzo
8. $rp(6)$ llamada recursiva; no se puede continuar
9. no quedan nodos por visitar

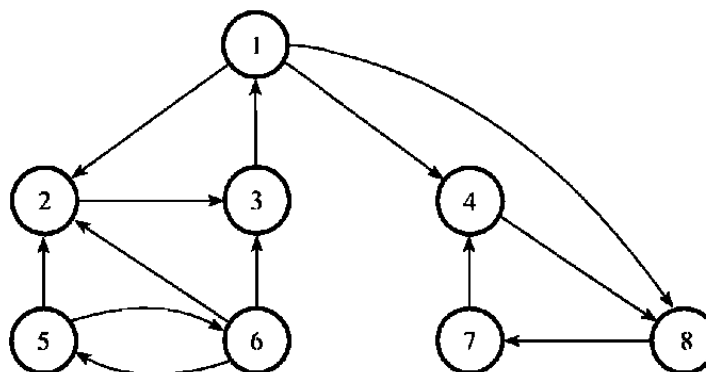


Figura 9.5. Un grafo dirigido

Un argumento idéntico al de la Sección 9.3 muestra que el tiempo que requiere este algoritmo también está en $\Theta(\max(a, n))$. En este caso, sin embargo, las aristas utilizadas para visitar todos los nodos de un grafo dirigido $G = \langle N, A \rangle$ pueden formar un bosque de varios árboles aunque G sea conexo. Esto es lo que sucede en nuestro ejemplo: las aristas utilizadas, a saber, $(1, 2)$, $(2, 3)$, $(1, 4)$, $(4, 8)$, $(8, 7)$ y $(5, 6)$ forman el bosque mostrado mediante líneas continuas en la figura 9.6.

Sea F el conjunto de aristas que hay en el bosque, de tal manera que $A \setminus F$ es el conjunto de aristas de G que no tienen una arista correspondiente dentro del bosque. En el caso de un grafo no dirigido, veíamos que las aristas del grafo que no tienen una arista correspondiente dentro del bosque unen necesariamente algún nodo con uno de sus antecesoros. En el caso de un grafo dirigido, sin embargo, pueden aparecer tres clases de aristas en $A \setminus F$. Se han mostrado mediante líneas discontinuas en la figura 9.6.

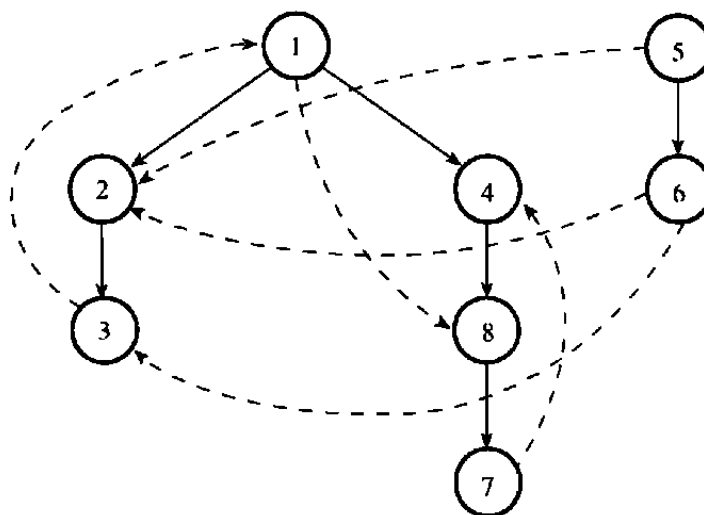


Figura 9.6. Un bosque de recorrido en profundidad

1. Las que, al igual que (3, 1) o (7, 4), van desde un nodo a uno de sus antecesores.
2. Las que, al igual que (1, 8), van de un nodo a uno de sus descendientes.
3. Las que, al igual que (5, 2) o (6, 3), unen un nodo con otro que no es ni antecesor ni descendiente suyo. Las aristas de este tipo están dirigidas, necesariamente, de derecha a izquierda.

9.4.1 Grafos acíclicos: ordenación topológica

Los grafos dirigidos acíclicos se pueden utilizar para representar un cierto número de relaciones interesantes. Esta clase incluye los árboles, pero es menos general que la clase de todos los grafos dirigidos. Por ejemplo, un grafo dirigido acíclico se puede utilizar para representar la estructura de una expresión aritmética que contenga subexpresiones repetidas: de esta manera, la figura 9.7 representa la estructura de la expresión

$$(a + b)(c + d) + (a + b)(c - d)$$

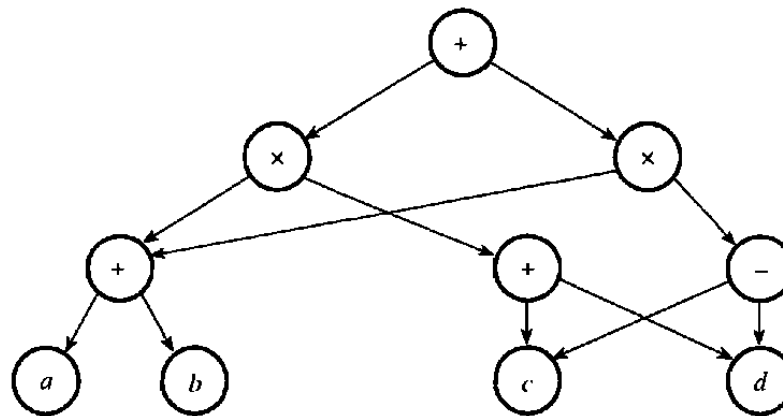


Figura 9.7. Un grafo dirigido acíclico

Estos grafos ofrecen una representación natural para las ordenaciones parciales, como la inclusión de conjuntos. La figura 9.8 ilustra una parte de otra ordenación parcial definida sobre los enteros positivos: aquí hay una arista del nodo i al nodo j si y sólo si i es un divisor propio de j .

Por último, los grafos dirigidos acíclicos suelen utilizarse para especificar la forma en que un proyecto complejo se desarrolla con el tiempo. Los nodos representan distintas fases del proyecto, desde el estado inicial hasta su terminación, y las aristas corresponden a las actividades que es preciso desarrollar para pasar de una fase a otra. La figura 9.9 da un ejemplo de este tipo de diagrama.

Se puede utilizar un recorrido en profundidad para detectar si un grafo dirigido dado es acíclico; véase el problema 9.26. También se puede utilizar para determinar una *ordenación topológica* de los nodos de un grafo dirigido acíclico. En este tipo de ordenación, los nodos del grafo se enumeran de tal manera que si existe una arista (i, j) entonces el nodo i precede al nodo j en la lista. Por ejemplo, para el grafo de la figura 9.8, el orden natural 1, 2, 3, 4, 6, 8, 12, 24 es adecuado, pero por otra

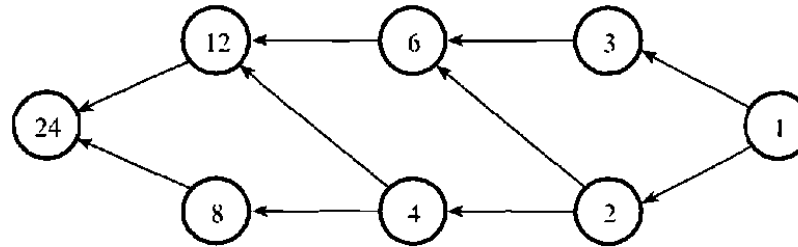


Figura 9.8. Otro grafo dirigido acíclico

parte el orden 1, 3, 2, 6, 4, 12, 8, 24 también es aceptable, e igualmente lo son otros más. Por otra parte, el orden 1, 3, 6, 2, 4, 12, 8, 24 no es válido, porque el grafo contiene una arista (2, 6) y por tanto 2 debe preceder a 6 en la lista.

La adaptación del procedimiento *rp* para hacer que sea una ordenación topológica es sencilla. Añada la línea

escribir v

al final del procedimiento *rp*, se ejecuta el procedimiento para el grafo en cuestión y se invierte entonces el orden de la lista de nodos resultante.

Para ver esto, consideremos lo que sucede cuando llegamos a un nodo v de un grafo dirigido acíclico G , empleando el procedimiento modificado. Algunos de los nodos que deben seguir a v en la ordenación topológica pueden haber sido visitados ya al seguir otra ruta diferente. En este caso, ya se encuentran en la lista tal como deberán estar cuando invirtamos la lista al finalizar la búsqueda. Todo nodo que deba preceder a v en la ordenación topológica se encuentra o bien en el camino que ya estamos explorando, en cuyo caso será marcado como visitado pero no presente en la lista aún, o bien todavía no ha sido visitado. En ambos casos, será añadido a la lista después del nodo v (una vez más, correctamente, porque hay que invertir la lista). Ahora el recorrido en profundidad explora los nodos no visitados que se puedan alcanzar desde v siguiendo las aristas de G . En la ordenación topológica, estos nodos deben ir después de v . Dado que tenemos intención de invertir la lista cuando haya terminado la búsqueda, al añadirlos a la lista durante la exploración que comienza en v , y al añadir v sólo cuando esta exploración haya terminado, obtenemos exactamente lo que necesitábamos.

9.5 RECORRIDO EN ANCHURA

Cuando un recorrido en profundidad llega a un nodo v , intenta a continuación visitar algún vecino de v , después algún vecino del vecino, y así sucesivamente. Cuando un recorrido en anchura llega a algún nodo v , por otra parte, primero visita todos los vecinos de v . Sólo examina nodos más alejados después de haber hecho esto. A diferencia del recorrido en profundidad, el recorrido en anchura no es naturalmente recursivo. Para poner de manifiesto las similitudes y diferencias entre los dos métodos, comenzaremos por dar una formulación no recursiva del al-

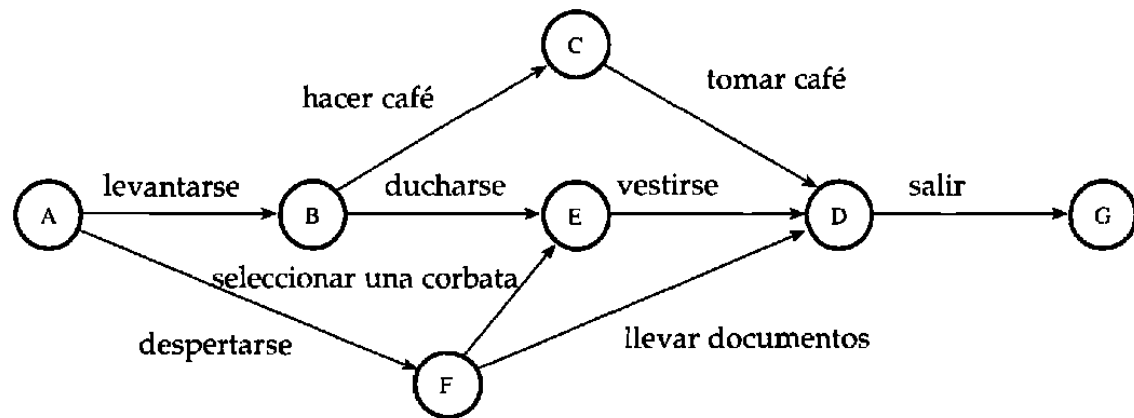


Figura 9.9. Y otro grafo dirigido acíclico

goritmo de recorrido en profundidad. Sea *pila* un tipo de datos que admite dos operaciones, **apilar** y **desapilar**. Se pretende que este tipo represente una lista de elementos que hay que manejar por el orden «último en llegar, primero en salir» (que suele conocerse con el nombre de LIFO, indicando «last in, first out», en inglés). La función *cima* denota el elemento que se encuentra en la parte superior de la pila. Lo que sigue es el algoritmo de recorrido en profundidad ya modificado:

procedimiento *rp2(v)*

P ← *pila vacía*

marca[v] ← *visitado*

mientras *P* no esté vacía **hacer**

mientras exista un nodo *w* adyacente a *cima(P)*

 tal que *marca[w]* ≠ *visitado*

hacer *marca[w]* ← *visitado*

apilar *w* en *P* {*w* es la nueva *cima(P)*}

desapilar *P*

La modificación del algoritmo no ha hecho que cambie su comportamiento. Lo único que hemos hecho es explicitar el apilamiento y desapilamiento de nodos que en la versión anterior era manejado entre bastidores por el mecanismo de pila implícito en cualquier lenguaje recursivo.

Para el algoritmo de recorrido en anchura, en contraste, necesitamos un tipo *cola* que admita las dos operaciones **poner** y **quitar**. Este tipo representa una lista de elementos que hay que manejar por el orden «primero en llegar, primero en salir» (FIFO, que indica «first in, first out»). La función *primero* denota el elemento que ocupa la primera posición de la cola. Véase a continuación el algoritmo de recorrido en anchura:


```
procedimiento ra(v)
    Q ← cola vacía
    marca[v] ← visitado
    poner v en Q
    mientras Q no esté vacía hacer
        u ← primero(Q)
        quitar u de Q
        para cada nodo w adyacente a u hacer
            si marca[w] ≠ visitado entonces marca[w] ← visitado
            poner w en Q
```

En ambos casos, necesitamos un programa principal que dé comienzo al recorrido:

```
procedimiento recorrido(G)
    para cada v ∈ N hacer marca[v] ← no visitado
    para cada v ∈ N hacer
        si marca[v] ≠ visitado entonces {bp2 o ra} (v)
```

Por ejemplo, en el grafo de la figura 9.3, si la búsqueda comienza en el nodo 1, y los vecinos de cada nodo se visitan por orden numérico, entonces el recorrido en anchura se desarrolla en la forma siguiente:

	Nodo visitado	Q
1.	1	2, 3, 4
2.	2	3, 4, 5, 6
3.	3	4, 5, 6
4.	4	5, 6, 7, 8
5.	5	6, 7, 8
6.	6	7, 8
7.	7	8
8.	8	—

Al igual que para el recorrido en profundidad, podemos asociar un árbol al recorrido en anchura. La figura 9.10 muestra el árbol generado por la búsqueda anterior. Las aristas del grafo que no tienen una arista correspondiente en el árbol se representan mediante líneas discontinuas; véase el problema 9.30. En general, si el grafo *G* que se está recorriendo no es conexo, el recorrido genera un bosque de árboles, uno por cada componente conexo de *G*.

Resulta sencillo mostrar que el tiempo requerido por un recorrido en anchura es del mismo orden que el requerido por un recorrido en profundidad, a saber, $\Theta(\text{máx}(a,n))$. Si se emplea la interpretación adecuada de la palabra «adyacente», el algoritmo de recorrido en anchura —una vez más, exactamente igual que el algoritmo de recorrido en profundidad— se puede aplicar sin modificación tanto a los grafos dirigidos como a los no dirigidos; véanse los problemas 9.31 y 9.32.

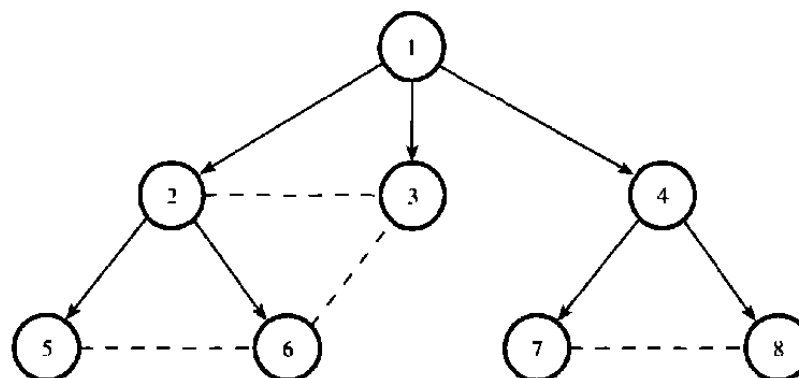


Figura 9.10. Un árbol de recorrido en anchura

El recorrido en anchura se emplea especialmente cuando haya que efectuar una exploración parcial de un grafo infinito (o tan grande que no resulte manejable), o para hallar el camino más corto desde un punto de un grafo hasta otro. Consideremos por ejemplo el problema siguiente. Se nos da el valor 1. Para construir otros valores, disponemos de dos operaciones: la multiplicación por 2 y la división por 3. Para la segunda operación, el operando debe ser mayor que 2 (por lo que no podemos llegar al 0), y se descartan los posibles decimales. Si se ejecutan las operaciones de izquierda a derecha, se puede obtener el valor 10 de la forma siguiente:

$$10 = 1 \times 2 \times 2 \times 2 \times 2 \div 3 \times 2$$

Deseamos obtener un valor concreto n . ¿Cómo podríamos hacerlo?

El problema se puede representar como una búsqueda en el grafo dirigido infinito de la figura 9.11. Aquí el valor dado 1 se encuentra en el nodo de la posición superior izquierda. A partir de ahí, cada nodo se une con los valores que se pueden obtener empleando las dos operaciones disponibles. Por ejemplo, a partir del valor 16 podemos obtener los dos valores nuevos 32 (multiplicando por 2) y 5 (dividiendo 16 por 3 y descartando los decimales). Por claridad, hemos omitido los enlaces que retroceden hacia valores que ya están disponibles; por ejemplo desde 8 hasta 2. Sin embargo, los enlaces hacia atrás están presentes en el grafo real. El grafo es infinito, porque una secuencia tal como 1, 2, ..., 512, ... puede continuar indefinidamente. No se trata de un árbol, porque el nodo 42, por ejemplo, se puede alcanzar tanto desde 128 como desde 21. Cuando se incluyen los enlaces hacia atrás, ni siquiera es acíclico.

Para resolver un caso dado del problema, esto es, la forma de construir un valor concreto de n , buscaremos en el grafo empezando por uno hasta encontrar el valor que estemos buscando. En este grafo infinito, sin embargo, un recorrido en profundidad puede no funcionar. Supongamos por ejemplo que $n = 13$. Si exploramos los vecinos de los nodos siguiendo el orden «primero multiplicar por 2, luego dividir por 3», entonces la búsqueda en profundidad va visitando sucesivamente los nodos 1, 2, 4, ..., y así sucesivamente, y se va por la rama superior y (dado que siempre hay un nuevo vecino que examinar) nunca encuentra una situación que le obligue a retroceder a un nodo anterior. En este caso, la búsqueda fallará con certeza. Por otra parte, si explora-

mos los vecinos de un nodo por el orden «primero división por 3, después multiplicación por 2», entonces la búsqueda baja primero al nodo 12; desde allí avanza sucesivamente hasta los nodos 24, 48, 96, 32 y 64, y desde el 64 se pierde en la parte superior derecha del grafo. Quizá tengamos la suerte de volver al nodo 13, hallando así la solución de nuestro problema, pero nada lo garantiza. Aun cuando la encontrásemos, la solución hallada sería más compleja de lo necesario. Si se programa este recorrido en profundidad en una computadora, se encontrará de hecho que se alcanza el valor dado 13 al cabo de 74 operaciones de multiplicación y división.

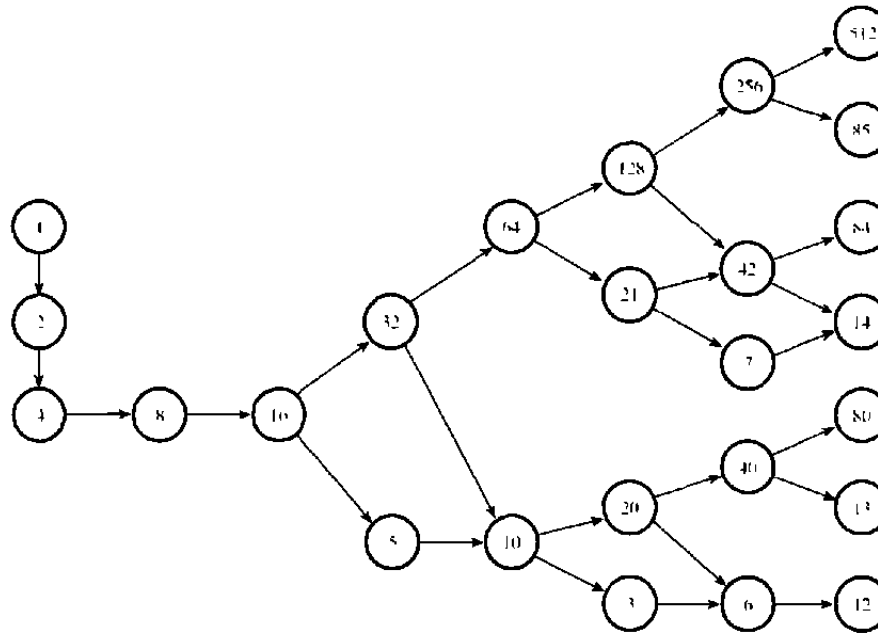


Figura 9.11. Multiplicación por 2 y división por 3

Por otra parte, una búsqueda en anchura tiene la certeza de hallar una solución si existe. Si examinamos los vecinos por el orden «primero multiplicar por 2, luego dividir por 3», una búsqueda en anchura que comience en uno visitará los nodos 1, 2, ..., 16, 32, 5, 64, 10, 128, 21, 20, 3, y así sucesivamente. No sólo estamos seguros de hallar el valor buscado si está en el grafo, sino que además la solución obtenida empleará el menor número posible de operaciones. En otras palabras, el camino hallado desde 1 hasta el valor deseado n será lo más corto posible. Lo mismo sucede si efectuamos la búsqueda examinando los vecinos por el orden «primero dividir por 3, luego multiplicar por 2». Utilizando cualquiera de estas dos búsquedas en anchura, resulta sencillo (incluso a mano) descubrir varias maneras de producir el valor 13 empleando solamente 9 operaciones. Por ejemplo

$$13 = 1 \times 2 \times 2 \times 2 \times 2 \div 3 \times 2 \times 2 \times 2 \div 3$$

Por supuesto, incluso una búsqueda en anchura puede no tener éxito. En nuestro ejemplo, puede ser que algunos valores de n no estén ni siquiera presentes en el grafo. (Desconociendo si esto es o no verdad, dejaremos esta cuestión como ejercicio para el lector.) En este caso, cualquier técnica de búsqueda

fallaría para los valores que no están presentes. Si un grafo incluye uno o más nodos con un número infinito de vecinos, pero sin caminos de longitud infinita, la búsqueda en profundidad puede tener éxito aun cuando falle el recorrido en anchura. Sin embargo, esta situación parece producirse con menos frecuencia que la inversa.

9.6 VUELTA ATRÁS

Tal como vimos anteriormente, hay distintos problemas que se pueden concebir en términos de grafos abstractos. Por ejemplo, veíamos en la Sección 9.1 que se pueden utilizar los nodos de un grafo para representar las situaciones de un juego de ajedrez, y las aristas para representar jugadas válidas. Con frecuencia, el problema original se traduce a buscar un nodo, un camino o un patrón específicos en el grafo asociado. Si el grafo contiene un número elevado de nodos, y especialmente si es infinito, puede resultar inútil o inabordable construirlo explícitamente en la memoria de una computadora antes de aplicar una de las técnicas de búsqueda que hemos empleado hasta el momento.

En tales situaciones utilizaremos un *grafo implícito*. Éste es un grafo para el cual se dispone de una descripción de sus nodos y aristas, de tal manera que se pueden construir partes relevantes del grafo a medida que progresa el recorrido. De esta manera, se ahorra tiempo de computación siempre que el recorrido tenga éxito antes de haber construido todo el grafo. La economía de espacio en memoria también puede ser dramática, sobre todo cuando se pueden descartar los nodos que hayan sido examinados ya, creando así espacio para poder explorar los nodos subsiguientes. Si el grafo en cuestión es infinito, las técnicas como ésta constituyen nuestra única esperanza de llegar a explorarlo. Esta sección, y las que siguen, detallan algunas formas estándar de organizar recorridos en un grafo implícito.

En su forma básica, la vuelta atrás se asemeja a un recorrido en profundidad dentro de un grafo dirigido. El grafo en cuestión suele ser un árbol, o por lo menos no contiene ciclos. Sea cual sea su estructura, existe sólo implícitamente. El objetivo del recorrido es encontrar soluciones para algún problema. Esto se consigue construyendo soluciones parciales a medida que progresa el recorrido; estas soluciones parciales limitan las regiones en las que se puede encontrar una solución completa. El recorrido tiene éxito si, procediendo de esta forma, se puede definir por completo una solución. En este caso el algoritmo puede bien detenerse (si lo único que se necesita es una solución del problema) o bien seguir buscando soluciones alternativas (si deseamos examinarlas todas). Por otra parte, el recorrido no tiene éxito si en alguna etapa la solución parcial construida hasta el momento no se puede completar. En tal caso, el recorrido vuelve atrás exactamente igual que en un recorrido en profundidad, eliminando sobre la marcha los elementos que se hubieran añadido en cada fase. Cuando vuelve a un nodo que tiene uno o más vecinos sin explorar, prosigue el recorrido de una solución.

9.6.1 El problema de la mochila (3)

Como primer ejemplo para ilustrar el principio general, volvamos al problema de la mochila que se describía en la Sección 8.4. Recordemos que se nos dan un cierto número de objetos y una mochila. En esta ocasión, sin embargo, en lugar de suponer que están disponibles n objetos, supondremos que lo que tenemos son n tipos de objetos, y que está disponible un número adecuado de objetos de cada tipo. Esto no altera el problema de forma sustancial. Para $i = 1, 2, \dots, n$ un objeto de tipo i tiene un peso positivo w_i y un valor positivo v_i . La mochila puede llevar un peso que no exceda de W . Nuestro objetivo es llenar la mochila de tal manera que se maximice el valor de los objetos incluidos, respetando al mismo tiempo la restricción de capacidad. Podemos tomar un objeto o prescindir de él, pero no podemos tomar una cierta parte de un objeto.

Supongamos para concretar que deseamos resolver un caso del problema con cuatro tipos de objetos, cuyos pesos son respectivamente 2, 3, 4 y 5 unidades, y cuyos valores son 3, 5, 6 y 10 respectivamente. La mochila puede llevar un máximo de 8 unidades de peso. Esto se puede hacer utilizando la técnica de vuelta atrás, explorando el árbol implícito de la figura 9.12.

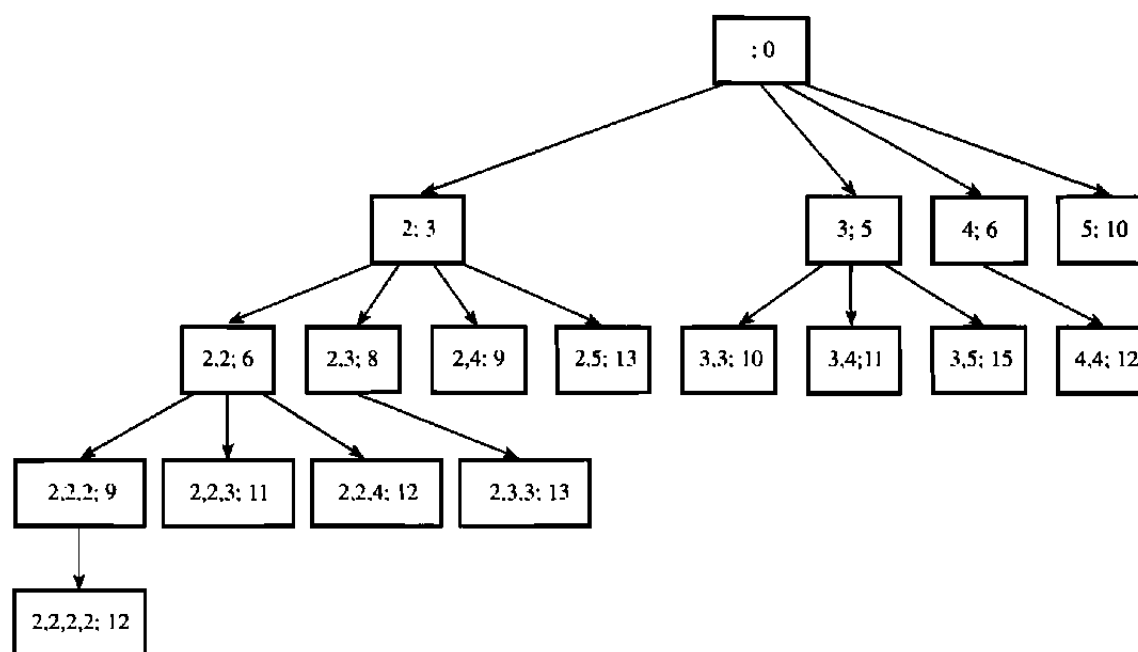


Figura 9.12. El árbol implícito para un problema de mochila

Aquí, un nodo tal como el (2, 3; 8) corresponde a una solución parcial de nuestro problema. Las cifras que están a la izquierda del punto y coma son los pesos de los objetos que hemos decidido incluir, y la cifra que está a la derecha es el valor actual de la carga. Bajar de un nodo a uno de sus hijos se corresponde con decidir qué clase de objeto se va a poner en la mochila a continuación. Sin pérdida de generalidad, podemos acordar que cargaremos los objetos en la mochila por orden creciente de peso. Esto no es esencial, y a decir verdad cualquier otro orden —por orden decre-

cienta de pesos, por ejemplo, o por valores— funcionaría igual de bien, pero así se reduce el tamaño del árbol que hay que explorar. Por ejemplo, una vez visitado el nodo (2, 2, 3; 11) no tiene sentido visitar posteriormente el (2, 3, 2; 11).

Inicialmente, la solución parcial está vacía. El algoritmo de vuelta atrás explora el árbol como un recorrido en profundidad, construyendo nodos y soluciones parciales a medida que avanza. En el ejemplo, el primer nodo que se visita es el (2; 3), el siguiente es el (2, 2; 6), el tercero es (2, 2, 2; 9) y el cuarto es (2, 2, 2, 2; 12). A medida que se visita cada nodo nuevo, se extiende la solución parcial. Después de visitar estos cuatro nodos, se bloquea el recorrido en profundidad: el nodo (2, 2, 2, 2; 12) no posee sucesores no visitados (de hecho, carece de sucesores), puesto que añadir más elementos a esta solución parcial violaría la restricción de capacidad. Dado que esta solución parcial puede resultar ser la solución óptima de nuestro caso, la memorizamos.

El algoritmo de recorrido en profundidad vuelve atrás ahora en busca de otras soluciones. En cada paso retrocedido por el árbol, se elimina el elemento correspondiente de la solución parcial. En el ejemplo, el recorrido vuelve primero a (2, 2, 2; 9), que también carece de sucesores no visitados; sin embargo, al retroceder un paso más por el árbol hasta el nodo (2, 2; 6), hay dos sucesores que quedan por visitar. Después de explorar los nodos (2, 2, 3; 11) y (2, 2, 4; 12), ninguno de los cuales mejora la solución memorizada anteriormente, la búsqueda retrocede otro paso más, y así sucesivamente. Explorando el árbol de esta manera, encontramos que (2, 3, 3; 13) es una solución mejor que la que ya tenemos, y después (3, 5; 15) es todavía mejor. Puesto que no se hace ninguna otra mejora antes de finalizar la búsqueda, ésta es la solución óptima del caso.

Programar el algoritmo es sencillo e ilustra la íntima relación existente entre recursión y recorrido en profundidad. Supongamos que los valores de n y de W , y los valores de la matrices $w[1..n]$ y de $v[1..n]$ correspondientes al caso que hay que resolver están disponibles como variables globales. La ordenación de los tipos de elementos es irrelevante. Se define una función *mochilava* de la forma siguiente:

```
función mochilava( $i, r$ )
    {Calcula el valor de la mejor carga que se puede
     construir empleando elementos de los tipos  $i$  a  $n$ 
     y cuyo peso total no sobrepase  $r$ }
     $b \leftarrow 0$ 
    {Se prueban por turno las clase de objetos admisibles }
    para  $k \leftarrow i$  hasta  $n$  hacer
        si  $w[k] \leq r$  entonces
             $b \leftarrow \max(b, v[k] + \textit{mochilava}(k, r - w[k]))$ 
    devolver  $b$ 
```

Ahora, para hallar el valor de la mejor carga, llamamos a *mochilava*(1, W). Cada llamada recursiva a *mochilava* se corresponde con extender el recorrido en profundidad hasta el nivel inmediatamente inferior del árbol, mientras que el bucle **para** se encarga de examinar todas las posibilidades en un nivel dado. En esta ver-

sión del programa, la composición de la carga que se está examinando está dada implícitamente por los valores de k en la pila recursiva. No es difícil adaptar el programa para que nos dé explícitamente la composición de la mejor carga junto con su valor; véase el Problema 9.42.

9.6.2 El problema de las ocho reinas

Para nuestro segundo ejemplo de vuelta atrás, consideremos el problema clásico consistente en situar las ocho reinas en un tablero de ajedrez de tal modo que ninguna de ellas amenace a ninguna de las demás. Recordemos que una reina amenaza a los cuadrados de la misma fila, de la misma columna o de las mismas diagonales.

La forma más evidente de resolver este problema consiste en probar sistemáticamente todas las formas de colocar ocho reinas en un tablero, determinando en cada ocasión si se ha encontrado una solución. Esta aproximación no tiene aplicación práctica, ni siquiera con una computadora, porque el número de situaciones que habría que examinar es $\binom{64}{8} = 4.426.165.368$. La primera mejora que podemos considerar consiste en no poner nunca más de una reina en una fila. Esto reduce la representación del tablero a un vector de ocho elementos, cada uno de los cuales da la posición de la reina dentro de la fila correspondiente. Por ejemplo, el vector (3, 1, 6, 2, 8, 6, 4, 7) representa la situación en que la reina de la fila 1 está en la columna 3, la reina de la fila 2 está en la columna 1, y así sucesivamente. Esta situación concreta no es una solución para nuestro problema, puesto que las reinas de las filas 3 y 6 se encuentran en la misma columna, y además hay dos pares de reinas en la misma diagonal. Empleando esta representación, podríamos escribir un algoritmo empleando ocho bucles anidados (para la función *solución*, véase el Problema 9.43):

```

programa reinas1
  para  $i_1 \leftarrow 1$  hasta 8 hacer
    para  $i_2 \leftarrow 1$  hasta 8 hacer
      ...
      para  $i_8 \leftarrow 1$  hasta 8 hacer
         $sol \leftarrow [i_1, i_2, \dots, i_8]$ 
        si solución(sol) entonces escribir sol
      parar
  escribir «no hay solución»

```

El número de situaciones que hay que considerar se ha reducido a $8^8 = 16.777.216$, aunque de hecho el algoritmo se detiene después de considerar solamente 1.299.852 situaciones.

La representación del tablero mediante un vector nos impide intentar poner dos reinas en una misma fila. Una vez que nos damos cuenta de esto, es natural ser igualmente sistemáticos en lo tocante a las columnas. Por tanto, ahora representaremos el tablero mediante un vector formado por ocho números *diferentes* entre 1 y 8, esto es, mediante una permutación de los ocho primeros números enteros. Esto produce el siguiente algoritmo:

```

programa reinas2
  sol ← permutación inicial
  mientras sol ≠ permutación final y no solución(sol) hacer
    sol ← permutación siguiente
  si solución(sol) entonces escribir sol
    sino escribir «no hay solución»

```

Hay varias maneras naturales de generar sistemáticamente todas las permutaciones de los n primeros números enteros. Por ejemplo, podemos poner cada valor correspondiente en la primera posición, y generar recursivamente, para cada valor inicial, todas las permutaciones de los $n-1$ elementos restantes. El procedimiento siguiente muestra la manera de hacer esto. Aquí $T[1..n]$ es una matriz global a la que se da como valor inicial $[1, 2, \dots, n]$, y la llamada inicial al procedimiento es $perm(1)$. Esta forma de generar permutaciones es también una especie de vuelta atrás.

```

procedimiento perm(i)
  si  $i = n$  entonces usar(T) {T es una nueva permutación}
    sino para  $j \leftarrow i$  hasta  $n$  hacer intercambiar  $T[i]$  y  $T[j]$ 
      perm(i+1)
    intercambiar  $T[i]$  y  $T[j]$ 

```

Este enfoque reduce el número de situaciones posibles a $8! = 40.320$. Si se utiliza el algoritmo precedente para generar las permutaciones, entonces en realidad solo se consideran 2.830 situaciones antes de que el algoritmo encuentre una solución. Es más complicado generar las permutaciones que generar todos los posibles vectores de ocho enteros entre 1 y 8. Por otra parte, resulta más sencillo verificar en este caso si una situación dada es una solución. Dado que ya sabemos que dos reinas no pueden estar ni en la misma fila ni en la misma columna, basta verificar que no estén en la misma diagonal.

Partiendo de un método muy grosero que ponía reinas absolutamente en todos los lugares del tablero, hemos progresado primero hasta un método que nunca pone dos reinas en la misma fila, y después a otro mejor que sólo considera aquellas situaciones en que dos reinas nunca pueden estar ni en la misma fila ni en la misma columna. Sin embargo, todos estos algoritmos comparten un defecto común: nunca comprueban si una situación es una solución mientras no se hayan colocado todas las reinas en el tablero. Por ejemplo, incluso el mejor de ellos hace 720 intentos inútiles para colocar las seis últimas reinas en el tablero cuando ha comenzado por poner las dos primeras en la diagonal principal, lugar en que desde luego se amenazan una a otra.

La vuelta atrás nos permite mejorar esto. Como primer paso, reformulamos el problema de las ocho reinas como un problema de búsqueda en un árbol. Decimos que un vector $V[1..k]$ de enteros entre 1 y 8 es k -prometedor, para $0 \leq k \leq 8$, si ninguna de las k reinas colocadas en las posiciones $(1, V[1]), (2, V[2]), \dots, (k, V[k])$ amenaza a ninguna de las otras. Matemáticamente, un vector V es k -prometedor si, para todo par de enteros i y j entre 1 y k , con $i \neq j$, tenemos que $V[i] - V[j] \notin \{i - j, 0, j - i\}$. Pa-

ra $k \leq 1$, todo vector V es k -prometedor. Las soluciones del problema de las 8 reinas se corresponden con aquellos vectores que son 8-prometedores.

Sea N el conjunto de vectores k -prometedores, $0 \leq k \leq 8$. Sea $G = \langle N, A \rangle$ el grafo dirigido tal que $(U, V) \in A$ si y sólo si existe un entero k , con $0 \leq k < 8$, tal que

- ◊ U es k -prometedor
- ◊ V es $(k + 1)$ -prometedor, y
- ◊ $U[i] = V[i]$ para todo $i \in [1..k]$

Este grafo es un árbol. Su raíz es el vector vacío correspondiente a $k = 0$. Sus hojas son o bien soluciones ($k = 8$) o posiciones sin salida ($k < 8$) tales como $[1, 4, 2, 5, 8]$: en tal situación, resulta imposible colocar una reina en la fila siguiente sin amenazar por lo menos a una de las reinas que ya están en el tablero. Las soluciones del problema de las ocho reinas se pueden obtener explorando este árbol. Sin embargo, no generamos explícitamente el árbol para explorarlo después. Más bien, se generan y abandonan los nodos en el transcurso de la exploración. El recorrido en profundidad es el método evidente, sobre todo si sólo necesitamos una solución.

Esta técnica posee dos ventajas con respecto al algoritmo que va probando sistemáticamente todas las permutaciones. En primer lugar, el número de nodos del árbol es menor que $8! = 40.320$. Aunque no resulta sencillo calcular teóricamente este número, es fácil contar los nodos empleando una computadora: hay 2.057. De hecho, basta con explorar 114 nodos para obtener una primera solución. En segundo lugar, para decidir si un vector es k -prometedor, sabiendo que es una extensión de un vector $(k-1)$ -prometedor, sólo necesitamos comprobar la última reina que haya que añadir. Esto se puede acelerar si asociamos a cada nodo prometedor el conjunto de columnas, el de diagonales positivas (a 45 grados) y el de diagonales negativas (a 135 grados) controlados por las reinas que ya están puestas.

En el procedimiento siguiente, $sol[1..8]$ es una matriz global. Para imprimir todas las soluciones del problema de las ocho reinas, se llama a $reinas(0, \emptyset, \emptyset, \emptyset)$:

procedimiento $reinas(k, col, diag45, diag135)$

$\{sol[1..k]$ es k -prometedor,

$col = \{sol[i] \mid 1 \leq i \leq k\}$,

$diag45 = \{sol[i] - i + 1 \mid 1 \leq i \leq k\}$ y

$diag135 = \{sol[i] + i - 1 \mid 1 \leq i \leq k\}$ }

si $k = 8$ **entonces** {un vector 8-prometedor es una solución}

escribir sol

sino {explorar las extensiones $(k+1)$ -prometedoras de sol }

para $j \leftarrow 1$ **hasta** 8 **hacer**

si $j \notin col$ y $j - k \notin diag45$ y $j + k \notin diag135$

entonces $sol[k + 1] \leftarrow j$

$\{sol[1..k+1]$ es $(k+1)$ -prometedor}

$reinas(k+1, col \cup \{j\}$,

$diag45 \cup \{j - k\}, diag135 \cup \{j + k\})$

Está claro que el problema se puede generalizar a un número arbitrario de reinas: ¿cómo se pueden situar n reinas en un «tablero» $n \times n$ de tal manera que ninguna de ellas amenace a ninguna de las demás? Tal como cabe esperar, la ventaja obtenida al utilizar la vuelta atrás en lugar de un enfoque exhaustivo se vuelve más pronunciada a medida que crece n . Por ejemplo, para $n = 12$ son 479.001.600 las posibles permutaciones que hay que considerar. Empleando el generador de permutaciones dado anteriormente, la primera solución que se encuentra corresponde a la 4.546.044-ésima situación examinada. Por otra parte, el árbol explorado por el algoritmo de vuelta atrás contiene solamente 856.189 nodos, y se obtiene una solución al visitar el nodo 262. El problema se puede generalizar todavía más colocando las «reinas» en tres dimensiones en un tablero $n \times n \times n$; véase el Problema 9.49.

9.6.3 El caso general

Los algoritmos de vuelta atrás se pueden utilizar aun cuando las soluciones buscadas no tengan todas necesariamente la misma longitud. Véase el esquema general:

```

procedimiento vueltaatrás( $v[1..k]$ )
    { $v$  es un vector  $k$ -prometedor}
    si  $v$  es una solución entonces escribir  $v$ 
    sino para cada vector  $(k+1)$ -prometedor  $w$ 
        tal que  $w[1..k] = v[1..k]$ 
            hacer vueltaatrás( $w[1..k+1]$ )
  
```

El **sino** debería estar presente si y sólo si resulta imposible disponer de dos soluciones diferentes tales que una sea prefijo de la otra.

Tanto el problema de la mochila como el problema de las n reinas se resolvían empleando una búsqueda en profundidad en el árbol correspondiente. Algunos problemas que se pueden formular en términos de explorar un grafo implícito tienen la propiedad consistente en que corresponden a un grafo infinito. En este caso puede ser necesario emplear un recorrido en anchura para evitar la exploración interminable de alguna rama infinita carente de resultados positivos. El recorrido en anchura también es apropiado si tenemos que encontrar una solución comenzando a partir de una cierta situación inicial y efectuando el menor número de pasos posible.

9.7 RAMIFICACIÓN Y PODA

Al igual que la vuelta atrás, la ramificación y poda es una técnica para explorar un grafo dirigido implícito. Una vez más, este grafo es normalmente acíclico, o incluso un árbol. Esta vez vamos a buscar la solución óptima de algún problema. En cada nodo, calculamos una cota del posible valor de aquellas soluciones que pudieran encontrarse más adelante en el grafo. Si la cota muestra que cualquiera de estas soluciones tiene que ser necesariamente peor que la mejor solución hallada hasta el momento, entonces no necesitamos seguir explorando esta parte del grafo.

En su versión más sencilla, el cálculo de cotas se combina con un recorrido en anchura o en profundidad, y solamente sirve para podar ciertas ramas de un árbol o para cerrar caminos de un grafo. Con más frecuencia, sin embargo, la cota calculada se utiliza también para seleccionar el camino que, entre los abiertos, parezca más prometedor para explorarlo primero.

En términos generales, podemos decir que las búsquedas en profundidad acaban de explorar los nodos por orden inverso al de su creación, empleando una pila para almacenar los nodos que se han generado pero no han sido examinados aún. Las búsquedas en amplitud terminan de explorar los nodos por el mismo orden en que son creados, utilizando una cola para almacenar los nodos que se han generado pero no han sido examinados aún. Ramificación y poda utiliza cálculos auxiliares para decidir en cada momento qué nodo debe explorarse a continuación y una lista con prioridad para almacenar los nodos que se han generado pero no han sido examinados aún. Recordemos que los montículos suelen ser ideales para almacenar listas con prioridad; véase la Sección 5.7. Ilustraremos la técnica con dos ejemplos.

9.7.1 El problema de la asignación

En el *problema de la asignación*, hay que asignar n tareas a n agentes, de forma que cada agente realice exactamente una tarea. Si al agente i , con $1 \leq i \leq n$, se le asigna la tarea j , con $1 \leq j \leq n$, entonces el coste de realizar esta tarea concreta será c_{ij} . Dada la matriz de costes completa, el problema consiste en asignar tareas a los agentes de tal manera que se minimice el coste total de ejecutar las n tareas.

Por ejemplo, consideremos tres agentes a , b y c , a los que hay que asignar las tareas 1, 2 y 3 con la matriz de costes siguiente:

	1	2	3
a	4	7	3
b	2	6	1
c	3	9	4

Si asignamos la tarea 1 al agente a , la tarea 2 al agente b y la tarea 3 al agente c , entonces nuestro coste total será $4 + 6 + 4 = 14$, mientras que si asignamos la tarea 3 al agente a , la tarea 2 al agente b y la tarea 1 al agente c , el coste será solamente $3 + 6 + 3 = 12$. En este caso concreto, el lector puede verificar que la asignación óptima es $a \rightarrow 2$, $b \rightarrow 3$ y $c \rightarrow 1$, cuyo coste es $7 + 1 + 3 = 11$.

El problema de la asignación tiene numerosas aplicaciones. Por ejemplo, en lugar de hablar de agentes y tareas, podemos formular el problema en términos de edificios y solares, donde c_{ij} es el coste de construir el edificio i en el solar j , y deseamos minimizar el coste total de los edificios. Es fácil inventar otros ejemplos. En general, con n agentes y n tareas, hay $n!$ posibles asignaciones para considerar que son demasiadas incluso para valores moderados de n . Por tanto, recurriremos a la ramificación y poda.

Supongamos que es preciso resolver el caso cuya matriz de costes se muestra en la figura 9.13. Para obtener una cota superior de la respuesta, obsérvese que $a \rightarrow 1$, $b \rightarrow 2$, $c \rightarrow 3$, $d \rightarrow 4$ es una posible solución cuyo coste es $11 + 15 + 19 + 28 = 73$. La solución óptima del problema no puede costar más que esto. Otra posible solución es $a \rightarrow 4$, $b \rightarrow 3$, $c \rightarrow 2$, $d \rightarrow 1$, cuyo coste se obtiene sumando los elementos de la otra diagonal de la matriz de costes, dando $40 + 13 + 17 + 17 = 87$. En este caso, la segunda solución no supone mejora respecto a la primera. Para obtener una cota inferior de la solución, podemos argumentar que sea quien sea el que ejecute la tarea 1, el coste será 11 como mínimo; sea quien sea el que ejecute la tarea 2, el coste será de 12 como mínimo, y así sucesivamente. Por tanto, sumando los elementos más pequeños de cada columna obtenemos una cota inferior de la respuesta. En el ejemplo, esto es $11 + 12 + 13 + 22 = 58$. Se obtiene una segunda cota inferior sumando los elementos más pequeños de cada fila, basándonos en que todos los agentes tienen que hacer algo. En este caso obtenemos $11 + 13 + 11 + 14 = 49$, que es menos útil que la cota inferior anterior. Juntando estos resultados, sabemos que la respuesta de nuestro caso se encuentra en algún lugar de $[58..73]$.

	1	2	3	4
a	11	12	18	40
b	14	15	13	22
c	11	17	19	23
d	17	14	20	28

Figura 9.13. La matriz de costes para el problema de asignación

Para resolver el problema mediante ramificación y poda, exploraremos un árbol cuyos nodos corresponden a asignaciones parciales. En la raíz del árbol no se han hecho asignaciones. En lo sucesivo, en cada nivel se determina la asignación de un agente más. Para cada nodo, calculamos una cota de las soluciones que se pueden obtener completando la asignación parcial correspondiente, y utilizamos esta cota para cerrar caminos y para guiar la búsqueda. Supongamos por ejemplo que empezando por la raíz decidimos efectuar en primer lugar la asignación del agente a . Dado que hay cuatro formas de hacer esto, hay cuatro ramas que parten de la raíz. La figura 9.14 ilustra la situación.

La cifra que aparece junto a cada nodo es una cota inferior de las soluciones que se pueden obtener completando la asignación parcial correspondiente. Ya hemos mostrado la forma en que se puede obtener la cota 58 en la raíz. Para calcular la cota del nodo $a \rightarrow 1$, por ejemplo, observemos en primer lugar que con esta asignación parcial la tarea 1 tendrá un coste 11. La tarea 2 será ejecutada por b , c o d así

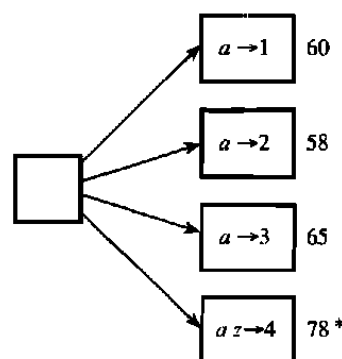


Figura 9.14. Después de asignar una tarea al agente *a*

que el mínimo coste posible es 14. De forma similar, las tareas 3 y 4 también serán ejecutadas por *b*, *c* o *d*, y sus costes mínimos posibles son 13 y 22 respectivamente. De esta manera, una cota inferior de cualquier solución que se obtenga completando la asignación parcial $a \rightarrow 1$ es $11 + 14 + 13 + 22 = 60$. De forma similar, para el nodo $a \rightarrow 2$, la tarea 2 será ejecutada por el agente *a* con un coste 12, mientras que las tareas 1, 3 y 4 serán ejecutadas por los agentes *b*, *c* y *d* con un coste mínimo de 11, 13 y 22 respectivamente. Por tanto, toda solución que incluya la asignación $a \rightarrow 2$ costará como mínimo $12 + 11 + 13 + 22 = 58$. Las otras dos cotas inferiores se obtienen de forma similar. Dado que sabemos que la solución óptima no puede sobrepasar 73, ya está claro que no tiene sentido seguir explorando el nodo $a \rightarrow 4$: toda solución obtenida completando esta asignación parcial costará al menos 78, así que no puede ser óptima. El asterisco de este nodo denota que está «muerto». Sin embargo, los otros tres nodos siguen estando vivos. El nodo $a \rightarrow 2$ tiene la cota inferior más pequeña. Argumentando que por tanto parece más prometedor que los otros, será éste el que exploremos a continuación. Hacemos esto fijando un elemento más de la asignación parcial, digamos el *b*. De esta manera llegamos a la situación que se muestra en la figura 9.15.

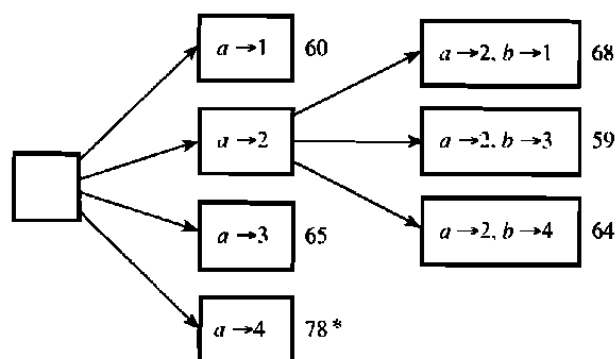


Figura 9.15. Después de asignar una tarea al agente *b*

Una vez más, la cifra que aparece junto a cada nodo nos da una cota inferior del coste de las soluciones que se pueden obtener completando la asignación parcial

correspondiente. Por ejemplo, en el nodo $a \rightarrow 2, b \rightarrow 1$, la tarea 1 costará 14 y la tarea 2 costará 12. Las tareas restantes 3 y 4 deben ser ejecutadas por c o d . El coste más pequeño posible para la tarea 3 es por tanto 19, mientras que para la tarea 4 es 23. Por tanto, una cota inferior de las soluciones posibles es $14 + 12 + 19 + 23 = 68$. Las otras dos cotas nuevas se calculan de forma similar.

El nodo más prometedor del árbol es ahora $a \rightarrow 2, b \rightarrow 3$ con una cota inferior de 59. Para seguir explorando el árbol comenzando en este nodo, fijamos un elemento más de la asignación parcial, digamos el c . Cuando se fijan las asignaciones de a, b y c , sin embargo, ya no nos queda opción alguna para la asignación de d , así que la solución está completa. Los nodos de la derecha de la figura 9.16, que muestra la siguiente fase de nuestra exploración, corresponden por tanto a soluciones completas.

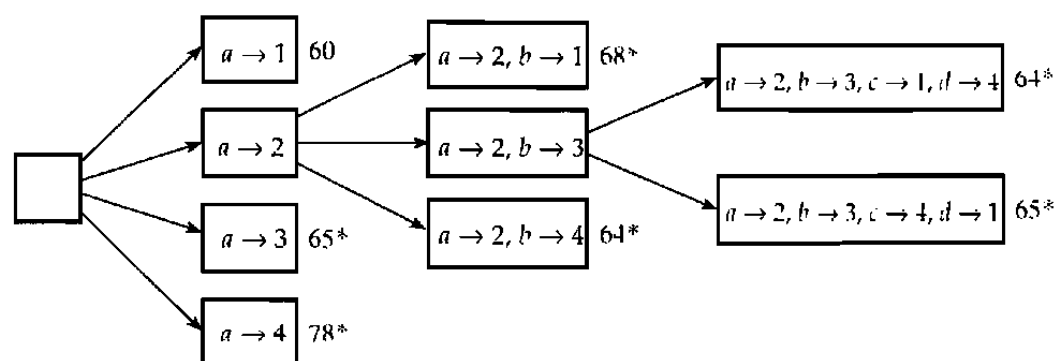


Figura 9.16. Después de asignar una tarea al agente c

La solución $a \rightarrow 2, b \rightarrow 3, c \rightarrow 1, d \rightarrow 4$, con un coste de 64, es mejor que cualquiera de las soluciones halladas al empezar, y nos proporciona una nueva cota superior para el óptimo. Gracias a esta nueva cota superior, podemos descartar los nodos $a \rightarrow 3$ y $a \rightarrow 2, b \rightarrow 1$ a efectos de posteriores exploraciones, tal como indican los asteriscos. Ninguna solución que complete estas asignaciones parciales puede ser tan buena como la que acabamos de encontrar. Si solamente deseamos una solución del caso, podemos eliminar también el nodo $a \rightarrow 2, b \rightarrow 4$.

El único nodo que sigue mereciendo la pena explorar es $a \rightarrow 1$. Procediendo igual que antes, al cabo de dos pasos obtenemos la situación final que se muestra en la figura 9.17. La mejor solución hallada es $a \rightarrow 1, b \rightarrow 3, c \rightarrow 4$ y $d \rightarrow 2$, con un coste de 61. En los nodos restantes no explorados, la cota inferior es mayor que 61, así que no tiene sentido seguir estudiándolos. La solución anterior es por tanto la solución óptima de nuestro caso.

El ejemplo ilustra que aun cuando en una fase anterior el nodo $a \rightarrow 2$ era el más prometedor, la solución óptima no llegó a surgir de él. Para obtener nuestra respuesta, hemos construido 15 de los 41 nodos (1 raíz, 4 de profundidad 1, 12 de profundidad 2, y 24 de profundidad 3) que están presentes en un árbol completo del tipo ilustrado. De las 24 soluciones posibles, sólo 6 (incluyendo las dos utilizadas para determinar la cota superior inicial) han sido examinadas.

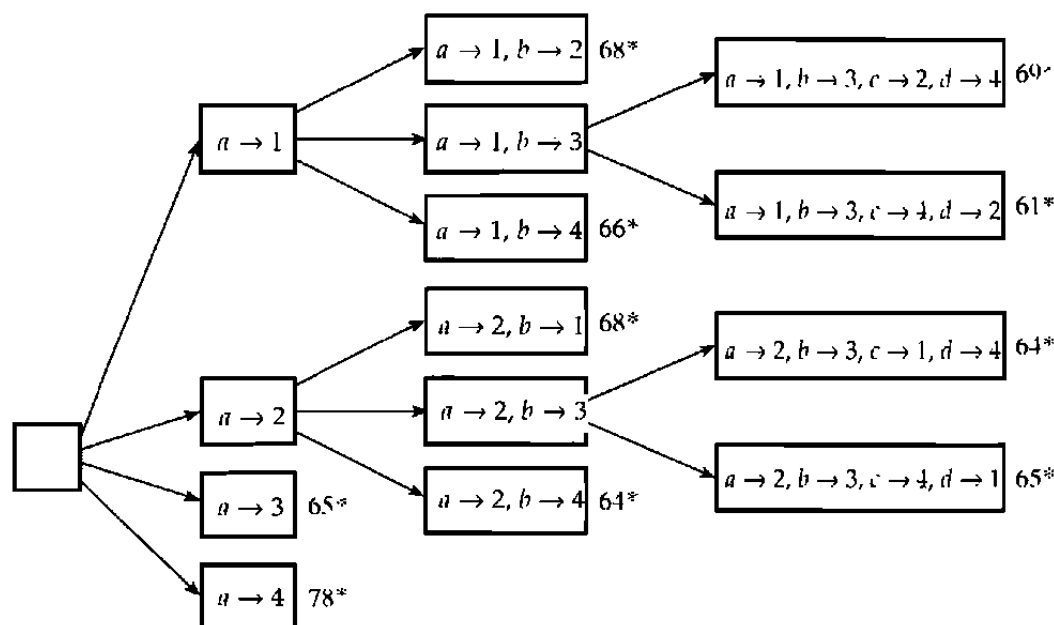


Figura 9.17. El árbol explorado en su totalidad

9.7.2 El problema de la mochila (4)

Como segundo ejemplo, consideremos el problema de la mochila; véanse las Secciones 8.4. y 9.6.1. Aquí se nos pide maximizar $\sum_{i=1}^n x_i v_i$ sometido a la restricción $\sum_{i=1}^n x_i w_i \leq W$, donde los v_i y w_i son todos estrictamente positivos, y los x_i son enteros no negativos. Este problema también se puede resolver por ramificación y poda.

Supongamos sin pérdida de generalidad que las variables están numeradas de tal manera que $v_i / w_i \geq v_{i+1} / w_{i+1}$. Entonces si los valores de $x_1, x_2, \dots, x_k, 0 \leq k \leq n$ quedan fijados, con $\sum_{i=1}^n x_i w_i \leq W$, es fácil ver que el valor que se puede obtener sumando más elementos de los tipos $k+1, \dots, n$ a la mochila no puede sobrepasar el valor

$$\sum_{i=1}^k x_i v_i + \left(W - \sum_{i=1}^k x_i w_i \right) v_{k+1} / w_{k+1}$$

Donde el primer término da el valor de los elementos que ya están en la mochila, y el segundo es una cota del valor que se puede añadir.

Para resolver el problema por ramificación y poda, exploramos un árbol en cuya raíz no está fijado el valor de ninguno de los x_i , y en cada nivel sucesivo se va determinando el valor de una variable más, por orden numérico de variables. En cada nodo que exploremos, sólo generamos aquellos sucesores que satisfagan la restricción de peso, de tal manera que cada nodo tiene un número finito de sucesores. Siempre que se genera un nodo, calculamos una cota superior del valor de

la solución que se puede obtener completando la carga parcialmente especificada, y utilizamos estas cotas superiores para cortar ramas inútiles y para guiar la exploración del árbol. Dejamos los detalles para el lector.

9.7.3 Consideraciones generales

La necesidad de mantener una lista de nodos que han sido generados pero que no han sido explorados en su totalidad, situados en diferentes niveles del árbol y preferiblemente ordenados por orden de las cotas correspondientes, hace que la ramificación y poda resulten difíciles de programar. El montículo es una estructura de datos ideal para almacenar esta lista. A diferencia del recorrido en profundidad, y de las técnicas relacionadas, el programador no dispone de una formulación recursiva elegante de la ramificación y poda. Sin embargo, la técnica es tan potente que suele emplearse en aplicaciones prácticas.

Resulta casi imposible dar una idea precisa de lo bien que se va a comportar esta técnica en un problema dado, empleando una cota dada. Siempre hay que llegar a un compromiso en lo concerniente a la calidad de la cota calculada. Con una cota mejor, examinamos menos nodos, y si tenemos suerte se nos guiará a una solución óptima con más rapidez. Por otra parte, lo más probable es que pasemos más tiempo en cada nodo calculando la cota correspondiente. En el caso peor, puede ocurrir que incluso una cota excelente no nos permita cortar ninguna rama del árbol, así que se desperdiciará todo el trabajo adicional efectuado en cada nodo. En la práctica, sin embargo, para problemas del tamaño que suele encontrarse en las aplicaciones, casi siempre es rentable invertir el tiempo necesario para calcular la mejor cota posible (dentro de lo razonable). Por ejemplo, se encuentran aplicaciones tales como programación lineal y que se manejan mediante ramificación y poda, obteniéndose la cota en cada nodo mediante la resolución de un problema asociado de programación lineal, con variables continuas.

9.8 EL PRINCIPIO DE MINIMAX

Sea cual sea la técnica utilizada, persiste el molesto hecho de que para un juego tal como el ajedrez queda descartado un examen exhaustivo del grafo asociado. En esta situación, tenemos que conformarnos con una búsqueda parcial en torno a la situación actual. Éste es el principio que subyace a una importante heurística denominada *minimax*. Aunque esta heurística no nos permite asegurar que ganaremos, siempre que esto sea posible, halla una jugada de la cual puede esperarse razonablemente que se cuente entre las mejores jugadas disponibles, explorando solamente una parte del grafo a partir de alguna posición dada. La exploración del grafo se detiene normalmente antes de alcanzar las posiciones terminales, empleando uno entre varios criterios posibles, y las posiciones en que se detiene la exploración se evalúan de forma heurística. En cierto sentido, esto no es sino una versión sistemática del método utilizado por ciertos jugadores humanos, que consiste en examinar anticipadamente un pequeño número de jugadas. Nos limitaremos a esbozar la técnica.

Supongamos entonces que se desea jugar una buena partida al ajedrez. El primer paso es definir una función estática de evaluación *eval* que atribuya un valor a cada posible situación. Idealmente, el valor de *eval*(*u*) debería crecer a medida que la situación de *u* sea más favorable para las blancas. Se acostumbra a dar valores no muy distintos de cero a las situaciones en que ninguno de los jugadores tiene una ventaja definida, y grandes valores negativos a las situaciones que favorecen a las negras. Esta función de evaluación debe de tener en cuenta muchos factores: el número y tipo de las piezas restantes por ambas partes, el control del centro, la libertad de movimiento y demás. Es preciso llegar a un compromiso entre la precisión de la función y el tiempo que se necesita para calcularla. Cuando se aplique a una situación terminal, la función de evaluación debería devolver $+\infty$ si hay jaque mate contra las negras, $-\infty$ si hay jaque mate contra las blancas, y cero si la partida acaba en tablas. Por ejemplo, una función de evaluación que tiene en cuenta correctamente los aspectos estáticos de la situación, pero que resulta demasiado sencilla para utilizarla en la realidad, podría ser la siguiente: para situaciones no terminales, se cuenta 1 punto por cada peón blanco, $3\frac{1}{4}$ puntos por cada alfil o caballo blanco, 5 puntos por cada torre, y 10 puntos por cada reina blanca; se sustrae un número equivalente de puntos por cada pieza negra.

Si la función de evaluación estática fuera perfecta, sería sencillo determinar la mejor jugada que se puede hacer. Supongamos que es el turno de las blancas para mover en la posición *u*. El mejor movimiento sería pasar a la posición *v* que maximice *eval*(*v*) entre todos los sucesores de *u*:

```

val ←  $-\infty$ 
para cada posición w que sea un sucesor de u hacer
    si eval(w) ≥ val entonces val ← eval(w)
    v ← w

```

Esta aproximación tan simple no tendría mucho éxito si empleáramos la función de evaluación sugerida anteriormente, porque no dudaría en sacrificar una reina ¡para capturar un peón!

Si la función de evaluación no es perfecta, una estrategia mejor es suponer que las negras van a responder con la jugada que minimice la función *eval*, puesto que cuanto más pequeño sea el valor de esta función, mejor será la jugada para las negras. Idealmente, las negras desearían un valor negativo muy grande. (Recordemos que llamamos jugada a cualquier acción, para evitar el término «media jugada».)

```

val ←  $-\infty$ 
para cada posición w que sea un sucesor de u hacer
    si w no tiene sucesor
        entonces valw ← eval(w)
        sino valw ←  $\min\{eval(x) \mid x \text{ es un sucesor de } w\}$ 
    si valw ≥ val entonces val ← valw
    v ← w

```

Ahora ya no aparece el problema consistente en sacrificar una reina para tomar un peón; naturalmente esta puede ser exactamente la regla equivocada si evita que las blancas encuentren la jugada ganadora. Quizá mirando más adelante el cambio pudiera resultar rentable. Por otra parte, seguro que evitamos aquellas jugadas que pudieran permitir a las negras dar mate inmediatamente (siempre y cuando *podamos* evitarlo).

Para añadir aspectos más dinámicos a la evaluación estática que proporciona *eval*, resulta preferible examinar varias jugadas por anticipado. Para examinar n jugadas por anticipado desde la posición u , las blancas deben pasar a la posición v dada por

```

 $val \leftarrow -\infty$ 
para cada posición  $w$  que sea un sucesor de  $u$  hacer
     $B \leftarrow \text{Negras}(w, n)$ 
    si  $B \leq val$  entonces  $val \leftarrow B$ 
     $v \leftarrow w$ 

```

en donde las funciones *Blancas* y *Negras* son las siguientes

```

función Negras( $w, n$ )
    si  $n = 0$  o  $w$  no tiene sucesor
        entonces devolver eval( $w$ )
    sino devolver  $\min\{\text{Blancas}(x, n - 1) \mid x \text{ es un sucesor de } w\}$ 

```

```

función Blancas( $w, n$ )
    si  $n = 0$  o  $w$  no tiene sucesor
        entonces devolver eval( $w$ )
    sino devolver  $\max\{\text{Negras}(x, n - 1) \mid x \text{ es un sucesor de } w\}$ 

```

Ahora vemos la razón por la cual esta técnica se llama minimax. Las negras intentan minimizar la ventaja que se les permite a las blancas, y las blancas, por su parte, intentan maximizar la ventaja que se obtiene en cada jugada.

Con más generalidad, supongamos que la figura 9.18 muestra una parte del grafo correspondiente a algún juego. Si los valores asociados a los nodos del nivel más bajo se obtienen aplicando la función *eval* a las situaciones correspondientes, los valores para los otros nodos se pueden calcular empleando la regla de minimax. En el ejemplo, suponemos que el jugador A está intentando maximizar la función de evaluación, y que el jugador B está intentando minimizarla. Si A juega para maximizar su ventaja, seleccionará el segundo de los tres movimientos posibles. Esto le asegura un valor mínimo de 10; véase sin embargo el problema 9.55.

La técnica básica de minimax se puede mejorar en diferentes aspectos. Por ejemplo, quizá merezca la pena explorar con más profundidad los movimientos

más prometedores. De manera similar, la exploración de ciertas ramas se puede abandonar muy pronto si la información acerca de ellas de que se dispone es ya suficiente para mostrar que no pueden llegar a tener influencia en el valor de los nodos pertenecientes a zonas más altas del árbol. Este segundo tipo de mejora, que no describiremos en este libro, se conoce con el nombre de *poda alfa-beta*.

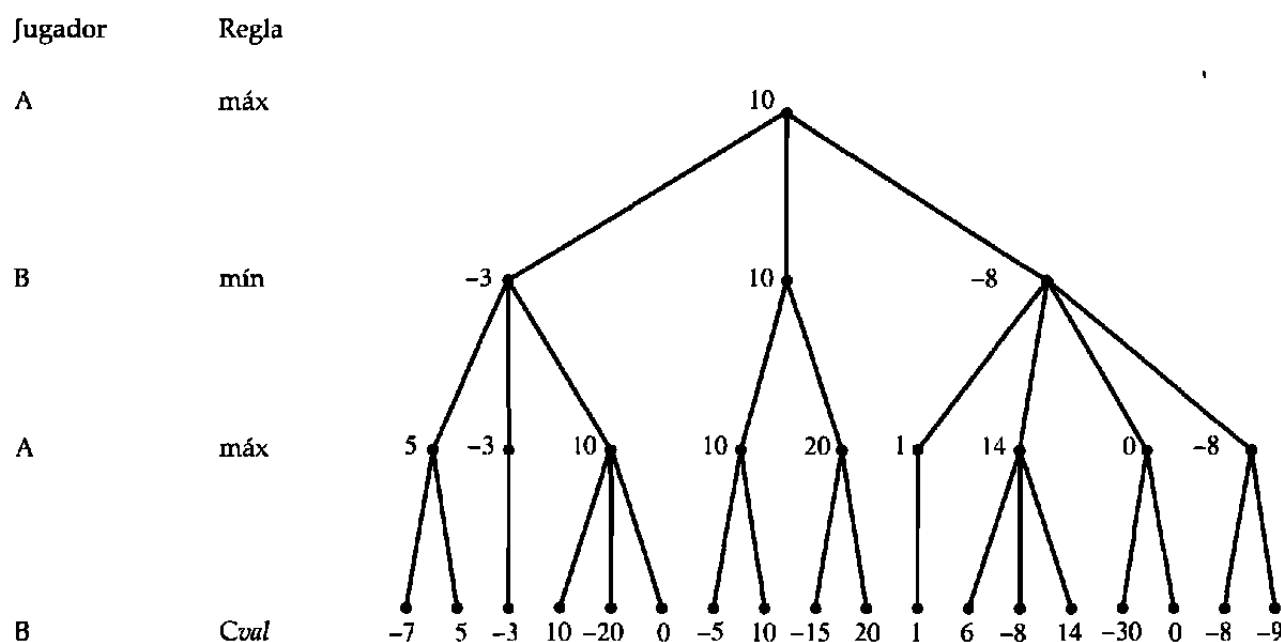


Figura 9.18. El principio de minimax

9.9 PROBLEMAS

Problema 9.1. Añadir los nodos $\langle 8, 7 \rangle$, $\langle 7, 6 \rangle$, $\langle 6, 5 \rangle$ y sus descendientes al grafo de la figura 9.1.

Problema 9.2. ¿Es posible que una situación de victoria del juego descrito en la Sección 9.1 tenga más de una situación de derrota entre sus sucesores? En otras palabras, ¿existen situaciones en las que están disponibles varias jugadas ganadoras? ¿Puede suceder esto en el caso de una situación ganadora inicial $\langle n, n-1 \rangle$?

Problema 9.3. Supongamos que se cambian las reglas del juego de la Sección 9.1 pa-

ra que pierda el jugador que se vea obligado a tomar la última cerilla. Ésta es la versión *misère* del juego. Supongamos también que el primer jugador tiene que tomar al menos una cerilla, y que tiene que dejar dos como mínimo. Entre las situaciones iniciales con entre tres y ocho cerillas, ¿cuáles son ahora las situaciones ganadoras para el primer jugador?

Problema 9.4. Modificar el algoritmo *ganarec* de la Sección 9.1 para que devuelva un entero k , donde $k = 0$ si la situación es de derrota, y $1 \leq k \leq j$ si se produce una situación de victoria al tomar k cerillas.

Problema 9.5. Demostrar que en el juego descrito en la Sección 9.1, el primer jugador

dispone de una estrategia ganadora si y sólo si el número inicial de cerillas no aparece en la sucesión de Fibonacci.

Problema 9.6. Consideremos un juego que no pueda proseguir durante un número infinito de jugadas, y en que ninguna situación ofrezca al jugador al que corresponda el turno un número de jugadas válidas que sea infinito. Sea G el grafo dirigido que corresponde a este juego. Demostrar que el método descrito en la Sección 9.1 permite etiquetar todos los nodos de G como victoria, derrota o tablas.

Problema 9.7. Considere el juego siguiente. Inicialmente, se pone un montón de n cerillas en la mesa, entre dos jugadores. Cada jugador, por turnos, puede (a) partir cualquier montón de la mesa en dos montones desiguales o (b) quitar una o dos cerillas de cualquier montón de la mesa. No puede hacer las dos cosas a la vez. Sólo puede partir un montón, y si decide tomar dos cerillas, deben salir ambas del mismo montón. Gana el jugador que tome la última cerilla.

Por ejemplo, supongamos que durante el juego se llega a la situación $\{5, 4\}$, esto es, que hay dos montones en la mesa, uno con 5 cerillas y otro con 4. El jugador al que corresponda mover puede pasar a $\{4, 3, 2\}$ o bien a $\{4, 4, 1\}$ partiendo el montón de 5 cerillas; puede pasar a $\{5, 3, 1\}$ partiendo el montón de 4 (pero no a $\{5, 2, 2\}$ puesto que los nuevos montones deben de ser desiguales), o bien a $\{4, 4\}$, $\{4, 3\}$, $\{5, 3\}$ o $\{5, 2\}$ tomando una o dos cerillas de cualquiera de los montones.

Dibujar el grafo del juego para $n = 5$. Si ambos juegan correctamente, ¿gana el primer jugador o el segundo?

Problema 9.8. Repetir el problema anterior para la versión *misère* del juego, en que pierde el jugador que se quede con la última cerilla.

Problema 9.9. Consideremos un juego del tipo descrito en la Sección 9.1. Cuando utilizamos un grafo de posiciones de victoria y de derrota para describir este juego, suponemos implícitamente que ambos jugadores moverán inteligentemente, para maximizar sus posibilidades de ganar. ¿Puede perder un jugador que esté en una situación de victoria si su oponente juega mal y comete un «error» inesperado?

Problema 9.10. Para cualquiera de las técnicas de recorrido de árboles que se mencionan en la Sección 9.2, demostrar que una implementación recursiva requiere un espacio de almacenamiento que está en $\Omega(n)$ en el caso peor.

Problema 9.11. Demostrar que cualquiera de las tres técnicas de recorrido de árboles mencionadas en la Sección 9.2 se puede implementar de tal forma que sólo necesite un tiempo que esté en $\Theta(n)$, y un espacio de almacenamiento en $\Theta(1)$, aun cuando los nodos no contengan un puntero a sus padres (en cuyo caso el problema es trivial).

Problema 9.12. Generalizar los conceptos de preorden y postorden a árboles arbitrarios (no binarios). Suponer que los árboles se representan como en la figura 5.7. Demostrar que estas dos técnicas siguen funcionando en un tiempo que está en el orden del número de nodos que haya en el árbol que hay que recorrer.

Problema 9.13. En la Sección 9.2 dábamos una forma de preconditionamiento para un árbol de tal forma que en lo sucesivo se pudiera verificar rápidamente si un nodo es o no un antecesor de otro. Existen varias maneras de llegar al mismo resultado. Demostrar, por ejemplo, que se puede hacer utilizando un recorrido en preorden seguido por un recorrido en preorden invertido, que visi-

ta primero un nodo y después sus subárboles de derecha a izquierda.

Si se representan los árboles tal como en la figura 5.7, ¿es este método más o menos eficiente que el dado en la Sección 9.2, o son comparables?

Problema 9.14. Mostrar cómo progresa un recorrido en profundidad a través del grafo de la figura 9.3 si el punto inicial es el nodo 6, y los vecinos de un nodo dado se visitan (a) en orden numérico y (b) en orden numérico decreciente.

Mostrar el árbol de recubrimiento y la numeración de los nodos del grafo generado por cada uno de estos recorridos.

Problema 9.15. Analizar el tiempo de ejecución del algoritmo *rp* si el grafo que hay que explorar se representa mediante una matriz de adyacencia (del tipo *grafoadya* de la Sección 5.4) en lugar de utilizar listas de nodos adyacentes.

Problema 9.16. Mostrar como se puede utilizar el recorrido en profundidad para hallar las componentes conexas de un grafo no dirigido.

Problema 9.17. Sea G un grafo no dirigido, y sea T el árbol de recubrimiento generado mediante un recorrido en profundidad de G . Demostrar que una arista de G que no tenga una arista correspondiente en T no puede unir nodos de distintas ramas del árbol, sino que necesariamente tiene que unir algún nodo v con uno de sus antecesores dentro de T .

Problema 9.18. Demostrar lo siguiente, o dar un contraejemplo:

- (a) Si un grafo es biconexo, entonces es bicoherente.
- (b) Si un grafo es bicoherente, entonces es biconexo.

Problema 9.19. Demostrar que un nodo v de un grafo conexo es un punto de articulación si y sólo si existen dos nodos a y b distintos de v tales que todo camino que una a con b pasa por v .

Problema 9.20. Demostrar que para cada par de nodos distintos u y v de un grafo biconexo existen al menos dos rutas que unen u con v y que no tienen nodos comunes salvo los nodos inicial y final.

Problema 9.21. En el algoritmo para hallar los puntos de articulación de un grafo no dirigido que se da en la Sección 9.3.1, mostrar cómo calcular los valores tanto de *prenum* como de *masalto* durante el recorrido en profundidad del grafo, e implementar el algoritmo correspondiente.

Problema 9.22. El ejemplo de la Sección 9.3.1 halla los puntos de articulación para el grafo de la figura 9.3 empleando un recorrido en profundidad que comienza en el nodo 1. Verificar que se encuentran los mismos puntos de articulación si el recorrido comienza en el nodo 6.

Problema 9.23. Ilustrar cómo funciona el algoritmo para hallar los puntos de articulación de un grafo no dirigido que se da en la Sección 9.3.1 sobre el grafo de la figura 9.19, empezando el recorrido (a) en el nodo 1 y (b) en el nodo 3.

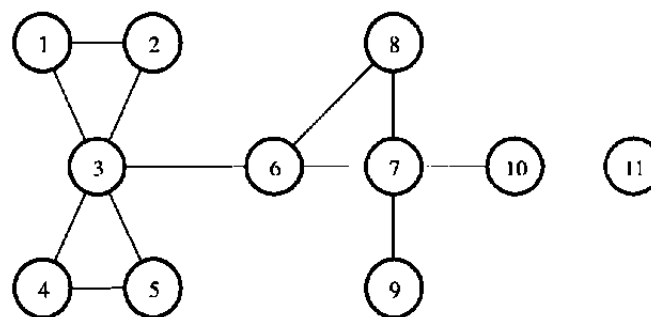


Figura 9.19. Un grafo con puntos de articulación

Problema 9.24. Un grafo es planar si se puede dibujar en un papel de tal manera que no se crucen ninguna de sus aristas. Utilizar un recorrido en profundidad para diseñar un algoritmo que sea capaz de decidir en tiempo lineal si un cierto grafo dado es o no planar.

Problema 9.25. Ilustrar el progreso del algoritmo de recorrido en profundidad aplicado al grafo de la figura 9.5 si el punto inicial es el nodo 1 y los vecinos de un nodo dado se examinan por orden numérico decreciente.

Problema 9.26. Sea F el bosque generado por un recorrido en profundidad en un grafo dirigido $G = \langle N, A \rangle$. Demostrar que G es acíclico si y sólo si el conjunto de aristas de G que no tienen una arista correspondiente en F no contiene ningún nodo que vaya de un nodo a alguno de sus antecesores dentro del bosque.

Problema 9.27. Para el grafo de la figura 9.8, ¿cuál es la ordenación topológica que se obtiene si se utiliza el procedimiento sugerido en la Sección 9.4.1, comenzando el recorrido en profundidad en el nodo 1 y visitando los vecinos de cada nodo por orden numérico?

Problema 9.28. ¿Qué falla en el argumento siguiente? Cuando se visita el nodo v de un grafo G empleando un recorrido en profundidad, se exploran inmediatamente todos los demás nodos que se puedan alcanzar desde v , siguiendo aristas de G . En la ordenación topológica, éstos otros nodos deben ir después de v . Por tanto, para obtener una ordenación topológica de estos nodos, basta con añadir una línea adicional

escribir v

al principio del procedimiento *rp*.

Problema 9.29. Un grafo dirigido es *fuertemente conexo* si existen caminos desde u hasta v y desde v hasta u para todo par de nodos u y v . Si un grafo dirigido no es fuertemente conexo, los mayores conjuntos de nodos tales que los subgrafos inducidos son fuertemente conexos se denominan *componentes fuertemente conexos* del grafo. Por ejemplo, los componentes fuertemente conexos del grafo de la figura 9.5 son $\{1, 2, 3\}$, $\{5, 6\}$ y $\{4, 7, 8\}$. Diseñar un algoritmo eficiente, basado en un recorrido en profundidad, para hallar las componentes fuertemente conexas de un grafo.

Problema 9.30. Después de un recorrido en anchura de un grafo no dirigido, sea F el bosque de árboles generado. Si $\{u, v\}$ es un arista de G que no posee una arista correspondiente en F (estas aristas se representan mediante líneas discontinuas en la figura 9.10), mostrar que los nodos u y v yacen en el mismo árbol de F , pero que ni u ni v son antecesores uno de otro.

Problema 9.31. Mostrar cómo progresa un recorrido en anchura a través del grafo de la figura 9.5, suponiendo que los vecinos de cada nodo se visiten en orden numérico, y que los puntos iniciales necesarios se seleccionen también por orden numérico.

Problema 9.32. Dibujar el bosque generado por el recorrido del problema 9.31, mostrando las aristas restantes del grafo mediante líneas discontinuas. ¿Cuántas clases de líneas «discontinuas» puede haber?

Problema 9.33. Justificar la afirmación de que un recorrido en profundidad del grafo de la figura 9.11, comenzando en el nodo 1 y visitando los vecinos por el orden «primero dividir por 3, luego multiplicar por 2», baja hasta el nodo 12 y después visita sucesivamente los nodos 24, 48, 96, 32 y 64.

Problema 9.34. Enumerar los quince primeros nodos visitados por un recorrido en anchura del grafo de la figura 9.11, comenzando en el nodo 1 y visitando los vecinos por el orden «primero dividir por 3, luego multiplicar por 2».

Problema 9.35. La Sección 9.5 da una forma de producir el valor 13, empezando desde 1 y utilizando las operaciones de multiplicación por 2 y división por 3. Encontrar otra forma de producir el valor 13 utilizando el mismo punto inicial y las mismas operaciones.

Problema 9.36. Un nodo p de un grafo dirigido $G = \langle N, A \rangle$ se denomina sumidero si para todo nodo $v \in N$, $v \neq p$, existe la arista (v, p) , mientras que no existe la arista (p, v) . Escribir un algoritmo que pueda detectar la presencia de un sumidero en G en un tiempo que esté en $O(n)$, en donde n es el número de nodos que haya en el grafo. El algoritmo debe aceptar el grafo representado por su matriz de adyacencia (del tipo *grafoadya* de la Sección 5.4). Obsérvese que un tiempo de ejecución en $O(n)$ para este problema es algo notable, dado que sólo para escribir los datos se requiere un espacio que está en $\Omega(n^2)$.

Problema 9.37. Un camino de Euler en un grafo no dirigido es un camino en la cual todas las aristas aparecen una sola vez. Diseñar un algoritmo que determine si un grafo dado posee un camino de Euler, y que imprima ese camino si existe. ¿Cuánto tiempo necesita su algoritmo?

Problema 9.38. Repetir el problema 9.37 para un grafo dirigido.

Problema 9.39. Dado un grafo dirigido o no dirigido, se dice que un camino es hamiltoniano cuando pasa exactamente una vez por todos los nodos del grafo, sin volver al nodo inicial. En un grafo dirigido, sin embar-

go, es preciso tener en cuenta la dirección de las aristas en el camino. Demostrar que si un grafo dirigido es completo (esto es, si todo par de nodos está unido al menos en una dirección) entonces tiene un camino hamiltoniano. En este caso, dar un algoritmo para encontrar el camino.

Problema 9.40. Dibujar el árbol de búsqueda explorado por un algoritmo de vuelta atrás que resuelva el mismo caso del problema de la mochila de la Sección 9.6.1, pero suponiendo esta vez que los elementos se cargan en orden decreciente de pesos.

Problema 9.41. Resolver el mismo caso del problema de la mochila de la Sección 9.6.1 por programación dinámica. Habrá que hacer primero el problema 8.5.1.

Problema 9.42. Adaptar la función *mochila* de la Sección 9.6.1 para dar la composición de la carga óptima, además de su valor.

Problema 9.43. Supongamos que el vector $q[1..8]$ representa las posiciones de ocho reinas en un tablero de ajedrez, con una reina en cada fila: si $q[i] = j$, $1 \leq i \leq 8$, $1 \leq j \leq 8$, la reina de la fila i está en la columna j . Escribir una función *solución*(q) que devuelva el valor falso si al menos una de las reinas amenaza a otra, y que devuelva el valor verdadero en caso contrario.

Problema 9.44. Dado que el algoritmo *reinas1* halla una solución y se detiene después de comprobar 1.299.852 situaciones, resolver el problema sin emplear una computadora.

Problema 9.45. Supongamos que el procedimiento *usar*(T) que se invoca desde el procedimiento *perm*(i) de la Sección 9.6 consiste simplemente en imprimir la matriz T en una línea nueva. Mostrar el resultado de llamar a *perm*(1) cuando $n = 4$.

Problema 9.46. Supongamos que el procedimiento $usar(T)$ que se invoca desde el procedimiento $perm(i)$ de la Sección 9.6 requiere un tiempo constante. ¿Cuánto tiempo se necesita, en función de n , para ejecutar la llamada $perm(1)$?

Rehacer el problema suponiendo ahora que $usar(T)$ requiere un tiempo que está en $\Theta(n)$.

Problema 9.47. ¿Para qué valores de n carece de solución el problema de las n reinas? Demostrar la respuesta.

Problema 9.48. ¿Cuántas soluciones tiene el problema de las ocho reinas? ¿Cuántas soluciones *distintas* hay si no distinguimos las soluciones que se pueden transformar unas en otras por rotaciones y reflexiones?

Problema 9.49. Investigar el problema consistente en situar k «reinas» en un tablero tridimensional $n \times n \times n$. Suponga que una «reina» en tres dimensiones amenaza las posiciones de las mismas filas, columnas o diagonales en que se encuentra, de la forma evidente. Claramente, k no puede sobrepasar el valor n^2 . Sin contar el caso trivial $n = 1$, ¿cuál es el valor más pequeño de n para el cual es posible una solución con n^2 «reinas»?

Problema 9.50. Una matriz booleana $M[1..n, 1..n]$ representa un laberinto cuadrado. En general, partiendo de un punto dado, podemos pasar a puntos adyacentes de la misma fila o de la misma columna. Si $M[i, j]$ es *verdadero*, se puede pasar por el punto (i, j) ; si $M[i, j]$ es *falso*, no se puede pasar por el punto (i, j) . La figura 9.20 da un ejemplo.

Dar un algoritmo de vuelta atrás que busque un camino, si existe, desde $(1,1)$ hasta (n, n) . Sin ser completamente formal, por ejemplo, se pueden decir cosas como «para todo punto v que sea un vecino de x hacer...», el algoritmo debe ser claro y preciso.

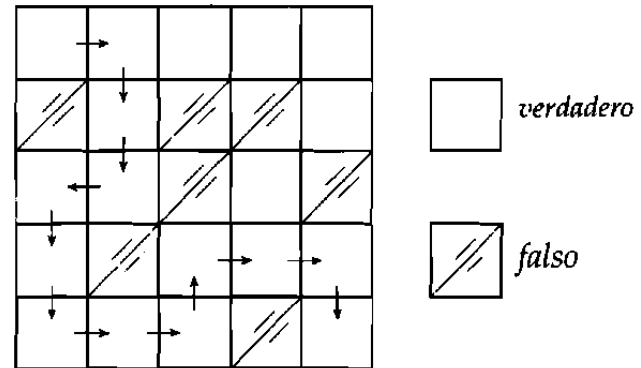


Figura 9.20. Un laberinto

Problema 9.51. El método de vuelta atrás sugerido en el problema 9.50 ilustra el principio, pero resulta muy ineficiente en la práctica. Buscar una forma mucho mejor de resolver el mismo problema.

Problema 9.52. En la Sección 9.7 se calculaban las cotas inferiores de los nodos del árbol de búsqueda suponiendo que toda tarea no asignada sería ejecutada por el agente sin asignación que pudiera hacerlo con menor coste. Esto es lo mismo que tachar las filas y columnas de la matriz de costes que correspondan a los agentes y tareas que ya tengan asignación, y sumar entonces los elementos mínimos de cada una de las columnas restantes. Una forma alternativa de calcular las cotas es suponer que todo agente sin asignación efectuará la tarea que pueda realizar con el mínimo coste. Esto es como tachar las filas y columnas de la matriz de costes correspondientes a los agentes y tareas que ya tengan asignación, y sumar después los elementos mínimos de todas las filas restantes. Mostrar cómo funciona un algoritmo de ramificación y poda para el caso de la figura 9.7 actuando utilizando este segundo método de calcular cotas inferiores.

Problema 9.53. Utilizar un algoritmo de ramificación y poda para resolver los problemas de asignación que tienen asociadas las siguientes matrices:

		1	2	3	4
(a)	<i>a</i>	94	1	54	68
	<i>b</i>	74	10	88	82
	<i>c</i>	62	88	8	76
	<i>d</i>	11	74	81	21

		1	2	3	4	5
(b)	<i>a</i>	11	17	8	16	20
	<i>b</i>	9	7	12	6	15
	<i>c</i>	13	16	15	12	16
	<i>d</i>	21	10	12	11	15
	<i>e</i>	14	10	12	11	15

Problema 9.54. Se dispone de cuatro tipos de objetos, cuyos pesos son respectivamente 7, 4, 3 y 2 unidades, y cuyos valores son 9, 5, 3 y 1. Podemos transportar un máximo de 10 unidades de peso. Los objetos no se pueden fragmentar en trozos. Determinar la carga más valiosa que se pueda

transportar, empleando (a) programación dinámica y (b) ramificación y poda. Para la parte (a), es necesario resolver primero el problema 8.51.

Problema 9.55. Examinando el árbol de la figura 9.18, dijimos que el jugador que va a mover esta situación tiene asegurado un valor de 10 como mínimo. ¿Es esto estrictamente cierto?

Problema 9.56. Supongamos que u corresponde a la situación inicial de las piezas en el juego del ajedrez. ¿Qué se puede decir acerca de $\text{blancas}(u, 12345)$, además del hecho de que tardaríamos demasiado tiempo en calcularlo, incluso empleando una computadora especializada? Justifique su respuesta.

Problema 9.57. El juego de tres en raya tridimensional se juega en un «tablero» $4 \times 4 \times 4$. Tal como en el tres en raya convencional, los jugadores X y O van marcando cuadrados alternativamente. El ganador es el primero que alinea cuatro de sus propias marcas en cualquier dirección, bien sea paralela a un lado del tablero o en una diagonal. Hay 76 maneras de ganar. Diseñar una función de evaluación para situaciones de este juego, y utilizarla para escribir un programa que juegue contra un oponente humano.

9.10 REFERENCIAS Y TEXTOS MÁS AVANZADOS

Existe un buen número de libros acerca de algoritmos para grafos o problemas de combinatoria que suelen plantearse en términos de grafos. Mencionaremos por orden cronológico a Christofides (1975), Lawler (1976), Reingold, Nievergelt y Deo (1977), Gondran y Minoux (1979), Even (1989), Papadimitriou y Steiglitz (1982), Tarjan (1983) y Melhorn (1984b). La noción matemática de grafo se trata extensamente en Berge (1958, 1970).

Berlekamp, Conway y Guy (1982) dan más versiones del juego de Nim que las que interesarán a la mayoría. El juego del problema 9.7 es una variante del juego de Grundy, que también se discute en Berlekamp, Conway y Guy (1982). El libro de Nilsson (1971) es una fuente inagotable de

ideas acerca de grafos y juegos, de la técnica de minimax y de la poda alfa-beta. En Good (1968) aparecen algunos algoritmos para jugar al ajedrez. Una grata descripción de la primera vez que un programa de computadora derrotó a un campeón mundial de *backgammon* se da en Deyong (1977). Para una descripción más técnica de esta hazaña, consúltese Berliner (1980). En 1994, Gari Kasparov, el campeón del mundo de ajedrez, fue derrotado por una microcomputadora Pentium. En el momento de escribir estas líneas, los seres humanos siguen invictos en el juego.

Una solución del problema 9.11 se da en Robson (1973). Para aprender más acerca del precondicionamiento, véase Brassard y Bratley (1988). Se pueden hallar muchos algoritmos basados en el recorrido en profundidad en Tarjan (1972), Hopcroft y Tarjan (1973), y Aho, Hopcroft y Ullman (1974, 1983). El problema 9.24 está resuelto en Hopcroft y Tarjan (1974). Véase también Rosenthal y Goldner (1977) para un algoritmo eficiente que, dado un grafo no dirigido que sea conexo pero no biconexo, busca un conjunto mínimo de aristas que se pueden añadir para hacer que el grafo sea biconexo.

El problema que implica multiplicar por 3 y dividir por 2 nos recuerda el problema de Collatz: véase el problema E16 en Guy (1981). La vuelta atrás se describe en Golomb y Baumert (1965), y se dan técnicas para analizar su eficiencia en Knuth (1975). El problema de las ocho reinas fue inventado por Bezzel en 1848; véase la descripción de Ball (1967). Irving (1984) da un algoritmo de vuelta atrás especialmente eficiente para hallar todas las soluciones del problema de las ocho reinas. El problema se puede resolver en tiempo lineal mediante un algoritmo de divide y vencerás debido a Abramson y Yung (1989), siempre que nos conformemos con una única solución. Volveremos a este problema en el Capítulo 10. El problema tridimensional de n^2 reinas mencionado en el problema 9.49 fue mencionado por McCarty (1978) y resuelto por Allison, Yee y McGaughey (1989): no hay soluciones para $n < 11$ pero existen 264 soluciones para $n = 11$.

La ramificación y poda se explica en Lawler y Wood (1966). El problema de la asignación es muy conocido en investigación operativa: véase Hillier y Lieberman (1967). Para un ejemplo de resolución del problema de la mochila empleando vuelta atrás, véase Hu (1981). El problema del viajante de comercio se resuelve empleando ramificación y poda en Bellmore y Nemhauser (1968).

Algoritmos probabilistas¹

10.1 INTRODUCCIÓN

Imagine que es usted la heroína de un cuento de hadas. Hay un tesoro oculto en un lugar que se describe en un mapa que no consigue descifrar. Se las ha arreglado para reducir la búsqueda a dos posibles escondites, que sin embargo distan bastante entre sí. Si estuviera en uno u otro de estos dos lugares, sabría de inmediato si es o no el correcto. Se necesitan cinco días para ir a cualquiera de los dos lugares, o para viajar desde uno hasta otro. Y el problema se complica más, porque hay un dragón que visita el tesoro todas las noches, y se lleva una parte a un inaccesible cubil en las montañas. Piensa usted que se necesitarán cuatro días de cálculo para resolver el misterio del mapa, y por tanto para saber con certeza dónde está oculto el tesoro, pero si se pone en marcha ya no podrá utilizar una computadora. Un elfo le ofrece mostrarle la forma de descifrar el mapa a cambio del equivalente del tesoro que se lleva el dragón en tres noches. ¿Debe de aceptar la oferta del elfo?

Evidentemente, es preferible dar el valor de tres noches de tesoro al elfo frente a permitir que el dragón se dedique al pillaje cuatro noches más. Sin embargo, si está dispuesta a correr un riesgo calculado, puede hacerlo mejor. Supongamos que es x el valor del tesoro que queda hoy, y que y es el valor del tesoro que se lleva el dragón todas las noches. Supongamos además que es $x > 9y$. Recordando que necesitará cinco días para alcanzar el escondite, puede esperar volver a casa con $x - 9y$ si espera cuatro días para acabar de descifrar el mapa. Si acepta la oferta del elfo, puede partir inmediatamente y volver con $x - 5y$, de lo cual $3y$ servirá para pagar al elfo; entonces le quedará $x - 8y$. Una estrategia mejor consiste en tirar una moneda al aire para decidir cuál de los dos escondites va a visitar primero, viajando hasta el otro si se ha equivocado. Esto le da una oportunidad de entre dos de volver a casa con $x - 5y$, y una oportu-

nidad de entre dos de volver a casa con $x - 10y$. Por tanto, el beneficio esperado es $x - 7,5y$.

Esta fábula se puede traducir al contexto de la algoritmia en la forma siguiente: cuando un algoritmo se enfrenta a una decisión, a veces es preferible seguir un curso de acción aleatorio, en lugar de invertir tiempo en determinar cuál de las alternativas es la mejor. Esta situación se produce cuando el tiempo necesario para determinar la opción óptima es prohibitivo en comparación con el tiempo que se ahorra de análisis en el caso medio al tomar esta decisión óptima.

La característica fundamental de los algoritmos probabilistas es que un mismo algoritmo se puede comportar de forma distinta cuando se aplica dos veces a un mismo caso. Su tiempo de ejecución, e incluso el resultado obtenido, pueden variar considerablemente entre usos consecutivos. Esto se puede explotar de muchas maneras. Por ejemplo, no se permite que un algoritmo determinista pierda el control (bucle infinito, división por cero) porque si hace esto en un caso dado, entonces nunca se podrá resolver ese caso con ese algoritmo. Por contraste, este comportamiento es admisible para un algoritmo probabilista siempre y cuando se produzca con una probabilidad razonablemente pequeña en cualquier caso dado: si el algoritmo se atasca, basta reiniciarlo para ese mismo caso y dispondremos de una nueva oportunidad de éxito. Otra ventaja de este enfoque es que si hay más de una respuesta correcta, se pueden obtener varias diferentes ejecutando el algoritmo probabilista más de una vez; un algoritmo determinista siempre llega a la misma respuesta, aunque por supuesto se puede programar para buscar varias.

Otra consecuencia del hecho de que los algoritmos probabilistas se pueden comportar de forma distinta cuando se ejecutan dos veces con las mismas entradas es que algunas veces les permitiremos que produzcan resultados erróneos. Supuesto que esto suceda con probabilidad suficientemente pequeña en cualquier caso dado, basta invocar el algoritmo varias veces sobre el caso deseado para llegar a una confianza arbitrariamente grande de que se ha obtenido la respuesta correcta. Por contraste, un algoritmo determinista que da respuestas erróneas con algunas entradas es inadmisibles para la mayoría de las aplicaciones, porque *siempre* fallará en esas entradas.

El análisis de algoritmos probabilistas suele ser complicado, y requiere estar familiarizado con resultados de probabilidad y estadística que van más allá de los presentados en la Sección 1.7.4. Por esta razón, en este capítulo se citan varios resultados sin demostración.

10.2 PROBABILISTA NO IMPLICA INCIERTO

Hay muchos algoritmos probabilistas que dan respuestas probabilistas, esto es, respuestas que no son necesariamente exactas. Hay aplicaciones críticas para las cuales no se pueden admitir esas respuestas inciertas. Sin embargo, es frecuente que la probabilidad de error se pueda hacer descender por debajo de la de un

error del equipo físico durante el significativamente mayor tiempo que el necesario para calcular la respuesta de forma determinista. Por tanto, paradójicamente, ¿puede suceder que la respuesta incierta dada por un algoritmo probabilista no sólo se obtenga más deprisa, sino que además podamos confiar más en ella que en la respuesta «garantizada» que se obtiene mediante ningún algoritmo determinista! Además, hay problemas para los cuales no se conoce ningún algoritmo (determinista o probabilista) que pueda dar la respuesta con certeza en una cantidad razonable de tiempo (incluso ignorando posibles errores del equipo físico), y sin embargo hay algoritmos probabilistas que pueden resolver rápidamente el problema si se admite una pequeña probabilidad de error. Tal es el caso, por ejemplo, si se desea determinar si un número de 1.000 dígitos es primo o compuesto.

Hay dos categorías de algoritmos probabilistas que no garantizan la corrección del resultado. Los algoritmos *numéricos* producen un *intervalo de confianza* de la forma «con una probabilidad del 90%, la respuesta correcta es 59 más o menos 3». Cuanto más tiempo se conceda a estos algoritmos numéricos, más preciso es el intervalo. Las encuestas de opinión ofrecen un ejemplo familiar de este tipo de respuesta. Además ilustran que un algoritmo probabilista puede ser mucho más eficiente, aunque menos preciso, que el correspondiente algoritmo determinista (una elección general). Los algoritmos numéricos son útiles si nos basta una aproximación de la respuesta correcta. En contraste, los denominados algoritmos de *Monte Carlo* dan la respuesta exacta con una elevada probabilidad, aunque a veces proporcionan una respuesta incorrecta. En general, no se puede saber si la respuesta proporcionada por un algoritmo de Monte Carlo es correcta, pero se puede reducir arbitrariamente la probabilidad de error dando más tiempo al algoritmo. (Por razones históricas, algunos autores utilizan el término «Monte Carlo» para cualquier algoritmo probabilista, y en particular para los que nosotros denominamos numéricos.)

Sin embargo, no todos los algoritmos probabilistas dan respuestas probabilísticas. Con frecuencia las opciones probabilistas se utilizan solamente como guía para llevar más rápidamente el algoritmo hacia la solución deseada, en cuyo caso la respuesta obtenida siempre es correcta. Además, puede ocurrir que la corrección de toda presunta solución pueda ser verificada eficientemente. Si resulta ser incorrecta, el algoritmo probabilista debería admitir que ha fracasado, en lugar de responder incorrectamente. Los algoritmos probabilistas que nunca dan una respuesta incorrecta se denominan de *Las Vegas*. Una vez más la posibilidad de que el algoritmo pueda admitir un fallo no es dramática: basta seguir aplicando el algoritmo a la entrada deseada hasta que tenga éxito.

Quizá resulte humorístico, pero ilustraremos la diferencia entre los distintos tipos de algoritmos probabilistas por la forma en que responderían a la pregunta ¿cuándo descubrió América Cristóbal Colón? Si lo invocásemos cinco veces, un algoritmo numérico podría responder:

Entre 1490 y 1500
 Entre 1485 y 1495
 Entre 1491 y 1501
 Entre 1480 y 1490
 Entre 1489 y 1499

Si le damos más tiempo al algoritmo, podemos reducir la probabilidad de un intervalo falso (parece ser un 20% en nuestra muestra, puesto que la cuarta respuesta no era correcta) o bien podemos reducir la amplitud de los intervalos (11 en nuestro ejemplo), o las dos cosas. Cuando se le hace la misma pregunta diez veces a un algoritmo de Monte Carlo, podría responder

1492, 1492, 1492, 1491, 1492, 1492, 357 a.C., 1492, 1492, 1492

Una vez más, tenemos una tasa de error del 20% en la respuesta, que se podría hacer más pequeña dando más tiempo al algoritmo. Obsérvese que en algunas ocasiones las respuestas incorrectas están cerca de la respuesta correcta, y a veces son completamente erróneas. Por último, un algoritmo de Las Vegas invocado 10 veces podría responder

1492, 1492, ¡Perdón!, 1492, 1492, 1492, 1492, ¡Perdón!, 1492

El algoritmo nunca da una respuesta incorrecta, pero de vez en cuando no responde.

10.3 TIEMPO ESPERADO FRENTE A TIEMPO PROMEDIO

Haremos una distinción importante entre las palabras «promedio» y «esperado». El tiempo *promedio* de un algoritmo determinista se describió en la Sección 2.4. Se refiere al tiempo promedio requerido por el algoritmo cuando se consideran igualmente probables todos los ejemplares posibles de un tamaño dado. Tal como se vio, el análisis del caso promedio puede inducir a confusión si la realidad es que algunos ejemplares son más probables que otros. Por contraste, el tiempo *esperado* de un algoritmo probabilista se define para cada ejemplar individual: se refiere al tiempo promedio que se necesitaría para resolver ese mismo ejemplar una y otra vez. Ésta es una noción robusta, que no depende de la distribución de probabilidades de los ejemplares que haya que resolver, porque las posibilidades implicadas quedan bajo control directo del algoritmo.

Resulta significativo hablar del tiempo esperado promedio y del tiempo esperado en el caso peor de un algoritmo probabilista. Este último, por ejemplo, se refiere al tiempo esperado que requiere el peor caso posible de un tamaño dado, y *no* al tiempo requerido si lamentablemente se tomasen las peores opciones probabilistas. Los algoritmos de Las Vegas pueden ser más eficientes que los algoritmos deterministas, pero sólo con respecto al tiempo esperado: siempre es posible que la mala fortuna obligue a un algoritmo de Las Vegas a seguir el camino más largo para llegar a una solución. Sin embargo, el comportamiento esperado de los algoritmos de Las Vegas puede ser mucho mejor que el de los algoritmos deterministas.

Por ejemplo, en la Sección 7.5 vimos un algoritmo que puede hallar la mediana de un vector en un tiempo lineal en el caso peor. En la Sección 10.7.2 estudiaremos un algoritmo de Las Vegas para hallar la mediana cuyo tiempo esperado en el caso peor también es lineal con respecto al tamaño del caso. Su rendimiento esperado es considerablemente mejor que el del algoritmo de la Sección 7.5 *en todos y cada uno de los casos*. El único precio que hay que pagar cuando se utiliza este algoritmo de Las Vegas es la improbable posibilidad de que en algunas ocasiones requiera un tiempo excesivo, independientemente del caso en cuestión, y esto no se deberá a otra cosa que a la mala suerte.

Se puede emplear un enfoque parecido para transformar *quicksort* en un algoritmo que ordena n elementos en un tiempo esperado en el caso peor que se encuentra en $O(n \log n)$, mientras que el algoritmo que vimos en la Sección 7.4.2 requiere un tiempo que está en $\Omega(n^2)$ cuando los elementos ya están ordenados. De forma similar, la *dispersión universal* ofrece acceso a una tabla de símbolos en un tiempo esperado constante en el caso peor, mientras que la dispersión clásica resulta desastrosa en el caso peor; véase la Sección 10.7.3. La ventaja principal de hacer aleatorio un algoritmo determinista que ofrezca un buen comportamiento medio a pesar de un caso peor muy malo es hacerlo menos vulnerable a una distribución de probabilidad inesperada de los casos que intente resolver alguna aplicación; véase el final de la Sección 2.4 y la Sección 10.7.

10.4 GENERACIÓN DE NÚMEROS PSEUDOALEATORIOS

A lo largo de este capítulo, suponemos que está disponible un generador de números aleatorios que se puede invocar con coste unitario. Sean a y b dos números reales tales que $a < b$. Una llamada a *uniforme*(a, b) devuelve un número real x seleccionado aleatoriamente en el intervalo $a \leq x < b$. La distribución de x es uniforme en el intervalo, y las sucesivas llamadas al generador devuelve valores independientes de x . Para generar enteros aleatorios, extendemos la notación a *uniforme*($i..j$), en donde i y j son enteros, $i \leq j$, y la llamada devuelve un entero k seleccionado aleatoriamente, uniformemente e independientemente dentro del intervalo $i \leq k \leq j$. Un caso especial útil es la función *tiramonedá*, que devuelven *cara* o *cruz*, cada una de ellas con una probabilidad del 50%. Obsérvese que *uniforme*(a, b), *uniforme*($i..j$) y *tiramonedá* son fáciles de obtener aun cuando sólo esté disponible *uniforme*(0, 1):

función *uniforme*(a, b)

devolver $a + (b - a) \times \text{uniforme}(0, 1)$

función *uniforme*($i..j$)

devolver $\lfloor \text{uniforme}(i, j + 1) \rfloor$

función *tiramonedá*

si *uniforme*(0..1) = 0 **entonces devolver** *cara*
sino devolver *cruz*

Los generadores verdaderamente aleatorios no suelen estar disponibles en la práctica. Además, no es realista esperar que $\text{uniforme}(0, 1)$ tome un valor *real* arbitrario entre 0 y 1. En la mayoría de las ocasiones, se emplean generadores *pseudoaleatorios*: se trata de procedimientos deterministas capaces de generar largas sucesiones de valores que parecen tener las propiedades de una sucesión aleatoria. Para dar comienzo a una sucesión, proporcionamos un valor inicial denominado *semilla*. Una misma semilla da lugar siempre a la misma sucesión. Para obtener sucesiones diferentes, podemos seleccionar, por ejemplo, una semilla que dependa de la fecha o de la hora. Casi todos los lenguajes de programación contienen uno de estos generadores, aunque hay que utilizar con precaución algunas implementaciones. Empleando un buen generador pseudoaleatorio, podemos esperar que sean válidos los resultados teóricos obtenidos en este capítulo concernientes a la eficiencia y fiabilidad de diferentes algoritmos probabilistas. Sin embargo, la poco práctica hipótesis de que está disponible un generador verdaderamente aleatorio resulta crucial cuando se efectúa el análisis.

La teoría de los generadores pseudoaleatorios va más allá de los límites de este libro, pero el principio general es sencillo. La mayoría de los generadores está basada en una pareja de funciones $f: X \rightarrow X$ y $g: X \rightarrow Y$, donde X es un conjunto suficientemente grande e Y es el dominio de los valores pseudoaleatorios que haya que generar. Se impone que tanto X como Y sean finitos, lo cual significa que sólo podemos tener la esperanza de aproximar el efecto de $\text{uniforme}(0, 1)$ empleando un Y adecuadamente grande. Por otra parte, $Y = \{0, 1\}$ resulta adecuado para muchas aplicaciones: corresponde a tirar una moneda imparcial. Para generar nuestra secuencia pseudoaleatoria, sea $s \in X$ una semilla. Esta semilla define una secuencia:

$$\begin{cases} x_0 = s \\ x_i = f(x_{i-1}) \text{ para todo } i > 0 \end{cases}$$

Finalmente, la sucesión pseudoaleatoria $y_0, y_1, y_2, \dots$ se define de la forma $y_i = g(x_i)$ para todo $i \geq 0$. Esta sucesión es esencialmente periódica, con un periodo que no puede sobrepasar el número de elementos que hay en X . Sin embargo, si X es suficientemente grande y si f y g (y a veces s) se seleccionan adecuadamente, entonces el periodo puede hacerse muy grande, y la sucesión puede ser, para casi todos los efectos prácticos, indistinguible de una sucesión verdaderamente aleatoria de elementos de Y .

Daremos un ejemplo sencillo para ilustrar este principio. Sean p y q dos números primos muy grandes distintos, ambos congruentes con 3 módulo 4, y sea n su producto. Supongamos que p y q son tan grandes que no resulta factible factorizar n . (En el momento de escribir esto, se consideran suficientes 100 cifras para cada número; véase la Sección 10.7.4.) Sea X el conjunto de enteros desde 0 hasta $n-1$, y sea $Y = \{0, 1\}$. Definimos $f(x) = x^2 \bmod n$ y $g(x) = x \bmod 2$. Siempre y cuando la semilla se seleccione uniformemente al azar entre los elementos de X que son relativamente primos con respecto a n , se ha demostrado que casi siempre es imposible distinguir la sucesión generada de esta manera de una sucesión binaria verdaderamente aleatoria. Para la mayoría de los propósitos algorítmicos, pueden resultar preferibles unos generadores pseudoaleatorios más rápidos pero menos seguros. Por ejemplo,

gozan de amplia aceptación los generadores pseudoaleatorios de congruencia lineal de la forma $f(x) = ax + b \bmod n$, para valores adecuados de a , b y n .

10.5 ALGORITMOS PROBABILISTAS NUMÉRICOS

La aleatoriedad se utilizó por primera vez en algoritmia para resolver aproximadamente problemas numéricos. Ya se utilizaba en el mundo secreto de la investigación atómica durante la Segunda Guerra Mundial, momento en que se dio a esta técnica el nombre de «Monte Carlo» (término que algunos autores utilizan todavía para denotar los algoritmos probabilistas numéricos, pero que emplearemos con un significado diferente en este libro). Un ejemplo familiar de un algoritmo numérico probabilista es la simulación. Esta técnica se puede emplear, por ejemplo, para estimar la longitud media de la cola en un sistema tan complejo que no sea factible obtener soluciones en forma cerrada, o incluso respuestas numéricas por métodos deterministas. Para ciertos problemas de la vida real, el cálculo de una solución exacta no es posible ni siquiera en principio, quizá debido a incertidumbres en los datos experimentales que haya que utilizar, o quizá porque una computadora digital sólo puede manejar valores binarios o decimales mientras que la respuesta que hay que calcular es irracional. Para otros problemas, existe una respuesta precisa pero se tardaría demasiado tiempo en calcularla exactamente.

La respuesta obtenida mediante un algoritmo probabilista numérico siempre es aproximada, pero su precisión esperada mejora a medida que crece el tiempo disponible para el algoritmo. El error suele ser inversamente proporcional a la raíz cuadrada de la cantidad de trabajo que hayamos efectuado, así que se necesita 100 veces más trabajo para obtener un dígito de precisión adicional.

10.5.1 La aguja de Buffon

En el siglo XVIII, Georges Louis Leclerc, conde de Buffon, demostró que si se arroja aleatoriamente una aguja (en una posición aleatoria, con un ángulo aleatorio, con una distribución uniforme) sobre un suelo hecho con planchas de anchura constante, si la aguja tiene una longitud que sea exactamente la mitad de la anchura de las planchas, y si la anchura de la rendija existente entre las planchas es cero, entonces la probabilidad de que una aguja caiga encima de una rendija es $1/\pi$. Este teorema puede servir para predecir aproximadamente cuántas agujas caerán encima de una rendija si dejamos caer n de ellas: la esperanza es n/π . La figura 10.1 ilustra la caída de 22 agujas, de las cuales 8 han caído encima de una rendija; nosotros habríamos esperado 7 análisis en el caso medio, puesto que es $\pi \approx 22/7$. Suponemos que las agujas se dejan caer al azar e independientemente unas de otras: para evitar interferencias, no deben superponerse, ni siquiera deben tocarse. Quizá sea mejor tirar y recoger la misma aguja varias veces.

Sin embargo, existe una interpretación del teorema de Buffon que resulta algorítmicamente más interesante. En lugar de utilizarlo para predecir el número de agujas que caerán encima de una rendija (reconocemos que esto no tiene demasiado interés), ¿por qué no utilizarlo al revés, para estimar el valor de π ? Suponga

que no dispone de una buena aproximación de π , pero conoce el teorema de Buffon. Imagine que deja caer un gran número de agujas, n , en el suelo, y que cuenta el número de agujas k que caen encima de una rendija. Ahora puede utilizar n/k como estimación de π . (Sería tener mala suerte si $k = 0$, pero esto resulta muy poco probable cuando n es grande.) Cuanto más precisa quiera que sea su estimación, más agujas tendrá que dejar caer. En principio, puede aproximarse cuanto quiera al valor de π sin más que dejar caer un número suficiente de agujas, o preferiblemente dejar caer una misma aguja un número suficiente de veces para evitar interferencias. Llevaremos a cabo un análisis más preciso más adelante en esta sección. En la práctica, esto no es un «algoritmo» útil, porque se pueden obtener aproximaciones de π mucho mejores empleando métodos deterministas. Sin embargo, esta aproximación a la «determinación experimental de π » fue muy utilizada en el siglo XIX, haciendo de éste uno de los primeros algoritmos probabilistas que se utilizaron.

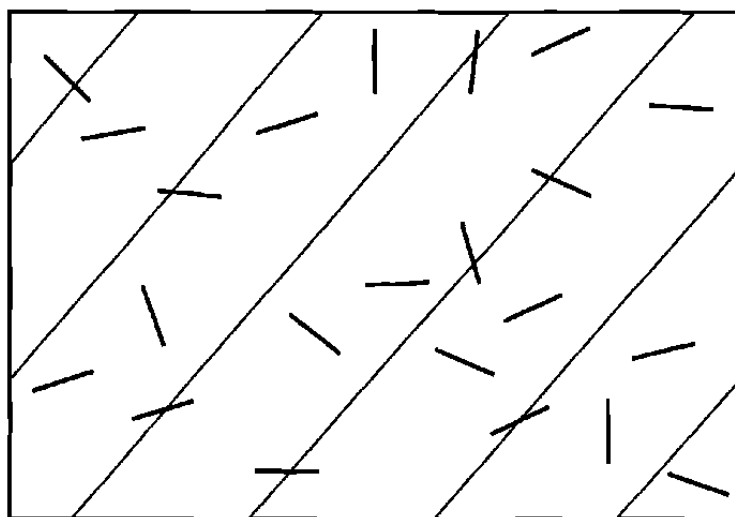


Figura 10.1 La aguja de Buffon

Hemos visto que el teorema de Buffon puede servir para predecir el número de agujas que caen encima de una rendija, y para estimar el valor de π . Una tercera aplicación posible para este teorema es estimar la anchura de las planchas (en términos de la longitud de la aguja). Recordemos que las dos aplicaciones anteriores daban por supuesto que las agujas son *precisamente* de longitud igual a la mitad de la anchura de las planchas. Si no es cierta esta suposición, entonces ya no es cierto que n/k se aproxime arbitrariamente a π a medida que vayamos tirando más y más agujas. Sean w y λ la anchura de las planchas y la longitud de las agujas, respectivamente. Buffon demostró un teorema más general: supuesto que sea $w \geq \lambda$, la probabilidad de que una aguja tirada aleatoriamente caiga encima de una rendija es $p = 2\lambda/w\pi$. En particular, $p = 1/\pi$ cuando $\lambda = w/2$, tal como hemos supuesto hasta el momento. Supongamos ahora que conocemos tanto π como la longitud de las agujas precisamente, pero que no tenemos a mano una regla para medir la

longitud de las planchas. Primero nos aseguramos de que las agujas sean más cortas que la anchura de las planchas. En este (¡improbable!) escenario, podemos estimar la anchura de las planchas con una precisión arbitraria dejando caer un número de agujas n suficientemente grande, y contando el número k de agujas que caen encima de una rendija. Nuestra estimación es, simplemente, $w \approx 2\lambda n/k\pi$. Como siempre, cuantas más agujas tiremos, más precisa será nuestra estimación.

Con una probabilidad elevada, los algoritmos para estimar el valor de π y la anchura de las planchas proporcionan un valor que converge a la respuesta correcta a medida que el número de agujas tiradas tiende a infinito (siempre y cuando las agujas tengan una longitud igual a la mitad de las planchas en el primer caso, y conociendo con precisión π y la longitud de las agujas en el segundo). La pregunta natural que se hace uno es: ¿con qué velocidad convergen estos algoritmos? Lamentablemente, son muy lentos: se necesita dejar caer la aguja 100 veces más que hasta el momento para obtener un dígito decimal adicional de precisión.

El análisis de la convergencia de los algoritmos probabilistas numéricos requiere más conocimientos de probabilidad y de estadística que los que normalmente supondríamos por parte de nuestros lectores. Sin embargo, vamos a seguir con un esbozo del enfoque básico para aquellos que tengan los conocimientos necesarios. Nos concentraremos en el análisis del algoritmo para estimar π . Consideremos un ε positivo y arbitrariamente pequeño. Asociamos una variable aleatoria X_i a cada aguja: $X_i = 1$ si la i -ésima aguja cae encima de una rendija, y $X_i = 0$ en caso contrario. Por el teorema de Buffon, $\Pr[X_i = 1] = 1/\pi$ para cada i . La esperanza y varianza de X_i son $E(X_i) = 1/\pi$ y $\text{Var}(X_i) = \frac{1}{\pi}(1 - \frac{1}{\pi})$, respectivamente. Ahora, supongamos que X representa la variable aleatoria correspondiente a nuestra estimación de $1/\pi$ después de dejar caer n agujas: $X = \sum_{i=1}^n X_i / n$. Para cualquier entero k entre 0 y n ,

$$\Pr[X = k/n] = \binom{n}{k} \left(\frac{1}{\pi}\right)^k \left(1 - \frac{1}{\pi}\right)^{n-k}.$$

Por ejemplo, cuando $n = 22$, $\Pr[X = \frac{7}{22}] \approx 18\%$ y $\Pr[X = \frac{6}{22} \leq X \leq \frac{8}{22}]$ es un poquito superior al 50%. Esta variable aleatoria tiene una esperanza $E(X) = 1/\pi$ y una varianza $\text{Var}(X) = \frac{1}{\pi}(1 - \frac{1}{\pi})/n$. Por el Teorema Central del Límite, la distribución de X es casi normal siempre y cuando n sea suficientemente grande; véase la Sección 1.7.4. Las tablas de la distribución normal nos dicen que es

$$\Pr[|X - E(X)| < 1,645 \sqrt{\text{Var}(X)}] \approx 90\%.$$

Utilizando nuestros valores para $E(X)$ y para $\text{Var}(X)$, inferimos que $\Pr[|X - \frac{1}{\pi}| < \varepsilon] \geq 90\%$, cuando

$$\varepsilon > 1,645 \sqrt{\frac{1}{\pi}(1 - \frac{1}{\pi})/n}$$

y por tanto cuando $n > 0,5881/\varepsilon^2$. Por tanto, basta con dejar caer al menos $0,5881/\varepsilon^2$ agujas para obtener un error absoluto en nuestra estimación de $1/\pi$ que no sobre-

pase ε nueve veces de cada diez. Esta dependencia de $1/\varepsilon^2$ explica la razón por la cual un dígito adicional de precisión —que significa que ε tiene que ser 10 veces menor— requiere 100 veces más agujas. Si no estamos satisfechos con un intervalo de confianza que sólo resulta fiable nueve veces de cada diez, se puede utilizar otra entrada de la tabla de distribución normal. Por ejemplo, el hecho de que

$$\Pr [|X - E(X)| < 2,576 \sqrt{\text{Var}(x)}] \approx 99\%$$

nos dice que $\Pr [|X - \frac{1}{\pi}| < \varepsilon] \approx 99$ supuesto que, $\varepsilon > 2,576 \sqrt{\frac{1}{\pi} (1 - \frac{1}{\pi}) / n}$ lo cual se satisface cuando $n > 1,440/\varepsilon^2$. Por tanto, una reducción de diez veces en la probabilidad de error resulta poco generosa en comparación con un aumento de una cifra de precisión. En general, una vez que hayamos decidido cuántas agujas vamos a dejar caer, existe una compensación entre el número de dígitos que obtenemos y la probabilidad de que esos dígitos sean correctos.

Por supuesto, nos interesa hallar un intervalo de confianza para nuestra estimación de π , y no de $1/\pi$. Un cálculo sencillo muestra que si $|X - \frac{1}{\pi}| < \varepsilon$ entonces $|\frac{1}{X} - \pi| < \frac{\varepsilon\pi^2}{1 - \varepsilon\pi}$. Y además $9,8 \varepsilon < \frac{\varepsilon\pi^2}{1 - \varepsilon\pi} < 10\varepsilon$ cuando $\varepsilon < 0,00415$, lo cual significa intuitivamente que un mismo número de agujas proporcionará una cifra decimal menos respecto a π que respecto a $1/\pi$, aun cuando el error relativo sea esencialmente el mismo puesto que π es aproximadamente 10 veces mayor que $1/\pi$. Resumiendo todo esto, suponiendo que necesitamos un error absoluto en nuestra estimación de π que no sobrepase $\varepsilon < 0,0415$ al menos el 99% de las veces, basta con dejar caer $144/\varepsilon^2$ agujas. Bastan unas pocas agujas menos en el límite de ε pequeño. ¡Esto es casi un millón y medio de agujas aunque nos conformemos con que nuestra estimación diste menos de 0,01 respecto al valor exacto de π en el 99% de las ocasiones! Nunca dijimos que esta aproximación fuera práctica, ¿verdad?

Para resumir, el usuario del algoritmo debería proporcionar dos parámetros: la fiabilidad deseada ρ , y la precisión ε . A partir de estos parámetros, con ayuda de una tabla de distribución normal, el algoritmo determina el número n de agujas que es necesario dejar caer para que la estimación resultante esté entre $\pi - \varepsilon$ y $\pi + \varepsilon$ con una probabilidad ρ como mínimo. Por ejemplo, el algoritmo seleccionaría un n próximo a un millón y medio si el usuario indicase $\rho = 99\%$ y $\varepsilon = 0,01$. Por último, el algoritmo deja caer n agujas al suelo, cuenta el número k de agujas que caen encima de una rendija, y proclama: «El valor de π está probablemente entre $\frac{n}{k} - \varepsilon$ y $\frac{n}{k} + \varepsilon$ ». Esta respuesta será correcta una fracción ρ de las veces si se repite el experimento varias veces con los mismos valores de ρ y de ε .

Queda una sutileza si se desea implementar un algoritmo similar para estimar un valor que no se conozca de antemano: ¡necesitamos el valor de π para nuestro análisis de convergencia! ¿Cómo podemos determinar el número requerido de agujas, n , como función únicamente de ρ y de ε ? Una solución es ser muy conservador en nuestra estimación de la varianza de las variables aleatorias implicadas. La varianza de X_i es $p(1-p)$ cuando $X_i = 1$ con probabilidad p , y $X_i = 0$ en caso contrario, lo cual es casi un cuarto (el caso peor es aquél en el que $p = 1/2$). Si determinamos n como si la varianza de X_i fuese un cuarto, entonces dejaremos caer más agujas que las estrictamente necesarias, pero la precisión deseada se obtendrá con una fiabilidad mayor que la re-

querida. Entonces el algoritmo puede estimar la fiabilidad real obtenida empleando una tabla de la distribución t de Student. Los detalles se pueden encontrar en cualquier texto estandar de estadística. Una solución alternativa es utilizar una pequeña muestra de tamaño arbitrario para obtener una primera estimación del valor requerido, utilizar esto para hacer una estimación mejor del tamaño de muestra necesario, y así sucesivamente. Una vez más, los detalles están en cualquier texto estándar.

10.5.2 Integración numérica

Esto nos lleva al más conocido de los algoritmos probabilistas numéricos: la integración de Monte Carlo, cuyo nombre es poco afortunado porque *no* es un ejemplo de algoritmo de Monte Carlo según la terminología empleada en este libro. Recordemos que si $f: \mathbb{R} \rightarrow \mathbb{R}^{\geq 0}$ es una función continua y si $a \leq b$, entonces el área de la superficie limitada por la curva $y = f(x)$, el eje x y las líneas verticales $x = a$ y $x = b$ es

$$I = \int_a^b f(x) dx$$

Consideremos ahora un rectángulo de anchura $b - a$ y de altura $I/(b - a)$ situado tal como indica la figura 10.2. El área del rectángulo también es I . Dado que el rectángulo y la superficie bajo la curva tienen la misma área, tienen que tener la misma altura media. Concluimos que la altura media de la curva entre a y b es $I/(b - a)$. Si consideramos que los puntos que están bajo el eje X tienen altura negativa, entonces esta interpretación es válida más generalmente para funciones continuas $f: \mathbb{R} \rightarrow \mathbb{R}$.

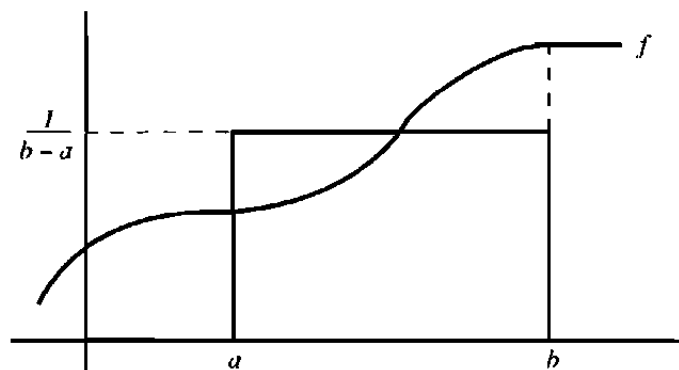


Figura 10.2 Integración numérica

Después de leer la sección anterior, un algoritmo probabilista para estimar la integral debería ocurrírseles inmediatamente: estimar la altura media de la curva por muestreo aleatorio y multiplicar el resultado por $b-a$.

```

función intMC( $f, n, a, b$ )
   $\text{suma} \leftarrow 0$ 
  para  $i \leftarrow 1$  hasta  $n$  hacer
     $x \leftarrow \text{uniforme}(a, b)$ 
     $\text{suma} \leftarrow \text{suma} + f(x)$ 
  devolver  $(b-a) \times (\text{suma}/n)$ 

```

Un análisis similar al de la aguja de Buffon muestra que la varianza de la estimación calculada mediante este algoritmo es inversamente proporcional al número de puntos de la muestra, y que la distribución de la estimación es aproximadamente normal cuando n es muy grande. Por tanto, el error esperado en la estimación es de nuevo inversamente proporcional a $\sqrt{n}$, así que se necesita 100 veces más trabajo para obtener un dígito adicional de precisión.

El algoritmo *intMC* es apenas más práctico que el método de Buffon para estimar π ; véase el problema 10.3. En general, se puede obtener una estimación mejor de la integral mediante métodos deterministas. Quizá el más sencillo es de espíritu similar a *intMC*, salvo que estima la altura media por muestreo determinista a intervalos regulares:

```
función intDET( $f, n, a, b$ )
    suma  $\leftarrow 0$ 
    delta  $\leftarrow (b - a) / n$ 
     $x \leftarrow a + \text{delta} / 2$ 
    para  $i \leftarrow 1$  hasta  $n$  hacer
        suma  $\leftarrow \text{suma} + f(x)$ 
         $x \leftarrow x + \text{delta}$ 
    devolver suma  $\times \text{delta}$ 
```

En general, este algoritmo determinista necesita muchas menos iteraciones que la integración de Monte Carlo para obtener un grado de precisión comparable. Esto es típico de la mayoría de las funciones que podamos desear integrar. Sin embargo, a todo algoritmo determinista de integración, incluso al más sofisticado, le corresponden funciones continuas diseñadas expresamente para engañar al algoritmo. Considérese por ejemplo la función $f(x) = \sin^2((100!) \pi x)$. Toda llamada a *intDET*($f, n, 0, 1$) con $1 \leq n \leq 100$ devuelve el valor 0, aun cuando el valor real de esta integral es $1/2$. Ninguna función puede hacerle esta jugada al algoritmo de Monte Carlo, aunque existe una probabilidad extremadamente pequeña de que el algoritmo pudiera cometer un error similar aun cuando f sea una función completamente normal.

Una razón mejor para utilizar la integración de Monte Carlo surge en la práctica cuando se tiene que evaluar una integral múltiple. Si se generaliza un algoritmo de integración determinista que utilice algún método sistemático para muestrear la función, adecuándolo a varias dimensiones, entonces el número de puntos de muestra necesarios para lograr una precisión dada crece de manera exponencial con la dimensión de la integral que haya que evaluar. Si se necesitan 100 puntos para evaluar una integral simple, entonces es probable que sea necesario utilizar los 10.000 puntos de una retícula 100×100 para alcanzar la misma precisión cuando se evalúe una integral doble; se necesitará un millón de puntos para una integral triple, y así sucesivamente. Por otra parte, empleando la integración de Monte Carlo, la dimensión de la integral suele tener poco efecto sobre la precisión obtenida, aunque es probable que la cantidad de trabajo para cada iteración aumente ligeramente con la dimensión. En la práctica, la integración de Monte Car-

lo se utiliza para evaluar integrales de cuatro o más dimensiones, porque no hay ninguna otra técnica sencilla que sea competitiva. Sin embargo, hay algoritmos mejores, que son más complicados. Por ejemplo, se puede mejorar la precisión de la respuesta empleando técnicas híbridas que son parcialmente sistemáticas y parcialmente probabilistas. Si la dimensión es fija, puede ser preferible seleccionar sistemáticamente los puntos de tal modo que estén igualmente espaciados y sea razonable su número; esta técnica se conoce con el nombre de *cuasi* Monte Carlo.

10.5.3 Conteo probabilista

¿Hasta dónde se puede contar con un registro de n bits? Empleando la notación binaria ordinaria, se puede contar desde 0 hasta $2^n - 1$. ¿Se puede ir más allá? Claramente, el registro no puede suponer más de 2^n valores distintos, puesto que ése es el número de formas distintas de n cifras binarias. Sin embargo, podríamos ir más allá de 2^n si estuviéramos dispuestos a descartar valores intermedios. Por ejemplo, se podría contar el doble si estuviéramos dispuestos a contar solamente los valores pares. Más exactamente, sea c nuestro registro de n bits. Algunas veces utilizaremos también c para denotar el entero que representa el registro en binario. Deseamos implementar dos procedimientos *iniciar*(c) y *pulsar*(c), y una función *contar*(c) tal que una llamada a *contar*(c) devuelva el número de llamadas a *pulsar*(c) habidas desde la última llamada a *iniciar*(c). En otras palabras, *iniciar* pone el contador a cero, *pulsar* le suma 1 y *contar* pide su valor actual. Los algoritmos deben ser capaces de mantener un número arbitrariamente grande de contadores $c_1, c_2, \dots$, y no se admiten los efectos secundarios: no se puede pasar información entre llamadas, salvo a través del registro que se transmite como parámetro explícito.

Sugeríamos más arriba que se podían saltar algunos valores para llegar más lejos. Sin embargo, esto carece de sentido si insistimos en utilizar una estrategia *determinista* de recuento. Dado que no se admiten efectos secundarios, no hay forma de que *pulsar* pueda sumar 1 al contador en llamadas alternativas: el comportamiento de *pulsar*(c) debe de estar completamente determinado a partir del valor actual de c . Si existe un valor de c tal que *pulsar*(c) deja intacto a c , entonces el contador se detendrá en ese punto hasta que se llame a *iniciar*(c). (Esto es razonable cuando el contador ha alcanzado su límite superior.) Dado que c sólo puede admitir 2^n valores diferentes, el contador se ve obligado a tomar un valor que ya haya tenido anteriormente al cabo de un máximo de 2^n llamadas a *pulsar*(c). Por tanto, es imposible contar de manera determinista más de 2^n sucesos en un registro de n bits.

Si relajamos el requisito consistente en que *contar* debe de devolver el número exacto de llamadas a *pulsar* desde el último *iniciar*, existe una estrategia probabilista evidente para contar dos veces más. El registro es un contador binario normal, que se pone a cero mediante una llamada a *iniciar*. Cada vez que se llama a *pulsar*, lanzamos al aire una moneda imparcial. Si sale cara, sumamos 1 al registro. Si sale cruz, sin embargo, no hacemos nada. Cuando se llama a *contar*, devuelve el doble del valor almacenado en el registro. Después de una llamada a *iniciar*, es fácil demostrar que el valor esperado que proporciona *contar* después de t llamadas

a pulsar es precisamente t , aun cuando t sea impar. Se puede demostrar que la varianza es razonablemente pequeña, así que la estimación proporcionada por *contar* es bastante precisa en la mayoría de las ocasiones; véase el problema 10.5.

No esperamos impresionarle con el párrafo anterior. Contar dos veces más significa simplemente llegar hasta $2^{n+1} - 2$, lo cual se podría haber logrado de forma determinista si hubiera estado disponible un solo *bit* más dentro del registro. Ahora mostraremos que las estrategias de conteo probabilista pueden llegar *exponencialmente* más allá: desde 0 hasta $2^{2^c-1} - 1$. Por tanto, bastan 8 *bits* para contar ¡más de 5×10^{76} sucesos! La idea consiste en mantener en el registro una estimación del logaritmo del número real de llamadas a pulsar. Más exactamente, *contar*(c) devuelve $2^c - 1$. Le restamos 1 para que se pueda representar un conteo de 0; además *contar*(0) = 0, tal como debe de ser, porque *iniciar*(c) pone c a 0. Queda por ver la forma en que se implementa *pulsar*.

Supongamos que $2^c - 1$ es una buena estimación del número de llamadas a pulsar desde la última iniciación. Después de un adicional a pulsar, una buena estimación del número total de pulsos sería 2^c , pero esto no se puede representar así con nuestra estrategia. Para soslayar el problema, sumaremos 1 a c con alguna probabilidad p que ya se determinará, y dejaremos c intacto en caso contrario. De esta manera, nuestra estimación del número de llamadas a pulsar pasa a ser $2^{c+1} - 1$ con probabilidad p , mientras que tiene una probabilidad $1 - p$ la posibilidad de que el contador persista con su valor $2^c - 1$. El valor esperado que devuelve *contar*(c) después de esta llamada a pulsar es por tanto

$$(2^{c+1} - 1)p + (2^c - 1)(1 - p) = 2^c + 2^c p - 1$$

y se ve que basta hacer $p = 2^{-c}$ para obtener 2^c , que es el valor esperado que se deseaba para *contar*(c). (Este razonamiento no es riguroso; hacer el problema 10.6 para una demostración de que el valor esperado que devuelve *contar* es el correcto.) Para resumir, obtenemos los algoritmos siguientes:

procedimiento *iniciar*(c)

$c \leftarrow 0$

procedimiento *pulsar*(c)

para $1 \leftarrow 1$ hasta n hacer

 si *tiramoneda* = cara entonces terminar

$c \leftarrow c + 1$

 {La probabilidad de desbordamiento es demasiado pequeña para que valga la pena comprobarlo }

función *contar*(c)

 devolver $2^c - 1$

Al guardar el logaritmo del conteo actual en c , se obtiene una ventaja adicional. Vimos en las secciones anteriores que los algoritmos probabilistas numéricos mantenían bajo control el error absoluto. Cuando se implementa un contador cuyo

rango alcance es tan grande como 5×10^6 , es más probable que se necesite mantener bajo control el error *relativo*. Afortunadamente, un pequeño error absoluto en el logaritmo del número de llamadas a pulsar se traduce en un pequeños error relativo del contador real. En particular, se garantiza que *contar* dará la respuesta exacta cuando el número de pulsos habidos desde la inicialización es 0 o 1. Se puede probar por inducción matemática que la varianza de *contar* al cabo de m llamadas a pulsar es $m(m-1)/2$. Esto no está demasiado bien, porque significa que la desviación estándar es alrededor del 70% del valor real de conteo: este proceso distinguiría un millón de mil millones, pero no sería fiable para distinguir un millón de dos millones.

Aunque hay aplicaciones para las cuales una carencia de precisión semejante resulta aceptable, ninguna aplicación real puede tener necesidad de contar hasta 5×10^6 sucesos. Afortunadamente, la varianza del conteo estimado se puede mejorar a costa de no llegar tan lejos al contar. Para esto, basta llevar la cuenta en el registro del logaritmo en una base que sea menor que dos. Por ejemplo, podríamos utilizar

$$\text{contar}(c) = \left[(1 + \varepsilon)^c - 1 \right] / \varepsilon \quad (10.1)$$

para ε pequeño. La división por ε es para mantener la propiedad deseable de que *contar* siempre da la respuesta exacta al cabo de 0 o 1 llamadas a pulsar. Con $\varepsilon = 1/30$, esto nos permite contar hasta más de 125.000 sucesos en un registro de 8 *bits*, con un error relativo menor que el 24% durante el 95% del tiempo. Por supuesto, la probabilidad de que *pulsar* vaya a incrementar c cuando sea invocado ya no debe de ser 2^{-1} : hay que darle un valor adecuado; véase el problema 10.8.

Para un tipo de conteo probabilista completamente diferente que ahorra tiempo en lugar de espacio, asegúrese de hacer el problema 10.12. En ciertas condiciones, este algoritmo puede contar aproximadamente el número de elementos que hay en un conjunto en un tiempo que esta en el orden de la raíz cuadrada de ese número.

10.6 ALGORITMOS DE MONTE CARLO

Existen problemas para los cuales no se conoce ningún algoritmo eficiente (probabilista o determinista) que sea capaz de obtener una solución correcta en todas las ocasiones. Los algoritmos de *Monte Carlo* cometen ocasionalmente un error, pero encuentran la solución correcta con una probabilidad alta sea cual sea el caso considerado. Esto es una afirmación más fuerte que decir que funciona correctamente en la mayoría de los casos, fallando tan sólo de vez en cuando en algunos casos especiales: no debe haber ningún caso en el cual la probabilidad de error sea elevada. Sin embargo, no suele darse ningún aviso cuando el algoritmo comete un error.

Sea p un número real tal que $0 < p < 1$. Decimos que un algoritmo de Monte Carlo es p -correcto si devuelve una respuesta correcta con probabilidad no menor que p , sea cual sea el caso considerado. En algunos casos, permitiremos que p dependa del tamaño del caso, pero nunca del caso en sí. La característica más importante de los algoritmos de Monte Carlo es que suele ser posible reducir arbitrariamente la probabilidad de error a costa de un ligero aumento del tiempo de cálculo. Llamamos a esto *amplificación de la ventaja estocástica*. Estudiaremos este fenómeno general en la Sección 10.6.4, pero primero consideramos dos ejemplos. El primero tiene escaso interés práctico, pero tiene la ventaja de su sencillez. Lo utilizamos para presentar las nociones clave. El segundo, por otra parte, tiene un indudable interés práctico.

10.6.1 Verificación de la multiplicación de matrices

Consideremos tres matrices $n \times n$, A , B y C . Se sospecha que $C = AB$. ¿Cómo se puede verificar si esto es o no verdad? La aproximación evidente consiste en multiplicar A por B , y comparar el resultado con C . Empleando un sencillo algoritmo de multiplicación matricial, esto requiere un tiempo que está en $\Theta(n^3)$. El algoritmo de Strassen es más rápido, y existen algoritmos de multiplicación aún más rápidos para valores muy grandes de n , pero el algoritmo asintóticamente más rápido conocido hasta el momento sigue requiriendo un tiempo que está en $\Omega(n^{2.37})$; véase la Sección 7.6. ¿Podremos hacerlo mejor si estamos dispuestos a admitir una pequeña probabilidad de error? Sorprendentemente, la respuesta es que para cualquier $\varepsilon > 0$ prefijado, basta un tiempo que está en $O(n^2)$ para resolver este problema con una probabilidad de error que sea como mucho ε . La constante oculta en la notación O , sin embargo, depende de ε .

Supongamos que $C \neq AB$. Sea $D = AB - C$. Por hipótesis, D no es idénticamente nulo. Sea i un entero tal que la i -ésima fila de D contiene al menos un elemento no nula. Consideremos cualquier subconjunto $S \subseteq \{1, 2, \dots, n\}$. Supongamos que $\sum_S(D)$ representa el vector de longitud n que se obtiene sumando componente a componente las filas de D indexadas por los elementos de S ; una suma vacía es igual a cero. Por ejemplo, $\sum_{\{1, 3, 4\}}(D)$ se obtiene sumando las filas primera, tercera y cuarta de D . Sea S' el mismo subconjunto que S , salvo que $i \in S'$ si y sólo si $i \notin S$. Dado que la i -ésima fila de D no es idénticamente 0, $\sum_S(D)$ y $\sum_{S'}(D)$ no pueden ser nulos simultáneamente. Supongamos ahora que seleccionamos S de forma aleatoria: para todo j entre 1 y n , decidimos mediante una moneda imparcial si se incluye o no j en S . Dado que es tan probable que $i \in S$ como que $i \notin S$, la probabilidad de que $\sum_S(D) \neq 0$ es como mínimo de un medio.

Por otra parte, $\sum_S(D)$ siempre es cero si $C = AB$, puesto que en este caso $D = 0$. Esto sugiere comprobar si $C = AB$ o no calculando $\sum_S(D)$ para un subconjunto S seleccionado aleatoriamente, y comparando con 0 el resultado. Pero ¿cómo se puede hacer esto eficientemente sin calcular primero D (y por tanto AB)?

La solución consiste en considerar $\sum_s(D)$ como una multiplicación de matrices independiente. Sea X un vector binario de longitud n tal que $X_i = 1$ si $j \in S$ y $X_i = 0$ en caso contrario. Es sencillo verificar que $\sum_s(D) = XD$, siempre que pensemos en X como en una matriz $1 \times n$. De esta manera, nuestra comprobación se corresponde con verificar si $X(AB - C)$ es o no cero, o equivalentemente si $XAB = XC$, para algún vector binario X seleccionado arbitrariamente. A primera vista, parece que calcular XAB debiera de requerir todavía más tiempo que calcular AB , pero esto no es así. Recordemos de la Sección 8.6 que el tiempo necesario para calcular una multiplicación matricial encadenada depende crucialmente del orden en que se multiplican las matrices. En este caso, sólo se necesita un tiempo que está en $\Theta(n^2)$ para calcular XAB en la forma $(XA)B$, porque éste es el tiempo que se necesita para multiplicar la matriz $1 \times n$ X por la matriz $n \times n$ A , y porque el resultado XC es de nuevo una matriz $1 \times n$ que podemos multiplicar por B en un tiempo que está en $\Theta(n^2)$. Esto sugiere el algoritmo siguiente:

```
función Freivalds( $A, B, C, n$ )
  para  $j \leftarrow 1$  hasta  $n$  hacer  $X_j \leftarrow \text{uniforme}(0..1)$ 
  si  $(XA)B = XC$  entonces devolver verdadero
  sino devolver falso
```

A la vista de lo dicho anteriormente, sabemos que este algoritmo devuelve una respuesta correcta con una probabilidad del 50% en todos los casos, por tanto, es $1/2$ -correcto por definición. Sin embargo no es p -correcto para todo p menor que $1/2$, porque su probabilidad de error es exactamente $1/2$ cuando C difiere de AB precisamente en una fila: se devuelve una respuesta incorrecta si y sólo si $X_i = 0$ donde la fila i supone la diferencia entre C y AB . ¿Es esto realmente interesante? Una probabilidad de error del 50% es intolerable en la práctica, ¿verdad? Hay algo peor: un «algoritmo» más sencillo para alcanzar esta probabilidad de error sería arrojar al aire una moneda imparcial y ¡devolver verdadero o falso dependiendo del resultado, sin siquiera examinar las tres matrices en cuestión! La observación clave es que siempre que $\text{Freivalds}(A, B, C, n)$ devuelve el valor falso, uno puede estar seguro de que la respuesta es correcta: la existencia de un vector X tal que XAB difiere de XC nos permite concluir con certeza que $AB \neq C$. Sólo cuando el algoritmo devuelve verdadero estamos inseguros a la hora de creer su afirmación.

Consideremos por ejemplo las matrices 3×3 siguientes:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \quad B = \begin{pmatrix} 3 & 1 & 4 \\ 1 & 5 & 9 \\ 2 & 6 & 5 \end{pmatrix} \quad C = \begin{pmatrix} 11 & 29 & 37 \\ 29 & 65 & 91 \\ 47 & 99 & 45 \end{pmatrix}$$

Una llamada a $\text{Freivalds}(A, B, C, 3)$ podría seleccionar $X = (1, 1, 0)$ en cuyo caso XA se obtiene sumando las filas primera y segunda de A , así que $XA = (5, 7, 9)$. Prosiguiendo, calculamos $(XA)B = (40, 94, 128)$. Este resultado se compara con $XC = (40,$

94, 128), que se obtiene sumando la primera y segunda fila de C . Con esta selección de X , *Freivalds* devuelve el valor *verdadero* puesto que es $XAB = XC$. Otra llamada a *Freivalds* podría seleccionar $X = (0, 1, 1)$. En este caso, los cálculos son

$$XA = (11, 13, 15), (XA)B = (76, 166, 236), XC = (76, 164, 136)$$

y *Freivalds* proporciona el valor *falso*. Tenemos más suerte: el hecho de que $XAB \neq XC$ es una demostración concluyente de que $AB \neq C$.

Esto sugiere aplicar *Freivalds*(A, B, C, n) varias veces sobre el mismo caso, empleando tiradas independientes de la moneda en cada ocasión. Si se obtiene la respuesta *falso*, incluso en una sola ocasión, podemos concluir que $AB \neq C$ independientemente del número de veces que se obtenga la respuesta *verdadero*. Consideremos ahora el siguiente algoritmo, en el cual k es un nuevo parámetro.

función *RepetirFreivalds*(A, B, C, n, k)
 para $i \leftarrow 1$ **hasta** k **hacer**
 si *Freivalds*(A, B, C, n) = *falso* **entonces devolver** *falso*
 devolver *verdadero*

Nos interesa la probabilidad de error de este nuevo algoritmo. Consideremos dos casos. Si realmente $AB = C$, entonces toda llamada a *Freivalds* devuelve necesariamente el valor *verdadero*, puesto que no existe la posibilidad de seleccionar aleatoriamente un X tal que $XAB \neq XC$, y por tanto *RepetirFreivalds* devuelve *verdadero* después de pasar k veces por el bucle. En este caso, la probabilidad de error es cero. Por otra parte, si $C \neq AB$, entonces la probabilidad de que una llamada a *Freivalds* devuelva (incorrectamente) *verdadero* es como máximo $1/2$. Estas probabilidades se multiplican, porque los lanzamientos de la moneda son independientes entre llamadas consecutivas. Por tanto, la probabilidad de que k llamadas consecutivas devuelvan todas ellas pueden devolver un valor erróneo de 2^{-k} como máximo. Pero ésta es la única forma en que *RepetirFreivalds* devuelve una respuesta errónea. Concluimos que nuestro nuevo algoritmo $(1 - 2^{-k})$ -correcto. Cuando $k = 10$, esto es mejor que 99,9%-correcto. Repetirlo 20 veces lleva la probabilidad de error a menos de una entre un millón. Este decrecimiento espectacularmente rápido de la probabilidad de error es típico de los algoritmos de Monte Carlo que resuelven problemas de decisión —esto es, problemas para los cuales la respuesta es o bien *verdadero* o bien *falso*— siempre y cuando una de las respuestas, en caso de ser obtenida, tenga garantizada su veracidad.

Alternativamente, se puede dar a los algoritmos de Monte Carlo una cota superior explícita para la probabilidad de error admisible:

función *FreivaldsEpsilon*(A, B, C, n, ϵ)
 $k \leftarrow \lceil \lg 1/\epsilon \rceil$
 devolver *RepetirFreivalds*(A, B, C, n, k)

Una ventaja de esta versión del algoritmo es que podemos analizar su tiempo de ejecución como función simultánea del tamaño del caso y de la probabilidad de error. En este caso, el algoritmo requiere claramente un tiempo que está en

$\Theta(n^2 \log^4 n)$.

Los algoritmos de esta sección tienen un interés práctico limitado, porque se necesitan $3n^2$ multiplicaciones escalares para calcular XAB y XC , en comparación con las n^3 que se necesitan para calcular AB por el método directo. Si insistimos en una probabilidad de error que no sea mayor que una entre un millón, y si de hecho $C = AB$, entonces las 20 ejecuciones necesarias de *Freivalds* efectúan $60n^2$ multiplicaciones escalares, lo cual no es una mejora con respecto a n^3 a no ser que n sea mayor que 60. Sin embargo, esta aproximación podría resultar de utilidad si fuera necesario verificar productos matriciales muy grandes.

10.6.2 Comprobación de primalidad²

Es posible que el algoritmo de Monte Carlo más famoso sea el que decide si un entero impar dado es o no primo. No se conoce ningún algoritmo para resolver este problema con certeza en un tiempo razonable cuando el número que hay que estudiar no tiene más que unos pocos centenares de dígitos decimales. Comprobar la *primalidad* de números grandes es algo más que un *divertimento* matemático: ya vimos en la Sección 7.8 que es crucial para la criptología moderna.

La historia de la comprobación probabilista de *primalidad* tiene sus raíces en Pierre de Fermat, el padre de la Teoría de los Números moderna. En 1640 enunció el teorema siguiente, que a veces se denomina teorema menor de Fermat.

Teorema 10.6.1 (Fermat). Sea n un primo. Entonces $a^{n-1} \bmod n = 1$ para cualquier entero a tal que $1 \leq a \leq n-1$.

Por ejemplo, sean $n = 7$ y $a = 5$. Se tiene que es

$$a^{n-1} = 5^6 = 15625 = 2232 \times 7 + 1$$

y por tanto es $a^{n-1} \bmod n = 1$, tal como debía de suceder. Considérese ahora la versión contrapositiva del teorema de Fermat: si n y a son enteros tales que $1 \leq a \leq n-1$ y si $a^{n-1} \bmod n \neq 1$ entonces n no es primo. Un ejemplo anecdótico lo proporcionó el mismo Fermat. En su búsqueda de una fórmula que sólo produjese números primos, formuló la hipótesis de que $F_n = 2^n + 1$ es primo para todo n . Obsérvese que $F_0 = 3$, $F_1 = 5$, $F_2 = 17$, $F_3 = 257$ y $F_4 = 65537$ son todos ellos números primos. Desafortunadamente, $F_5 = 4294967297$ era un número demasiado grande para que Fermat pudiera intentar demostrar su primalidad. Tuvo que pasar casi un siglo para que Euler factorizase $F_5 = 641 \times 6700417$, probando así la falsedad de la conjetura de Fermat. Irónicamente, Fermat podría haber utilizado su propio teorema para alcanzar la misma conclusión. Estaba dentro de sus posibilidades inventar un algoritmo eficiente de exponenciación modular parecido a *expomod*, y utilizarlo

para calcular $3^{F_5-1} \bmod F_5 = 3029026160 \neq 1$; véase la Sección 7.8. Aunque es tedioso, este cálculo no requiere nada más que 32 cuadrados modulares sucesivos, puesto que $F_5 - 1 = 2^{32}$. A partir de la versión contrapositiva de su propio teorema, Fermat podría haber sabido que F_5 no es primo.

Esto sugiere el siguiente algoritmo probabilista para la comprobación de *primalidad*. Suponemos que $n \geq 2$:

función *Fermat*(n)

$a \leftarrow \text{uniforme}(1..n-1)$

si *expomod*($a, n-1, n$) = 1 **entonces devolver** *verdadero*

sino devolver *falso*

Visto lo anterior, sabemos que n es compuesto cuando una llamada a *Fermat*(n) devuelva el valor *falso* porque la existencia de un a entre 1 y $n-1$ tal que $a^{n-1} \bmod n \neq 1$ basta para demostrar esto, por el teorema menor de Fermat. Es notable que cuando este algoritmo, indica que n es compuesto, no ofrece ni una sola indicación acerca de sus divisores. Teniendo en cuenta los conocimientos algorítmicos actuales, la factorización es mucho más difícil que la comprobación de primalidad. Recuerde que la supuesta diferencia de dificultad entre estos problemas es crucial para el sistema criptográfico RSA descrito en la Sección 7.8: la facilidad de comprobar la primalidad es necesaria para su implementación, mientras que la dificultad de la factorización es condición indispensable para su seguridad.

¿Qué se puede decir, sin embargo, si *Fermat*(n) devuelve el valor *verdadero*? Para concluir que n es primo, necesitaríamos el recíproco y el contrapositivo del teorema de Fermat. Este resultado diría que $a^{n-1} \bmod n$ nunca es igual a 1 cuando n es compuesto y $1 \leq a \leq n-1$. Desafortunadamente, esto no es cierto, porque $1^{n-1} \bmod n = 1$ para todo $n \geq 2$. Además, $(n-1)^{n-1} \bmod n = 1$ para todo $n \geq 3$, porque $(n-1)^2 \bmod n = 1$. El menor contraejemplo no trivial del recíproco es que $4^{14} \bmod 15 = 1$ a pesar de que 15 sea compuesto. Un entero a tal que $2 \leq a \leq n-2$ y que $a^{n-1} \bmod n = 1$ se denomina *testigo falso de primalidad* para n si de hecho n es compuesto. Con tal de modificar la comprobación de Fermat para seleccionar aleatoriamente a entre 2 y $n-2$, sólo fallará en los números compuestos cuando seleccionemos un testigo falso.

Las buenas noticias son que los testigos falsos son bastante escasos. Aunque sólo 5 de entre los 332 impares compuestos menores que 1.000 presumen de no tener falsos testigos, más de la mitad tiene sólo dos testigos falsos y menos del 16% de ellos tiene más de 15. Además, sólo existen 4.490 testigos falsos para todos estos números compuestos juntos. Comparemos esta cantidad con 178.878, que es el número total de testigos candidatos que existe para el mismo conjunto de números. La probabilidad media de error para la comprobación de Fermat aplicada a números impares compuestos menores que 1.000 es menor que 3,3%. Resulta aún menor si consideramos números más grandes.

Las malas noticias son que hay números compuestos que admiten una proporción significativa de falsos testigos. El caso peor entre los números compuestos menores que 1.000 es 561, que admite 318 testigos falsos: son más de la mitad de los testigos po-

tenciales. Un caso más convincente es el que plantea un número de 15 dígitos: *Fermat*(651693055693681) devuelve *verdadero* con una probabilidad mayor que 99,9965% ¡a pesar de que este número es compuesto! Con más generalidad, para cualquier $\delta > 0$ existe un número infinito de números compuestos para los cuales la comprobación de Fermat descubre que son compuestos con una probabilidad menor que δ . En otras palabras, la comprobación de Fermat no es p -correcta para ningún $p > 0$ que fijemos. Consiguientemente, la probabilidad de error no se puede reducir por debajo de un ϵ arbitrariamente pequeño por repetición de la comprobación de Fermat durante un cierto número prefijado de veces, según la técnica que veíamos en la sección anterior.

Afortunadamente, una pequeña modificación de la comprobación de Fermat resuelve esta dificultad. Sea n un entero impar mayor que 4, y sean s y t enteros tales que $n - 1 = 2^t$, donde t es impar. Obsérvese que $s > 0$ puesto que $n - 1$ es par. Sea $B(n)$ el conjunto formado por estos enteros, definido en la forma siguiente: $a \in B(n)$ si y sólo si $2 \leq a \leq n - 2$ y

◇ $a^t \bmod n = 1$, o bien

◇ existe un entero i , $0 \leq i < s$, tal que $a^{2^i} \bmod n = n - 1$

Esto puede parecer complicado, pero es sencillo implementarlo eficientemente. Siempre y cuando n sea impar y $2 \leq a \leq n - 2$, una llamada a *pruebaB*(a, n) devuelve el valor *verdadero* si y sólo si $a \in B(n)$.

función *pruebaB*(a, n)

$s \leftarrow 0; t \leftarrow n - 1$

repetir

$s \leftarrow s + 1; t \leftarrow t \div 2;$

hasta que $t \bmod 2 = 1$

$x \leftarrow \text{expomod}(a, t, n)$

si $x = 1$ **o** $x = n - 1$ **entonces devolver** *verdadero*

para $i \leftarrow 1$ **hasta** $s - 1$ **hacer**

$x \leftarrow x^2 \bmod n$

si $x = n - 1$ **entonces devolver** *verdadero*

devolver *falso*

Por ejemplo, veamos si 158 pertenece a $B(289)$. Hacemos $s = 5$ y $t = 9$ porque $n - 1 = 288 = 2^5 \times 9$. Entonces calculamos

$$x = a^t \bmod n = 158^9 \bmod 289 = 131$$

No concluye aquí la prueba, porque 131 no es 1 es $n - 1 = 288$. A continuación, elevamos x al cuadrado (módulo n) hasta 4 veces ($s - 1 = 4$) para ver si obtenemos 288.

$$a^{2^t} \bmod n = 131^2 \bmod 289 = 110$$

$$a^{2^{2^t}} \bmod n = 110^2 \bmod 289 = 251$$

$$a^{2^{2^{2^t}}} \bmod n = 251^2 \bmod 289 = 288$$

Llegados aquí, nos detenemos porque hemos encontrado que es $a^{2^i} \bmod n = n - 1$ para $i=3 < s$, y concluimos que $158 \in B(298)$.

Una extensión del teorema de Fermat demuestra que $a \in B(n)$ para todo $2 \leq a \leq n-2$ cuando n es primo. Por otra parte, se dice que n es un *pseudoprimo fuerte* con base a y que a es un *testigo falso fuerte* de primalidad para n siempre que $n > 4$ es un número impar compuesto y $a \in B(n)$. Por ejemplo, ya hemos visto que 158 es un testigo falso fuerte de primalidad para 289 puesto que $289 = 17^2$ es compuesto. Esto proporciona una comprobación mejor que la de Fermat porque los testigos falsos fuertes son automáticamente testigos falsos respecto a la prueba de Fermat, pero no a la inversa. De hecho, los testigos falsos fuertes son mucho más escasos que los testigos falsos. Por ejemplo, 4 es un testigo falso de primalidad para 15, pero no es un testigo falso fuerte porque $4^7 \bmod 15 = 4$. También vimos que el 561 admite 318 falsos testigos, pero sólo 8 de éstos son testigos falsos fuertes. Considerando todos los impares compuestos menores que 1.000, la probabilidad media de seleccionar aleatoriamente un testigo falso fuerte es menor que el 1%; más del 72% de estos números compuestos ni siquiera admite un único falso testigo fuerte. La superioridad de la prueba fuerte queda ilustrada de la mejor manera posible por el hecho consistente en que todos los enteros compuestos entre 5 y 10^{13} no son pseudoprimos fuertes en al menos una de las bases 2, 3, 5, 7 o 61. En otras palabras, cinco llamadas a *pruebaB* bastan para decidir de *forma determinista* acerca de la primalidad de cualquier entero hasta 10^{13} . Hay algo más importante: a diferencia de la comprobación de Fermat, se garantiza que la proporción de testigos falsos fuertes es pequeña *para todo número impar compuesto*. Más precisamente, tenemos el teorema siguiente.

Teorema 10.6.2. Consideremos un número impar arbitrario $n > 4$:

◇ Si n es primo, entonces $B(n) = \{a \mid 2 \leq a \leq n-2\}$

◇ Si n es compuesto, entonces $|B(n)| \leq (n-9)/4$

Consiguientemente, *pruebaB*(a, n) siempre devuelve el valor *verdadero* cuando n es un primo mayor que 4, y $2 \leq a \leq n-2$, mientras que devuelve el valor *falso* con una probabilidad de más de $3/4$ cuando n es un entero impar compuesto mayor que 4 y a se selecciona aleatoriamente entre 2 y $n-2$. En otras palabras, el siguiente algoritmo de Monte Carlo es $3/4$ -correcto para la comprobación de primalidad; se conoce con el nombre de comprobación de Miller-Rabin:

función *MillRab*(n)

{Este algoritmo sólo debe invocarse si $n > 4$ es impar}

$a \leftarrow \text{uniforme}(2..n-2)$

devolver *pruebaB*(a, n)

Dado que se garantiza que la respuesta *falso* sea correcta, la técnica que vimos en la sección anterior es aplicable para reducir muy rápidamente la probabilidad de error.

función *RepetirMillRab*(n, k)
 {Este algoritmo sólo debe invocarse si $n > 4$ es impar}
para $i \leftarrow 1$ **hasta** k **hacer**
 si *MillRab*(n) = falso **entonces devolver** falso
devolver verdadero

Este algoritmo siempre devuelve la respuesta correcta cuando $n > 4$ es primo. Cuando $n > 4$ es un impar compuesto, cada llamada a *MillRab* tiene como mucho una probabilidad de $1/4$ de llegar a un testigo falso fuerte, devolviendo *verdadero* de manera errónea. Dado que la única manera de que *RepetirMillRab* devuelva el valor *verdadero* en este caso es encontrar aleatoriamente k testigos falsos fuertes seguidos, esto sucede con una probabilidad que es como mucho de 4^{-k} . Por ejemplo, basta con tomar $k = 10$ para reducir la probabilidad a menos de una entre un millón. En conclusión, *RepetirMillRab*(n, k) es un algoritmo de Monte Carlo $(1 - 4^{-k})$ -correcto para la comprobación de primalidad.

¿Cuánto tiempo se necesita para decidir la primalidad de n con una probabilidad de error acotada por ε ? Debemos repetir la comprobación de Miller-Rabin k veces, en donde es $4^{-k} \leq \varepsilon$, que equivale a $2^{2k} \geq 1/\varepsilon$. Esto se consigue mediante $k = \lceil \frac{1}{2} \lg \frac{1}{\varepsilon} \rceil$. Cada llamada a *MillRab* requiere una exponenciación modular, con t como exponente, y $s-1$ elevaciones al cuadrado modulares. Sabemos por la Sección 7.8 que la exponenciación requiere un número de multiplicaciones y elevaciones al cuadrado modulares que está en $O(\log t)$. Contando las elevaciones al cuadrado como multiplicaciones, y dado que es $\lg n > \lg(n-1) = \lg(2^s) t = s + \lg t$, el tiempo requerido por una llamada a *MillRab* está dominado por un número de multiplicaciones modulares que está en $O(\log n)$. Si se efectúan éstas según el algoritmo clásico, cada una de ellas requiere un tiempo que está en $O(\log^2 n)$ porque reducimos módulo n después de cada multiplicación.

Resumiendo, el tiempo total necesario para decidir acerca de la primalidad de n con una probabilidad de error acotada por ε está por tanto en $O(\log^3 n \log \frac{1}{\varepsilon})$. Esto es enteramente razonable en la práctica para números de mil dígitos y una probabilidad de error menor que 10^{-100} .

10.6.3 ¿Puede un número ser probablemente primo?

Supongamos que se aplica diez veces la comprobación de Miller-Rabin a algún entero impar n , y que se obtiene la respuesta *verdadero* en las diez ocasiones. ¿Qué podemos deducir? Hemos visto que si n era compuesto, esto podría suceder con una probabilidad máxima de $4^{-10} = 2^{-20}$, que es menor que uno entre un millón. Puede tentarnos concluir que « n es primo, salvo por una probabilidad de error dada por 2^{-20} ». Sin embargo, esta afirmación carece de sentido: ¡todo entero mayor que 1 es o bien primo o bien compuesto! Lo mejor que podemos decir es «creo que n es primo; en caso contrario he observado un fenómeno cuya probabilidad de ocurrir era menor que uno entre un millón». Además, hasta esta afirmación prudencial debe considerarse con cautela si los testigos de la comprobación se han obtenido con un generador pseudoaleatorio, en lugar de ser realmente alea-

torios: quizá n sea compuesto y no hemos conseguido hallar un testigo, pero no por tener mala suerte sino porque el generador pseudoaleatorio era defectuoso; véase la Sección 10.4. Sin embargo, este fallo es improbable si se emplea un buen generador, y no nos preocuparemos más acerca de esta posibilidad.

Dado que es imposible llegar a estar seguros de que un número dado es primo mediante este algoritmo de Monte Carlo —sólo se pueden certificar los números compuestos— y dado que los primos son relevantes a efectos criptográficos, quizá se pregunte si merece la pena arriesgarse a obtener un «primo falso» para ahorrar tiempo de cálculo. Si su vida dependiera de la primalidad de un número dado, ¿no invertiría el tiempo necesario para certificar su primalidad empleando el mejor método determinista? Digamos que se tiene que decidir entre ejecutar *RepetirMillRab*(n , 150) o emplear algún método más sofisticado, que requiere un tiempo significativamente mayor pero garantiza una respuesta correcta. ¿Cuál de estas aproximaciones tiene menos probabilidades de inducirnos a error? Tal como se discute en la Sección 10.2, el método sofisticado tiene más probabilidades de ser incorrecto, a pesar de sus resultados, porque la probabilidad de que se produzca un error de *hardware* no detectado durante su (más extenso) periodo de cálculo puede ser mayor que $4^{-150} \approx 10^{-100}$. Además, uno puede tener más confianza en que su implementación de la comprobación de Miller-Rabin esté libre de errores que en el algoritmo más complicado. Así llegamos a la paradójica conclusión consistente en que un primo probable, sea cual fuere el significado dado a esta frase, ¡puede ser más fiable que uno cuya primalidad se haya «demostrado» de forma determinista! Obsérvese sin embargo que existen métodos probabilistas que producen primos aleatorios *certificados* en un tiempo menor que el que se necesita para hallar primos probabilistas mediante el uso repetido de la prueba de Miller-Rabin. Dado que se garantiza que estos métodos van a proporcionar un número primo, pertenecen a la clase de algoritmos de Las Vegas.

La importancia de interpretar correctamente el resultado de un algoritmo de Monte Carlo se puede ilustrar muy bien si se desea *generar* un número de ℓ dígitos que sea probablemente primo, quizá con propósitos criptográficos. El algoritmo evidente consiste en seguir seleccionando enteros impares aleatorios de ℓ dígitos hasta encontrar uno que pase por un número suficiente de aplicaciones de la comprobación de Miller-Rabin. Más exactamente, considérese el siguiente algoritmo, que sólo debería de utilizarse con $\ell > 1$:

función *primoaleatorio*(ℓ , k)

repetir

{Seleccionar un n impar aleatoriamente entre $10^{\ell-1}$ y $10^{\ell} - 1$ }

$n \rightarrow 1 + 2 \times \text{uniforme}(10^{\ell-1} / 2..10^{\ell} / 2 - 1)$

hasta que *RepetirMillRab*(n , k)

devolver n

¿Qué se puede decir acerca del resultado de este algoritmo? Ya sabemos que no se puede afirmar que el número obtenido sea primo con una probabilidad de al menos $1 - 4^{-k}$. Sin embargo, tiene sentido investigar el número medio de «números primos falsos» (un eufemismo de «números compuestos») que se pro-